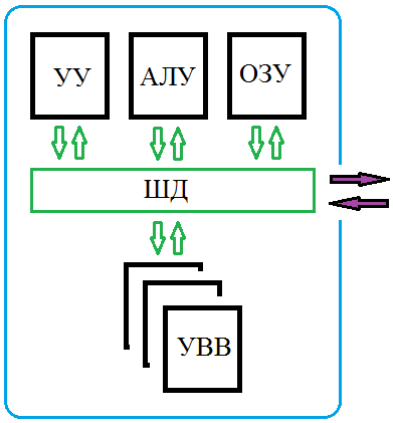
Лекция 1. Введение в параллельные вычисления.

1. Устройство “обыкновенного” компьютера.

1.1. Аппаратное устройство компьютера.

В упрощенном варианте компьютер (системный блок) иммет вид, показанный на Рис. 1.



УУ – управляющее устройство,
АЛУ – арифметико-логическое устройство,
ОЗУ – оперативно-запоминающее устройство,
УВВ – устройства ввода/вывода,
ШД – шина обмена данными.

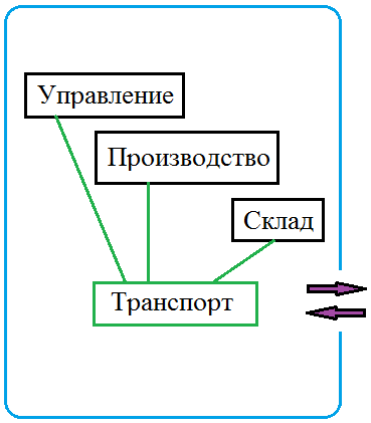


Рис. 1. Компьютер

Рис. 2. Фабрика

1.2. Программное обеспечение компьютера.

Операционная система, средства подготовки программ, специализированные системы хранения и обработки данных, средства визуализации, прикладные программы пользователей.

1.3. Узкие места и проблемы использования.

Модель: компьютер – предприятие широкого профиля по оказанию информационно-вычислительных услуг (Рис. 2). В нем: АЛУ – производственный сектор, УУ – сектор оперативного управления, ОЗУ – склад, шина – транспортный сектор, операционная система – организационный сектор, компилятор – технологический сектор.

Пиковая производительность компьютера – суммарная производительность АЛУ.

Реальная производительность компьютера – производительность компьютера при выполнении конкретных задач.

Эффективность работы компьютера – отношение его реальной производительности к пиковой.

Узкие места:

1) состав, принцип работы и временные характеристики АЛУ
2) состав, размер и временные характеристики ОЗУ
3) структура и пропускная способность шины
4) структура и эффективность работы операционной системы
5) эффективность компиляторов

2. Методы повышения производительности обыкновенного компьютера.

1) усложнение и наращивание аппаратных средств
2) модернизация программного обеспечения
3) параллельная обработка данных внутри компьютера
4) многопроцессорные вычислительные системы, суперкомпьютеры
5) вычислительные сети компьютеров и суперкомпьютеров

3. Параллельная обработка данных (пример).

Пример – Завтрак джентльмена.

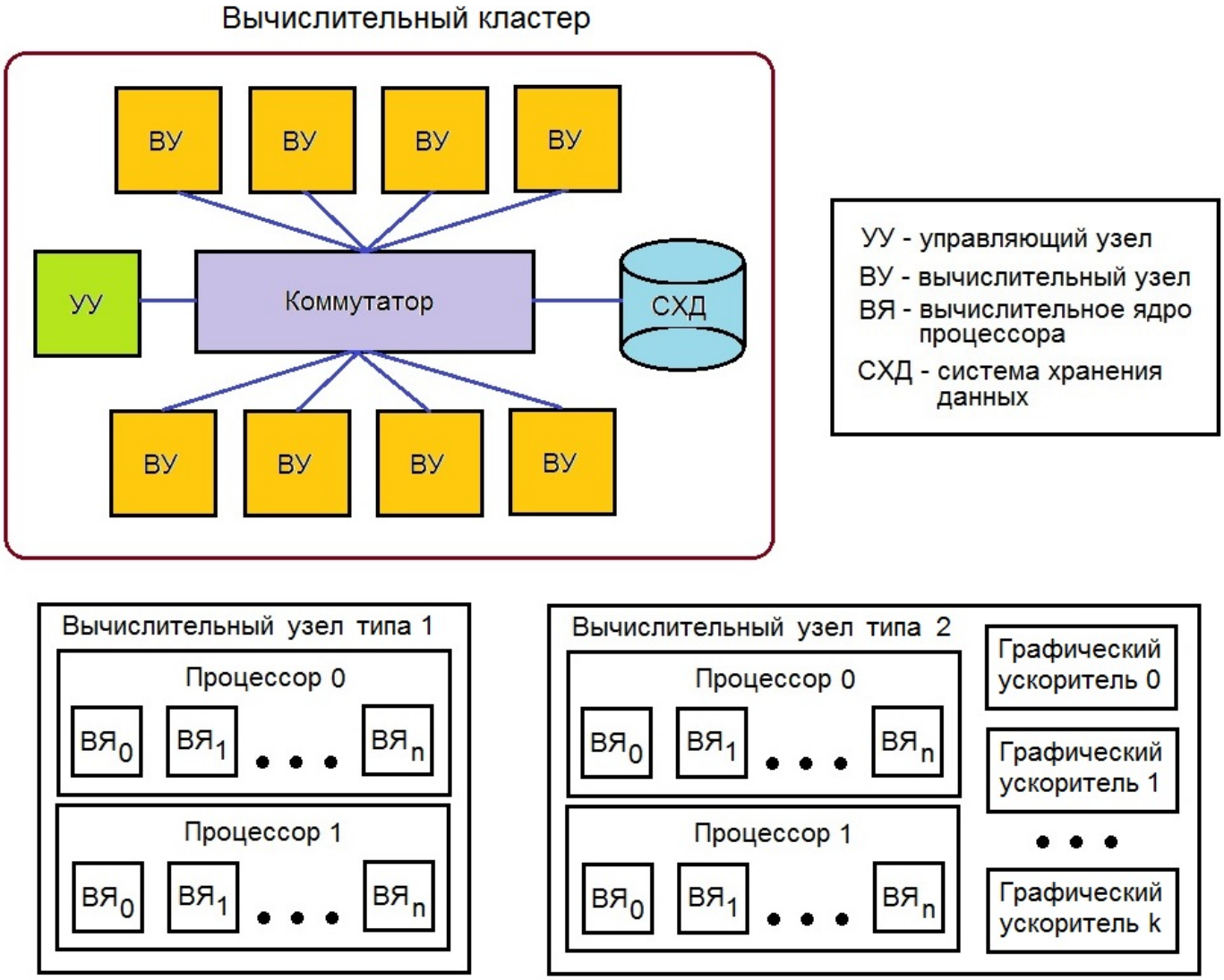
Некий джентльмен, собираясь утром на службу, был занят приготовлением завтрака и чтением газеты. Приготовление завтрака состояло из нескольких простых операций, которые однако занимали много времени. Поэтому для экономии времени джентльмен иногда просил жену, дочь и сына помочь ему. Варианты распределения работы при приготовлении завтрака и чтения газеты показаны в таблице:

№	Газета, 10 мин	Хот-дог, 7 мин	Салат, 5 мин	Кофе, 3 мин	Итоговое время, мин	Занятость участников
1	Дж.	Дж.	Дж.	Дж.	25	высокая
2	Дж.	Жена	Жена	Дж.	13	высокая
3	Дж.	Жена	Дочь	Жена	10	средняя
4	Дж.	Жена	Дочь	Сын	10	низкая

Вывод – для каждого вида деятельности необходимо использовать оптимальное количество исполнителей.

4. Многопроцессорные вычислительные системы и их применение.

МВС – вычислительная система, имеющая более одного вычислительного устройства (процессора). Типичный вариант конструкции МВС (вычислительного кластера, суперкомпьютера) показан на рис. 3:



Для чего применяются МВС:

1) Решение задач “большого вызова” (прогноз погоды на планете)
2) Решение задач реального времени (управление полетами и перевозками)
3) Сокращение времени решения задачи (моделирование технологических процессов)
4) Построение систем высокой надежности (моделирование процессов в ядерных реакторах)
5) Одновременное обслуживание многих потоков данных (система продажи авиабилетов)
6) Информационное обслуживание распределенных баз данных (параллельные поисковые системы)

5. Проблемы использования МВС.

Проблемы использования МВС напрямую связаны со следующими узкими местами:

1) архитектура и производительность узлов МВС
2) структура и пропускная способность межпроцессорных коммуникаций
3) эффективность работы операционной системы МВС
4) эффективность средств разработки и отладки параллельных программ
5) эффективность параллельных алгоритмов

Лекция 2. Методы повышения производительности вычислительных систем.

1. Наращивание аппаратных средств. Пример.

Вычислитель	год	Частота, МГц	Увели-чение	Производи-тельность	Увели-чение
EDSAC	1949	0.5	1	100 оп/с ~ 1 Flops	1
HPSuperdome	2001	770	1440	2 GFlops	$2*10^9$
Intel Itanium2	2004	1600	3200	6 GFlops	$6*10^9$
Intel Xeon	2004	3600	7200	4 GFlops	$4*10^9$
Intel Core 2 Duo	2006	3000	6000	2x4=8 GFlops	$8*10^9$
Intel Xeon 64	2007	3730	7460	6x7=42 GFlops	$4.2*10^{10}$
Intel i7-990X, X5690, E7-8800	2011	3460	6920	10x7=70 GFlops	$7*10^{10}$
Nvidia Tesla m2090, 512я	2011	1331	2662	665 GFlops*)	$6.65*10^{11}$
Nvidia Tesla K20X, 2688я	2013	732	1464	1310 GFlops*)	$1.31*10^{12}$
Intel Xeon Phi 5120P, 61я, 244п	2014	1330	2660	1010 GFlops	$1.01*10^{12}$
Nvidia Tesla TCSK40M, 2880я	2015	745	1490	1430 GFlops*)	$1.43*10^{12}$
Nvidia Tesla TCSK80M, 4992я	2016	562	1124	2900 GFlops*)	$2.9*10^{12}$
Nvidia Tesla TCSP100, 3584я	2016	1328	2656	4700 GFlops*)	$4.7*10^{12}$
Intel Xeon Phi KNL 7290X, 36пх2ях4п OmniPath (100 ГБ/с)	2016	1500	3000	3000 GFlops	$3*10^{12}$
NVidia Volta V100, 640 тензорных ядер, 5120 видеоядер (CUDA), NVLINK (300 ГБ/с), PCIE (32 ГБ/с), ОЗУ 32 Гб	2018	1455	2910	7800 GFlops*)	$7.8*10^{12}$
NVidia A100 Tensor Core, 512 тензорных ядер, 8192 видеоядер (CUDA), NVLINK2 (600 ГБ/с), PCIE (64 Гб/с), ОЗУ 40 Гб	2020	1410	2820	9700 GFlops*)	$9.7*10^{12}$
NVidia H100 PCIe Tensor Core, 456 тензорных ядер, 14592 видеоядер (CUDA), NVLINK4 (900 ГБ/с), PCIE (128 Гб/с), ОЗУ 80 Гб	2022	1125 (1755)	2250 (3510)	26000 GFlops (51000 TFlow)	$26*10^{12}$
NVidia H100 SXM Tensor Core, 528 тензорных ядер, 16896 видеоядер (CUDA), NVLINK4 (900 ГБ/с), PCIE (128 Гб/с), ОЗУ 80 Гб	2022	1125 (1755)	2250 (3510)	34000 GFlops (67000 TFlow)	$34*10^{12}$
NVIDIA H200 SXM Tensor Core GPU, 528 тензорных ядер, 16896 видеоядер (CUDA), NVLINK4 (900 ГБ/с), PCIE (128 Гб/с), ОЗУ 141 Гб	2025	1980	3960	34000 GFlops (67000 TFlow)	$34*10^{12}$

*) Двойная точность (double precision)

Частота – развитие элем. базы, производительность – развитие архитектуры.

Процессоры Эльбрус.

Советские ЭВМ: серия БЭСМ (большие электронно-счетные машины)

БЭСМ6 (1967) => интегральная БЭСМ6 (1977) => суперкомпьютер Эльбрус (1988).

Процессор Эльбрус-2000 основан на архитектуре ELBRUS (англ. *ExpLicit Basic Resources Utilization Scheduling* — «явное планирование использования основных ресурсов»).

Компьютер Эльбрус-3М при частоте 300 МГц в режиме совместимости с IA-32 обогнал 500 МГц Intel Pentium III в тестах SPEC. При работе в «родных» кодах Эльбруса процессор показал скорость, сравнимую с Pentium 4 2 ГГц. «Эльбрус-2С+» — гибридный (2 ядра «Эльбрус» + кластер из 4-х DSP ядер Мультикор) процессор, 90 нм, 2011.

- «Эльбрус-2СМ» — упрощенный вариант «Эльбрус-2С+» (без кластера DSP ядер), 90 нм, 300 МГц, к 2014.
- «Эльбрус-4С» (другое название «Эльбрус-2S») — 64 Гфлоп, 65 нм, к 2013/2014 г.
- «Эльбрус-1С+» — Экономичный микропроцессор с одним ядром Эльбрус и встроенным графическим ядром, 40 нм, к 2015 г.

- «Эльбрус-8С» — 8 ядер Эльбрус, частота >1 ГГц, производ. > 150 Гфлоп, 28 нм, к 2015 г.
- «Эльбрус-16С» — 1 Тфлоп, ≤28 нм, к 2018 г.

	Эльбрус-2С+	Эльбрус-4С	Эльбрус-8С	Эльбрус-16С
Год выпуска	2011	2014	2015-2018	2018 (план)
Тактовая частота	500 МГц	800 Мгц	1300 МГц	1500 Мгц
Разрядность	хз	32/64 бит	64 бит	64/128 бит
К-во ядер	2	4	8	8/16
Кэш первого уровня	64 Кб	128 Кб	—	—
Кэш второго уровня	1 Мб	8 Мб	4 Мб	4 Мб
Кэш третьего уровня	—	—	16 Мб	16 Мб
Поддержка ОЗУ	DDR2-800	3 x DDR3-1600	4 x DDR3-1600	4 x DDR4-2400
Техпроцесс	90 нм	65 нм	28 нм	28 нм (или 16)
Потребление энергии	25 Вт	45 Вт	75-100 Вт	60-90 Вт

Сравнение производительности

Процессор, № (Имя)	Число ядер	GFlops, с ДТ	Частота, GHz	L3 cache, MB	Техпроцесс, нм	Тип ОЗУ	ОЗУ MAX	Слотов ОЗУ
Core i7 975	4	50	3.3	8	45	DDR3-1066	24	3
Эльбрус-4С (E2S)	4	50	0.8	0	65	DDR3-1600	48	3
2x Xeon x5677	4	104	3.5	12	32	DDR3-1333	288	9
i5-2500K (Sandy Bridge)	4	118	3.3	6	32	DDR3-1333	32	2
Эльбрус-8С (P1)	8	125	1.3	16	28	DDR3-1600	64	8
i7-4960X (Ivy Bridge)	6	244	3.6	16	22	DDR3-1866	64	4
i7-6950X (Broadwell E)	10	247	3.0	26	14	DDR4-3000	128	4
Эльбрус-8С2 (P9)	8	288	1.6	16	28	DDR4-2600	2048	8
i7-5960X (Haswell)	8	350	3.5	20	22	DDR4-2400	128	4
AMD A10-7850K	12	427	3.7	0	28	DDR3-2133	64	4
Эльбрус-16С	16	750	2.0	32	16	DDR4-2600	2048	8
Xeon E7-8890 v4	24	844	2.2	60	14	DDR4-1600	3078	12
Xeon E7-8894 v4	24	921	2.4	60	14	DDR4-1600	3078	12
Эльбрус-32С	32	2000-4000	2.0	≥32	14	DDR4-3200	2048	8
Xeon E5-2699 V5	32	2500	2.1	45	14	DDR4-3200	3078	12
Xeon Phi™ Processor 7210	72	3000	1.5	36	14	DDR4-2400	3078	12

Российские процессоры Байкал-Т1 и Байкал-М.

Процессоры Байкал-Т1 предназначены больше для промышленного сегмента. Процессоры Байкал-Т1 можно использовать для маршрутизаторов, роутеров и другого телекоммуникационного оборудования, для тонких клиентов и офисной техники, для мультимедийных центров, систем ЧПУ.

Процессоры Байкал-М смогут предназначены для рабочих ПК, для промышленной автоматизации и для управления зданиями. Будет работать на 8 ядрах ARMv8-A и иметь на борту до восьми графических ядер ARM Mali-T628. Процессор Байкал-М - система на кристалле, включающая энергоэффективные процессорные ядра с архитектурой ARMv8, графическую подсистему и набор высокоскоростных интерфейсов. Байкал-М может использоваться в качестве доверенного процессора с широкими возможностями защиты данных в ряде устройств B2C и B2B сегментов. Области применения Байкал-М моноблок, автоматизированное рабочее место, графическая рабочая станция; домашний (офисный) медиа-центр; сервер и терминал видеоконференций; микросервер; NAS уровня небольшого предприятия; маршрутизатор / брандмауэр. Широкое распространение архитектуры ARMv8 (AArch64)

позволяет использовать огромное количество готового прикладного и системного программного обеспечения. Поддерживаются операционные системы Linux и Android, в том числе на уровне бинарных дистрибутивов и пакетов. Доступны драйверы многочисленных устройств, подключаемых к шинам PCIe и USB. В состав поставляемого <Байкал Электроникс> комплекта программного обеспечения входит ядро Linux в исходных текстах и скомпилированном виде, а также драйверы для встроенных в Baikal-M контроллеров.

Основные характеристики процессора Байкал-М:

- 8 ядер ARM Cortex-A57 (разрядность 64 бит);
- рабочая частота до 2 ГГц;
- аппаратная поддержка виртуализации и технологии Trust Zone;
- два 64-битных канала DDR3/DDR4-2133 с поддержкой ECC;
- кэш-память - 4 МБ (L2) + 8 МБ (L3);
- 8-ядерный графический сопроцессор Mali-T628;
- видеотракт, обеспечивающий поддержку HDMI, LVDS
- аппаратное декодирование видео;
- встроенный контроллер PCI Express поддерживает 16 линий PCIe Gen. 3;
- 2 контроллера 10-гигабитной сети Ethernet, 2 контроллера гигабитной сети Ethernet;
- контроллеры поддерживают виртуальные сети VLAN и приоритезацию трафика;
- 2 контроллера SATA 6G, обеспечивающих скорость обмена данными до 6 Гбит/с каждый;
- 2 канала USB v.3.0 и 4 канала USB v.2.0;
- энергопотребление - не более 30 Вт.

2. Параллельная обработка данных – эффективный метод повышения производительности.

Два вида параллельной обработки: параллельный и конвеерный.

Пример: сложение векторов **c** = **a** + **b** размерности 10, 5-ти тактное сложение.

T	Последовательный	T	Параллельный, 2 проц.	T	Конвеерный, длина конв.=4
0	a10…a1, b10…b1 -> +	0	a10…a1, b10…b1 -> + +	0	a10…a1, b10…b1 -> ++++
1	a1+b1	1	a1+b1, a2+b2	1	a1+b1
2	a1+b1	2	a1+b1, a2+b2	2	a2+b2, a1+b1
3	a1+b1	3	a1+b1, a2+b2	3	a3+b3, a2+b2, a1+b1
4	a1+b1	4	a1+b1, a2+b2	4	a4+b4, a3+b3, a2+b2, a1+b1
5	a1+b1 = c1	5	a1+b1=c1, a2+b2=c2	5	a4+b4, a3+b3, a2+b2, a1+b1=c1
6	a2+b2	6	a3+b3, a4+b4	6	a5+b5, a4+b4, a3+b3, a2+b2=c2
7	a2+b2	7	a3+b3, a4+b4	7	a6+b6, a5+b5, a4+b4, a3+b3=c3
8	a2+b2	8	a3+b3, a4+b4	8	a7+b7, a6+b6, a5+b5, a4+b4=c4
9	a2+b2	9	a3+b3, a4+b4	9	a8+b8, a7+b7, a6+b6, a5+b5
10	a2+b2=c2	10	a3+b3=c3, a4+b4=c4	10	a8+b8, a7+b7, a6+b6, a5+b5=c5
…	…	…	…	11	a9+b9, a8+b8, a7+b7, a6+b6=c6
46	a10+b10	21	a9+b9, a10+b10	12	a10+b10, a9+b9, a8+b8, a7+b7=c7
47	a10+b10	22	a9+b9, a10+b10	13	a10+b10, a9+b9, a8+b8=c8
48	a10+b10	23	a9+b9, a10+b10	14	a10+b10, a9+b9
49	a10+b10	24	a9+b9, a10+b10	15	a10+b10, a9+b9=c9
50	a10+b10=c10	25	a9+b9=c9, a10+b10=c10	16	a10+b10=c10
	$Q = N_d * N_t$ $P = 1 / (N_t*\tau)$		$Q = N_d * N_t / N_p$ $P = N_p / (N_t*\tau)$		$Q = N_d * N_t / L_k + L_k - 1$ $P = L_k / ((N_t+(1-1/L_k)/N_d)*\tau)$

Q – общее число тактов, N_d – кол-во обработанных данных, N_t – число тактов на операцию, L_k – длина конвеера. Возникли понятия – ***скалярные и векторные команды, скалярные, конвеерные и векторные устройства.*** **Общая формула:** $Q = N_i + N_d * N_t / L_k + L_k - 1$, N_i – кол-во тактов для инициализации векторного устройства. ***Реальная производительность*** любого устройства $P = N_d / T$, T – общее время выполнения задачи. ***Пиковая производительность*** любого устройства $P_{max} = 1/\tau$, τ - длительность такта. $T = Q*\tau \Rightarrow P = N_d / (Q*\tau)$ и P –> P_{max} с ростом N_d у конвеерных и векторных устройств!!!

3. Аппаратные решения, использующие параллелизм.

1) переход от разрядно-последовательной к разрядно-параллельной арифметике
2) переход от разрядно-последовательной к разрядно-параллельной памяти
3) параллельная работа УУ, АЛУ и УВВ за счет введения каналов ВВ (спец. процессоров)
4) совмещение арифм. действий с чтением/записью в память за счет каналов прямого доступа к памяти

5) совмещение вызова команд и данных для них с помощью каналов команд и данных
6) использование одного УУ и многих АЛУ (ФУ) – чистый параллелизм
7) применение конвеерного принципа обработки команд
8) использование многих конвеерных ФУ – параллелизм + конвеерная обработка
9) опережающий просмотр потоков команд и данных, упреждающее выполнение команд
10) применение векторных и матричных ФУ (процессоров)
11) введение векторных команд процессора
Зам.: для увеличения производительности вычислительной системы (ВС) с помощью аппаратного параллелизма необходимо увеличить ***локальность вычислений и локальность используемых данных.***

4. Увеличение интеллектуальности АПС.

Зам.: Реальная производительность зависит от степени поддержки параллелизма в аппаратно-программной среде (АПС) выч. системы. Каждая составляющая АПС вносит вклад в производительность.
С точки зрения управляющей структуры ВС сформировались два класса процессоров:
1) универсальные (центральные) процессоры;
2) специализированные процессоры.
Появились также процессоры, активно использующие параллелизм на уровне машинных команд
1) суперскалярные процессоры – обнаруживают во время выполнения задачи параллелизм в машинном коде и реализуют его – не требуется особая компиляция программ – непредсказуемость результата, динамические возможности;
2) VLIW-процессоры (Very Large Instruction Word) – выполняют длинные команды, состоящие из набора параллельных операций – требуется особая компиляция программ, четко расписывающая параллелизм – предсказуемость результата, ограниченные возможности архитектуры процессора.
Сформировались также два основных класса МВС
1) компьютеры с общей (разделяемой) памятью – **мультипроцессорные системы** – P1->M, P2->M, …
2) компьютеры с распределенной памятью – **мультикомпьютерные системы** – (P1,M1)->KC, (P2,M2)->KC,…
SMP-системы – представители 1) с одинаковыми процессорами (Symmetric Multi Processors).
Появление МВС выявило две проблемы:
а) как достичь их максимальной производительности;
б) как разработать для этого эффективное ПО.
Решение этих проблем сильно зависит от ***системы коммутации процессоров.***
Способы коммутации мультипроцессорных систем:
1) общая шина (недостаток – малое число устройств);
2) разделение памяти на модули;
3) применение матричных и каскадных коммутаторов доступа к модулям памяти
(недостаток матричных – много оборудования - n^2 устройств; у каскадных – задержка доступа из-за длинного пути - кол-во каскадов $log_2 n$ по $n/2$ коммутаторов в каждом; n – число модулей памяти и/или процессоров)
4) другие топологии коммутации: линейка, кольцо, звезда, решетки, тор, двоичный гиперкуб, клика (зам.: 3) и 4) применяются уже и в мультикомпьютерных ВС)
5) гибридные топологии коммутации
6) кластерное устройство доступа – внутри кластера по общей шине, сами кластеры – шина или коммутация др. типа
Проект NUMA - неоднородный доступ к памяти – единое адресное пространство, локальная и межкластерная шина доступа к распределенным модулям памяти: (P, M, IO, лок. шина, контроллер памяти) -> межкластерная шина
Проблема NUMA – некогерентность кэш-памяти –> ccNUMA (cache coherent NUMA)
Технологии параллельного программирования
1) спец. комментарии в языках высокого уровня, описывающие параллельные части кода (Fortran, C - OpenMP)
2) расширение языков программирования (High Performance Fortran - HPF, mpC)
3) специальные языки параллельного программирования (Occam, HOPMA)
4) библиотеки и интерфейсы, поддерживающие взаимодействие параллельных процессов (PVM, MPI, Linda)

5) параллельные предметные библиотеки (BLAS, Lapack, ScaLapack, Cray Scientific Library, HP Mathematical Library, PETsc)
6) специализированные пакеты и программные комплексы (GAMESS – квантово-химические расчеты, Fluent – газодинамические расчеты, ANSYS – механические расчеты и т.д.)

Лекция 3. Производительность системы функциональных устройств.

1. Оценка реальной производительности ВС.

Любая ВС – совокупность функциональных устройств (ФУ). ФУ бывают двух типов:

Простое ФУ – если на нем никакая другая операция не может начаться, пока не кончится предыдущая (монопольное использование всего оборудования для выполнения операции).

Конвеерное ФУ – ФУ, которое может выполнять одновременно несколько элементарных операций в режиме конвеера (обычно это линейная цепочка простых элементарных ФУ, имеющих одинаковые времена срабатывания). Элементарные ФУ – ***ступени конвеера***, их число – ***длина конвеера***. Простое ФУ = конвеерное ФУ с длиной 1. Все ФУ выполняют одну или несколько операций, скорость их работы измеряется в операциях в секунду.

Предположение 1 – время выполнения конкретной операции на конкретном ФУ фиксировано, не равно 0 и не зависит от других ФУ.

Предположение 2 – любое ФУ не может одновременно выполнять операцию и сохранять результат предыдущего срабатывания – т.е. не имеет собственной памяти. Результат предыдущего срабатывания хранится в нем до начала следующей операции, а затем пропадает.

Предположение 3 – все ФУ работают по индивидуальным командам: команда + аргументы попадают на вход ФУ. При этом не происходит тупиковых ситуаций.

*Предположение 4 – Если отрезок времени работы ФУ равен **T**, а длительность операции **τ**, то целое число операций **Q**, выполненных за время **T**, равно либо **[T/τ]**, либо **[T/τ]-1**. При больших **T** эти величины асимптотически совпадают.*

Стоимость операции ФУ – время ее реализации на ФУ (время срабатывания ФУ).

Стоимость работы ФУ – время последовательной реализации всех операций на простых ФУ с аналогичными временами срабатывания.

Загруженность ФУ (***0<=Z<=1***) – отношение стоимости реально выполненной работы S_R за фиксированный промежуток времени ***T*** к максимально возможной стоимости S_M. ***Q*** – число выполненных ФУ операций за время ***T***.

Утв. 1. $S_M = T$ для простого ФУ, $S_M = L*T$ для конвеерного ФУ с длиной конвеера L, $S_R = Q*\tau$.

Реальная производительность ВС (P_R) – количество операций, выполненных ВС в среднем за единицу времени.

Пиковая производительность ВС (P_M) – максимальное количество операций, которое может быть выполнено ВС за единицу времени при отсутствии связей между ее ФУ.

Утв. 2. $P_R = \sum_{k=1}^{m} Z_k P_{M,k}$, $P_M = \sum_{k=1}^{m} P_{M,k}$, где m – кол-во ФУ, $P_{M,k}$ и Z_k – их пиковые производительности и загруженности. Док-во:

Простое ФУ (L=1): $P_M=1/\tau$, $S_R=Q*\tau$, $S_M=T$, $Z=S_R/S_M=Q*\tau/T$, $P_R = Q/T = (Q*\tau/T) * (1/\tau) = Z * P_M$ – ч.т.д.

Конвеерное ФУ (L>1): $P_M=L/\tau$, $S_R=Q*\tau$, $S_M=L*T$, $Z=(Q*\tau)/(L*T)$, $P_R=Q/T=(Q*\tau)/(L*T) * (L/\tau) = Z * P_M$ – ч.т.д.

Аддитивность формул доказывает Утв. 2 для любого кол-ва устройств.

Для одного устройства: $P_R = Z * P_M$ => для повышения P_R надо увеличивать Z.

Для системы устройств: $Z = \sum_{k=1}^{m} \alpha_k Z_k, \quad \alpha_k = P_{M,k} / P_M \geq 0, \quad \sum_{k=1}^{m} \alpha_k = 1$ => 0<= Z <= 1.

Вывод: для увеличения загруженности системы необходимо повышать загруженность каждого ФУ.

Пример: ВС имеет два устройства - сумматор и умножитель с одинаковой пиковой производительностью. Решаем задачу C = A + B. При обычной схеме решения имеем $P_R = 0.5\ P_M$. Можно ли повысить реальную производительность? Да: C = A + 1*B => $P_R = P_M$ – но кому это нужно?

2. Ускорение и эффективность использования МВС.

Опр. 1. Ускорение - отношение $S_m = T_1/T_m$ ($0<=S_m<=m$) времен выполнения одной задачи на системе из m и одного одинаковых процессоров (ФУ). ***Эффективность*** $E_m = S_m /m$ или $E_m = S_m /m*100\%$.

Это определение годится только для систем с одинаковыми процессорами (ФУ). На неоднородных ВС с различными ФУ ***эффективность совпадает с загруженностью***, а ускорение определяется иначе:

Опр. 2. Ускорение на неоднородной ВС есть $S_m = P_R / \max_{1\leq k\leq m} P_{M,k} = \sum_{k=1}^{m} Z_k P_{M,k} / \max_{1\leq k\leq m} P_{M,k}$.

Утв. 3. В системе из m устройств с одинаковой пиковой производительностью ($P_{M,1} = ... = P_{M,m}$):

1) $Z = \frac{1}{m}\sum_{k=1}^{m} Z_k$ 2) $P_R = \sum_{k=1}^{m} P_{R,k}$ 3) $P_M = mP_{M,1}$ 4) $S_m = \sum_{k=1}^{m} Z_k$ 5) если все ФУ простые => $S_m = T_1 / T_m$.

Рассмотрим ВС из m простых ФУ. Пусть между ними установлены какие-то функциональные связи, которые не меняются со временем. Их можно отобразить в виде ориентированного мультиграфа: точки – ФУ, дуги – связи между ФУ (если результат с выхода одного ФУ передается сразу на вход другого). Назовем этот мультиграф ***графом ВС***. Предположим: исходные данные введены в ВС мгновенно, результаты работы одного ФУ передаются другому без задержки. Исследуем ***максимальную производительность*** такой ВС.

Утв. 4. Если все ФУ простые, а граф ВС связный => $P_{R,\max} = m \cdot \min_{1 \le k \le m} P_{M,k}$.

Следствие 1: При выполнении условий Утв. 4:

1) асимптотически каждое ФУ выполняет одно и тоже число операций в секунду;
2) загруженность любого ФУ не превосходит загруженность самого непроизводительного ФУ;
3) если загруженность какого-либо ФУ = 1, то это самое непроизводительное ФУ;
4) загруженность ВС $Z \le Z_{\max} = m \cdot \min_{1 \le k \le m} P_{M,k} / P_M$;
5) ускорение ВС $S_m \le S_{m,\max}^{(tech)} = m \cdot \min_{1 \le k \le m} P_{M,k} / \max_{1 \le k \le m} P_{M,k}$.

Следствие 2 (1-ый Закон Амдала): Производительность ВС, состоящей из связанных ФУ, в общем случае определяется самым непроизводительным устройством.

Следствие 3: Если все ФУ простые, а граф ВС связный => асимптотическая производительность ВС будет максимальной, если все устройства имеют одинаковые пиковые производительности.

Смысл: Расписание подачи команд должно минимизировать простои устройств!

3. Связь аппаратных и программных ограничений.

Рассмотрим ВС из m простых универсальных ФУ. Пусть Q – число всех элементарных операций алгоритма, q – максимальное число элементарных операций в параллельных ветвях алгоритма (***высота параллельной формы алгоритма***), k – число параллельных ветвей (***ширина параллельной формы алгоритма***).

A = {O₁,...,O_Q} = последовательный алгоритм и его операции

$$\left|\begin{matrix} O_{11} \\ \dots \\ O_{1Q_1} \end{matrix}\right| \dots \left|\begin{matrix} O_{k1} \\ \dots \\ O_{kQ_k} \end{matrix}\right| \Rightarrow \left|\begin{matrix} O_{01} \\ \dots \\ O_{0Q_0} \end{matrix}\right|$$

параллельная форма алгоритма

$Q_1,...,Q_k$ - выполняем независимо

Q_0 - выполнение зависит от $Q_1,...,Q_k$

Утв. 5. Максимально возможное ускорение $S_m \le S_{m,\max}^{(alg)} = Q / q$ при любом числе устройств m.

<u>Док-во:</u> из утв. 3-4) имеем $S_m = Z_1 + \dots + Z_m = Q_1*\tau/T + \dots + Q_m*\tau/T = Q*\tau/T$, $T >= q*\tau$ => $S_m <= Q/q$.

Следствие 1: Минимальное число ФУ ВС, при котором может быть достигнуто максимально возможное ускорение, равно ширине алгоритма, т.е. m >= k.

Пусть Q_0 операций из Q выполняются последовательно и $\beta=Q_0/Q$ – ***доля последовательных вычислений***.

Следствие 2 (2-й закон Амдала): Если ВС состоит из m одинаковых простых универсальных ФУ и при выполнении параллельной части алгоритма они все загружены полностью, то $S_{m,\max} = m / [m*\beta+(1-\beta)]$.

Док-во: из утв. 3-4) $S_m = Z_1 + \dots + Z_m$, и все последовательные операции выполняются на первом ФУ => $Z_1 = [\beta*Q*\tau+ (1-\beta)*Q*\tau/m]/T=1$, $Z_k = [(1-\beta)*Q*\tau/m]/T$ (k>1) => $S_m = 1 + (m-1)*[(1-\beta)*Q*\tau/m]/[\beta*Q*\tau+ (1-\beta)*Q*\tau/m]$ => $S_m = 1 + (m-1)*(1-\beta)/[m*\beta+(1-\beta)] = m / [m*\beta+(1-\beta)]$ – ч.т.д.

Следствие 3 (3-й закон Амдала): Если ВС состоит из простых одинаковых универсальных ФУ, то при любом режиме работы ее ускорение <= $1/\beta$.

Зам. 1: На МВС с большим числом процессоров эффективно решаются задачи, где β=0.01-0.001.

Зам. 2: 3-й закон Амдала используется для прогнозирования ускорения.

Резюме: Из характеристик ВС получаем m и $S_{m,\max}^{(tech)}$, из свойств алгоритма k и $S_{k,\max}^{(alg)}$ и сравниваем. Первичен в любом случае алгоритм, то есть используемое в реальных вычислениях $m \le k$.

Лекция 4. Архитектуры современных вычислительных систем.

1. Влияние архитектуры МВС на эффективность решения задачи.

Цикл решения задачи: Физический объект – математическая модель – дискретная модель – алгоритм решения – программная реализация – вычисления на ЭВМ – анализ результатов. *На всех этапах* может понадобиться как *понятие параллелизма*, так и *необходимость его использования при вычислениях* для получения новых знаний об объекте. *При написании параллельной программы необходимо знание аппаратной среды*, т.е. архитектуры МВС. Иначе код может оказаться неэффективным или вообще не пригодным для расчетов.

2. Классификация архитектур ВС.

Основные параметры ВС: количество и архитектура процессоров, организация памяти, топология и скорость межпроцессорных коммуникаций, синхронность работы отдельных устройств, способ выполнения элементарных операций, технология программирования и др. Классификация ВС призвана эффективно решать задачу *отображения алгоритма решения конкретной задачи на архитектуру конкретной ВС*.

1) Классификация Флина (1966 г.) – в основе понятие потока команд и данных. 4 класса архитектур:

SISD (Single Instruction stream / Single Data stream): УУ<--(ПК)-->ПР<--(ПД)-->ОЗУ (потоки команд и данных)

Пример: классические послед. ЭВМ – PDP-11, VAX 11/780, CDC 6600 (скалярная) и CDC 7600 (векторная).

SIMD (Single Instruction stream / Multiple Data stream): УУ<--(ПК)-->ПР<==(ПД)==>ОЗУ

Пример: ILLIAC IV (векторная), Cray-1 (векторно-конвеерная), все ВС которые имеют матрицы процессоров.

MISD (Multiple Instruction stream / Single Data stream): УУ<==(ПК)==>ПР<--(ПД)-->ОЗУ

Пример: данный класс пуст – таких машин не удалось пока создать

MIMD (Multiple Instruction stream / Multiple Data stream): УУ<==(ПК)==>ПР<==(ПД)==>ОЗУ

Пример: все мультипроцессорные системы: Cm*, C.mmp, Cray Y-MP, BBN Butterfly, Intel Paragon, Cray T3D

Все векторно-конвеерные машины тоже можно сюда отнести.

Эта классификация является основой современной, но имеет 2 недостатка: 1) не исчерпывает все виды ВС и 2) класс MIMD слишком широк.

2) Классификация Хокни – уточнение класса MIMD.

Класс MIMD подразделяется на 3 группы: **конвеерные, переключаемые, сети**. Переключаемые делятся на **ВС с общей и распределенной памятью**. Сети делятся на ВС, в которых процессоры связаны в виде **регулярных решеток, гиперкубов, иерархических структур, структур, изменяющих конфигурацию**.

Идея: множественный поток команд может обрабатываться ***либо*** одним конвеерным устройством, работающем в режиме разделения времени для отдельных потоков, ***либо*** каждый поток обрабатывается своим собственным устройством.

1-й класс MIMD (конвеерные однопроцессорные).

2-й класс MIMD (многопроцессорные) делится на 2: **переключаемые** (аппаратная связь каждый с каждым через переключатель) и **сети** (прямая аппаратная связь только с ближайшими соседями, удаленная связь с остальными через систему маршрутизации). Переключаемые разделяются на системы с **общей** и **распределенной** памятью, доступ к которой осуществляется через переключатель. Сетевые ВС все имеют распределенную память и подразделяются только в соответствии с топологией сети. Примеры:

конвеерные - процессорные модули в Denelcor HEP, компьютеры Tera MTA;

переключаемые с общей памятью – BBN Butterfly, Cray C90;

переключаемые с распределенной памятью – PASM, PRINGLE, IBM SP2;

сети – 1CAP (звезда); ***транспьютерные системы***, Parsytec PowerXplorer, Intel Paragon, Cray T3D (регулярные решетки разной размерности, подробнее о транспьютерах); NCube, Intel iPSC (гиперкубы); Cm*, CEDAR (кластеры – компьютеры на одном коммутаторе или свиче); Parsytec CC (сети компьютеров с иерархической структурой в виде пирамид, деревьев и т.д.); сети компьютеров, изменяющие конфигурацию (интеллектуальные коммутаторы и свичи).

Зам. Есть на самом деле еще один класс ***гибридные ВС*** – сочетающие несколько архитектур. Пример: Connection Machine 2 – на внешнем уровне гиперкуб, каждый узел которого есть кластер процессоров с полной связью.

3) Классификация Фенга (1972 г.) - классификация по двум признакам – число бит ***n*** в машинном слове, обрабатываемых параллельно при выполнении машинных инструкций (сейчас совпадает с длиной машинного слова), число слов ***m***, одновременно обрабатываемых ВС: ***C=(n,m)*** – ***n*** – число битовых слоев, ***m*** – ширина битового слоя. ***P=n*m*** – максимальная степень параллелизма ВС.

В результате имеем 4 класса ВС:

- разрядно-последовательные и пословно-последовательные (***n=m=1***);
- разрядно-последовательные, пословно-параллельные (***n=1, m>1***);
- разрядно-параллельные, пословно-последовательные (***n>1, m=1***);
- разрядно- параллельные и пословно-параллельные (***n>1, m>1***).

Недостаток: нет разницы между процессорными матрицами, векторно-конвеерными и многопроцессорными системами (т.е. не уточняется способ вычисления ***m***). Достоинство – введение единой метрики для сравнения ВС.

4) Классификация Хендлера – классификация на основе различия между тремя уровнями обработки данных: уровень **выполнения программы** (выборка и дешифрация команд УУ); уровень **выполнения команд** (исполнение команд АЛУ); уровень **битовой обработки** (обработка битов элементарными логическими схемами - ЭЛС).

Пусть ***k*** – число УУ, ***d*** – число АЛУ, ***w*** – число групп ЭЛС в каждом АЛУ. Тогда ***t(C)=(k,d,w)*** – характеристика компьютера без учета наличия конвеера. Если учесть такую возможность, то после расширения трехуровневой модели получаем характеристику ***t(C)=(k, k', d, d', w, w')***, где ***k***-число процессоров, работающих параллельно, ***k'***-глубина макроконвеера из процессоров, ***d***-число АЛУ в каждом, параллельно работающем процессоре, ***d'***-глубина конвеера в каждом АЛУ, ***w***-число разрядов, обрабатываемых в АЛУ параллельно, ***w'***-число ступеней в конвеере ЭЛС, обрабатывающих отдельные разряды. ***P = k*k'*d*d'*w*w'*** – максимальная степень параллелизма.

Дополнительно Хендлер вводит ***три операции*** для описания более сложных (гибридных) архитектур и учета других устройств ВС, отличных от процессоров: «x» – конвеерная обработка данных; «+» – параллельная обработка данных, «v» – альтернативная обработка данных (одна и та же ВС может работать в нескольких режимах).

5) Классификация Шнайдера (1988) – классификация на основе разделения потоков команд и данных на адреса и их содержимое. Обозначения: $S = \{(a_1<t_1>)\ (a_2<t_2>)\ldots,\ (b_1<u_1>)\ (b_2<u_2>)\ldots,\ (c_1<v_1>)\ (c_2<v_2>)\ldots\}$ – ***поток ссылок,*** конечное множество бесконечных последовательностей пар. Каждая пара содержит адрес и набор из n значений (неотрицательных чисел). ***I – поток команд*** - если значения рассматривать как команды, ***D – поток данных***, если значения рассматривать как данные. Последовательность пар адрес-значение это история выполнения команд или перемещения данных. Число последовательностей в потоке равно числу инструкций (различных данных), которые компьютер может выполнять (обрабатывать) одновременно. $S_a = <a_1\ b_1\ c_1>\ <a_2\ b_2\ c_2>\ \ldots$ - ***последовательность адресов*** потока S. $S_v = <t_1\ u_1\ v_1>\ <t_2\ u_2\ v_2>\ \ldots$ - ***последовательность значений*** потока S. $w(S_x)=n$ – ***ширина*** последовательности S_x, равная длине элементов. => $w(S_a)=|S|$, $w(S_v)=n*|S|$, $|S|$ - мощность множества S.

Пара (I,D) – ***вычислительный шаблон (ВШ)***. Все ВС можно классифицировать по ВШ.

ВС может исполнить ВШ (I,D), если она в состоянии:

1) выдать $w(I_a)$ адресов команд для одновременной выборки из памяти;
2) декодировать и проинтерпретировать одновременно $w(I_v)$ команд;
3) выдать одновременно $w(D_a)$ адресов операндов;
4) выполнить одновременно $w(D_v)$ операций над различными данными.

Если эти условия выполнены, то ВС описывается как $I_{w(Ia),w(Iv)}\ D_{w(Da),w(Dv)}$.

Классификация Флина в этой интерпретации:

SISD: $|I|=|D|=1$, $w(I_a)=w(D_a)=w(I_v)=w(D_v)=1$ => $I_{1,1}\ D_{1,1}$

SIMD: $|I|=1$, $w(I_a)=w(I_v)=1$, $|D|>=1$, $w(D_a)>1$, $w(D_v)>1$

Для более детальной классификации вводятся три значения, которые может принимать w: w=s (s=1), w=c (c – небольшая константа – жесткое ограничение аппаратуры – пример VLIW процессоры), w=m (m – величина от 1 до большого конечного числа – компьютер по данному параметру масштабируем).

В итоге получаем следующие классы систем:

$I_{ss}D_{ss}$ – SISD системы; $I_{ss}D_{sc}$ – SISD системы, способные выполнять одну операцию над несколькими данными, расположенными по одному адресу, но с разным смещением (режим обработки двух полуслов);
$I_{ss}D_{sm}$ – SIMD системы без возможности получения уникального адреса данных в каждом процессорном элементе;
$I_{ss}D_{mm}$ – SIMD системы с возможностью изменения адресов операндов в каждом процессорном элементе;
$I_{sc}D_{cc}$ – системы, выбирающие и исполняющие одновременно несколько команд, для доступа к которым используется один адрес (VLIW-компьютеры); $I_{mm}D_{mm}$ – все MIMD системы.
Результат: гибкая систематизация SIMD, но MIMD системы опять в одном классе.

6) Классификация Скилликорна (1989) – специальное расширение классификации Флина, основанное на описании архитектуры с помощью составляющих ее 4-х компонетов:
а) процессор команд (IP – Instruction Processor) – интерпретатор команд как отдельное ФУ (может отсутствовать);
б) процессор данных (DP – Data Processor) – АЛУ;
в) иерархия памяти (IM – Instruction Memory, DM – Data Memory) – ЗУ для команд и данных, пересылаемых между процессорами;
г) переключатель (SW – Switch) – абстрактное устройство, обеспечивающее связь между процессорами и памятью (может отсутствовать).
Функции процессора команд схожи с функциями УУ последовательных машин: на основе своего состояния и информации от DP процессор команд 1) определяет адрес следующей команды, 2) осуществляет доступ к IM для ее выборки; 3) получает и декодирует команду; 4) сообщает DP команду, которую надо выполнить; 5) определяет и посылает DP адреса операндов команды в DM; 6) получает от DP информацию о результате выполнения команды.
Функции процессора данных схожи с функциями АЛУ: процессор данных 1) получает от IP команду, которую надо выполнить; 2) получает от IP адреса ее операндов; 3) выбирает операнды из DM; 4) выполняет команду; 5) запоминает результат в DM; 6) возвращает в IP информацию о состоянии после выполнения команды.
Схема: DM <=(их адреса) (операнды)=> DP <=(команды) (инфо о состоянии)=> IP <=(команды) (их адреса)=> IM. Она не содержит переключателей. Однако в других ВС они есть. Поэтому ввел 4 типа абстрактных переключателей:
1-1 – переключатель, связывающий пару ФУ;
n-n – переключатель, связывающий попарно устройства из двух множеств;
1-n – переключатель, связывающий одно устройство со всеми ФУ из некоторого множества;
nxn – переключатель, обеспечивающий связь каждый с каждым для двух множеств ФУ.
Примеры:
1) все матричные процессоры имеют переключатель типа ***1-n*** для связи единственного процессора каманд с множеством процессоров данных;
2) в системах, где каждый процессор данных имеет локальную память, эта связь обозначается как ***n-n***;
3) если каждый процессор команд может связаться с любым другим процессором, то эта связь обозначается как ***nxn***.
В итоге классификация Скилликорна включает восемь параметров:
1) кол-во IP; 2) кол-во IM; 3) тип переключателя IP-IM; 4) кол-во DP; 5) кол-во DM; 6) тип переключателя DP-DM; 7) тип переключателя IP-DP; 8) тип переключателя DP-DP.
Примеры: Connection Machine 2 ***(1, 1, 1-1, n, n, n-n, 1-n, nxn)***; BBN Butterfly ***(n, n, n-n, n, n, nxn, n-n, -)***.

3. Особенности современных вычислительных систем. Гибридные архитектуры.

1) **векторно-конвеерные системы** - Cray-1(1976), Cray C90 (1990), NEC Earth Simulator (2002);
2) **системы с общей памятью** – SMP-системы, ccNUMA – HP Superdome (2001);
3) **системы с распределенной памятью** –
3a) **суперкомпьютеры с внешней шиной** Intel Paragon, IBM SP1/SP2, Cray T3D/T3E (1990), Parsytec CC (1995);
3b) **кластеры** – Beowulf (1994), Avalon (1998), Velocity+ (2000), МВС-1000М (2001), МВС-15К (2005), МВС-100К (2008), МВС-10П (2013);
3c) **сети МВС, метакомпьютеры** – отличаются неоднородностью.
4) **гибридные системы**.

Первые две группы бьют рекорды производительности по вычислительной мощности, приходящейся на один процессор, однако число процессоров жестко ограничено <=> не масштабируемы. Третья группа – наиболее распространены, не имеют ограничений по числу процессоров (масштабируемы), объему ОЗУ и внешней памяти. Четвёртая группа имеет самый высокий показатель - пиковая производительность на единицу потребляемой мощности.

4. Структура и особенности гибридных ВС.

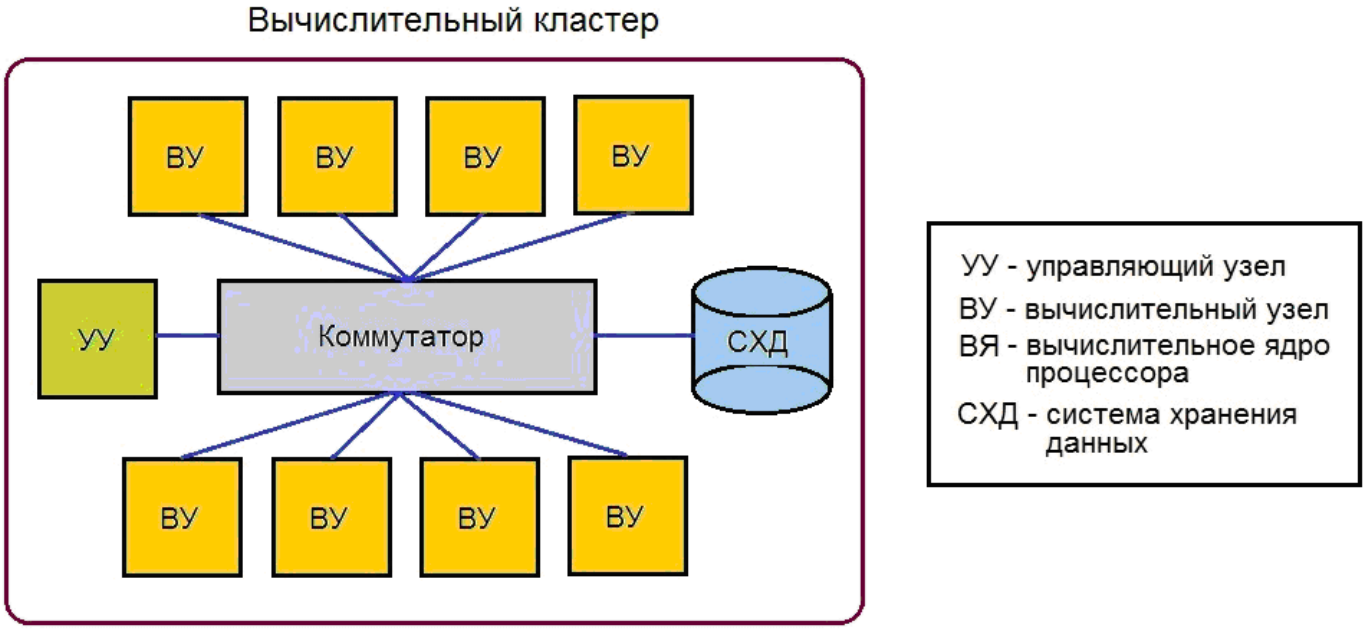


Рис. 1. Архитектура вычислительного кластера.

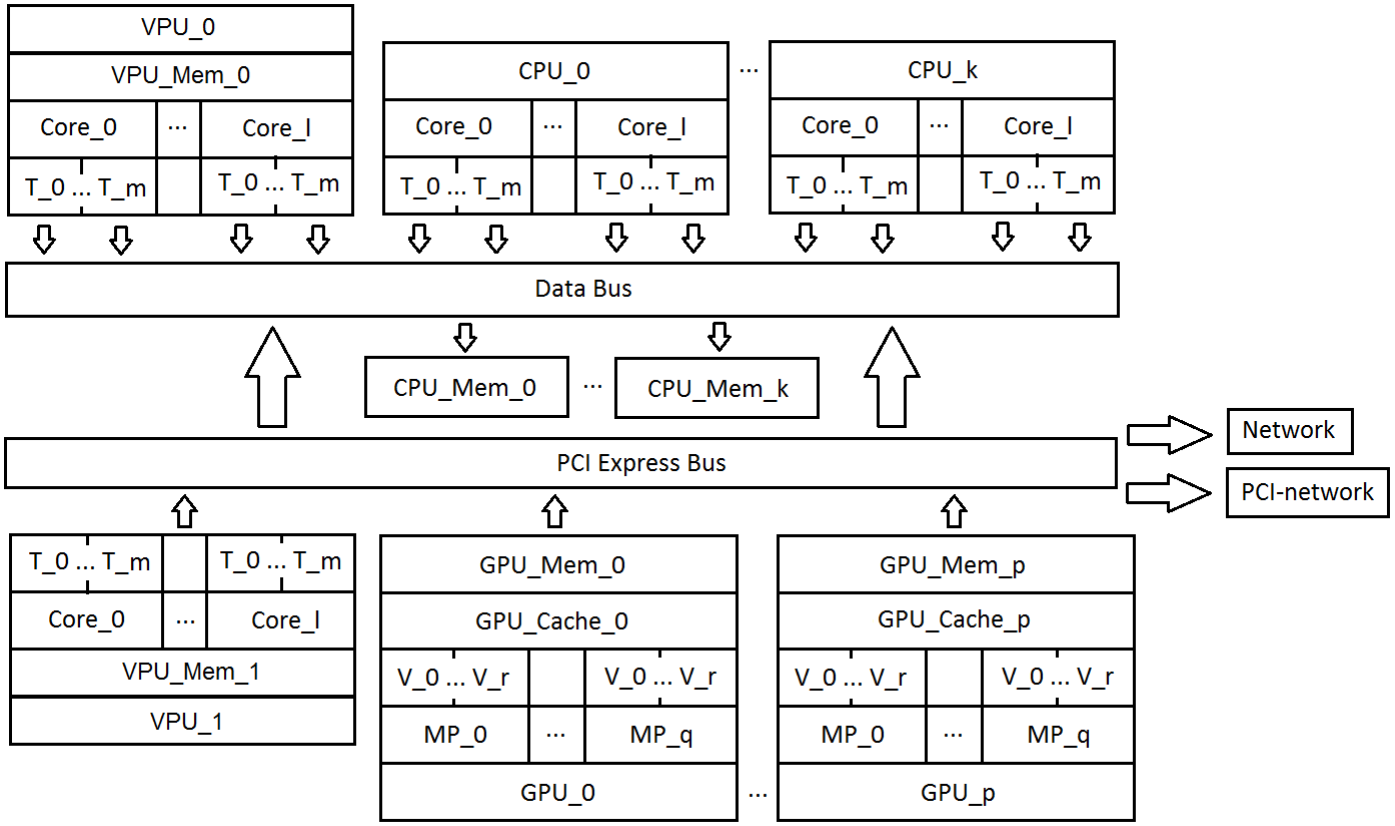


Рис. 2. Архитектура узла гибридного вычислителя.

Новые виды вычислителей: APU = CPU+GPU, IPU=CPU+GPU+Mem

APU (сокр. от **Accelerated or Agregated Processing Unit**, ускоренное или агрегированное обрабатывающее устройство) – гибридный центральный процессор, который объединяет центральный процессор с графическим процессором в одном кристалле.

Лекция 5. Принципы построения параллельных алгоритмов. Виды параллелизма.

1. Параллельные вычисления как взаимодействие последовательных процессов.

Вычисления - последовательный процесс выполнения арифметических или логических действий. ***Параллельные вычисления*** – ***одновременное*** выполнение ***нескольких*** последовательных вычислений, результаты которых используются для получения ***общего*** результата. => ***Параллельные вычисления есть взаимодействие последовательных процессов вычислений*** (см. Хоар, Дейкстра).

В случае последовательного алгоритма необходимо доказательство его конечности и правильности (корректности результата), а также его корректная реализация на выбранном языке программирования. Даже при соблюдении этих условий имеется ограничение по входным и выходным данным, которое делает исполнение программы либо невозможным, либо приводит к неверным результатам.

В случае параллельного алгоритма не существует формального способа доказательства его правильности. Для конкретной его реализации в виде программы на некотором языке тем более. Не помогает никакая система тестов. Причина неадекватного поведения параллельных программ – в ***неопределенности*** в порядке выполнения действий, необходимых для получения результата. Каждый параллельный процесс "живет в своем мире по своим часам". Следствие неопределенности – образование ***тупиков*** (программа ничего не делает) и выдачи ***неверных результатов***.

Тем не менее, существуют способы разработки параллельных алгоритмов и программ, позволяющие на практике решать поставленные задачи. Их суть – ограничение в свободе выбора структуры алгоритма.

2. Виды параллелизма.

Параллелизм – 1) ***свойство последовательного алгоритма*** совершать независимо все или некоторые действия; 2) ***способ преобразования*** последовательного алгоритма в параллельный.

Рассмотрим второе определение. А именно виды параллелизма, которые успешно применяются:

1) ***Алгоритмический (внутренний, функциональный)*** – разбиение последовательного алгоритма на отдельные независимые части. Здесь число параллельных ветвей параллельного алгоритма не превосходит числа независимых частей исходного алгоритма, т.е. в отличие от следующих этот способ ***не масштабируемый*** (нельзя взять произвольное число процессоров).

$$A = \{O_1,\dots,O_N\} = \left|\begin{matrix} O_{11} \\ \dots \\ O_{1N_1} \end{matrix}\right| \dots \left|\begin{matrix} O_{k1} \\ \dots \\ O_{kN_k} \end{matrix}\right| \Rightarrow \left|\begin{matrix} O_{01} \\ \dots \\ O_{0N_0} \end{matrix}\right|$$

последовательный алгоритм и его операции

параллельная форма алгоритма

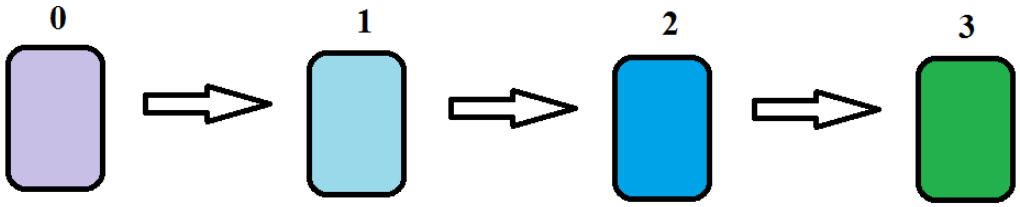
$Q_1,\dots,Q_k$ - **выполняем независимо**

Q_0 - **выполнение зависит от** $Q_1,\dots,Q_k$

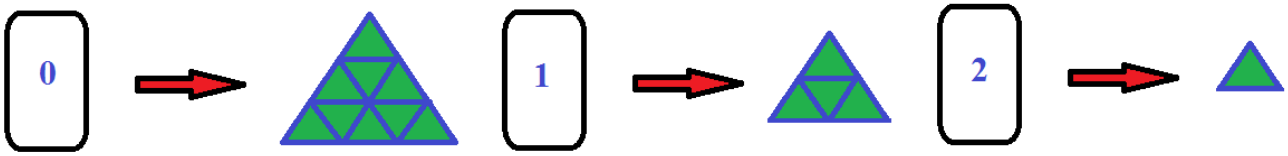
2) ***Геометрический (координатный)*** – разбиение последовательного алгоритма на группы однородных действий над однотипными данными, при котором связи между группами отсутствуют или минимальны. Последнее свойство называют свойством ***локальности*** алгоритма. Локальный алгоритм допускает разбиение данных на части соответственно числу вычислителей, т.е. ***масштабируем***.

СУММА(1:1000) = СУММА(1:500)+СУММА(501:1000)

3) ***Конвеерный*** – объединение различных операций последовательного алгоритма в конвеер.



4) ***Коллективное решение (процессорная ферма)*** – разбиение последовательного алгоритма на элементарные фрагменты (кванты), которые выполняются не одинаковыми по производительности вычислителями.



Пример: ***Стена Фокса*** (длиной L и высотой H). Ее складывают n каменщиков. Работа – заливка раствора и укладка кирпичей.

Алгоритмический параллелизм – два каменщика, один заливает раствор, другой кладет кирпичи (***разбиение по типу работ***). Способ не эффективный из-за ограничения на число работников. Если n=m+k работников, m – заливщики, k<<m – каменщики – ***неэффективное решение***.

Геометрический параллелизм – вся стена разбита на одинаковые участки по длине, каждый каменщик кладет свой участок и общается только с соседями на стыках слоев (***разбиение по x***). Способ эффективен, когда длина участков велика. При коротких участках возникает проблема ожидания друг друга на стыковке слоев.

Конвеерный параллелизм – распределение работы каменщиков по слоям: каждый делает один или несколько целых слоев стены (***разбиение по y***). Способ предполагает простои работников. Эффективен, если стена достаточно высокая и длинная, а все каменщики работают с одинаковой скоростью. Темп работы зависит от самого медленного из них.

Коллективное решение – каждому каменщику выдается некоторое количество кирпичей и раствора; каждый кладет стену в любом готовом к кладке и не занятом другими месте, пока не закончатся материалы; затем снова берет порцию материалов и продолжает процесс (***квантование работ***). Требуется синхронизация при получении материалов и выборе участков работ => есть простои. Эффективен при большом объеме работ (число квантов работы >> числа работников) и быстрой раздаче материалов и свободных участков.

3. Теоретическое ускорение и эффективность параллельных алгоритмов.

Параллельный алгоритм (ПА) определим как последовательность алгоритмов $\{A_k\}$ ($k=1,2,\ldots,p,\ldots$), где каждый член A_k – реализация алгоритма на k вычислителях.

Существуют три класса ПА:

1) $\{A_1, 0, 0, \ldots\}$ – вырожденные ПА;

2) $\{A_1, A_2, \ldots, A_k, 0, 0, \ldots\}$ – ПА с ограниченным параллелизмом;

3) $\{A_1, A_2, \ldots, A_k, \ldots, A_{2k}, \ldots\}$ – ПА с неограниченным параллелизмом.

При теоретической разработке параллельных алгоритмов и оценке их ускорения и эффективности примем следующие ***соглашения***:

а) ВС состоит из p универсальных одинаковых по производительности вычислителей;

б) решаем только большие задачи, которые обеспечивают максимальную загруженность вычислителей.

При этих условиях решить задачу на всей ВС нельзя быстрее, чем в p раз, по сравнению с ее решением на одном вычислителе.

В соответствии с соглашениями: $S_k = T_1/T_k$ – ускорение, $E_k = S_k/k$ или $(S_k/k)*100\%$ – эффективность для k-го члена последовательности, T_1, T_k – оценки времен.

Цель программиста – добиться сокращения времени решения задачи за счет распараллеливания. Однако часто это достигается неэффективным способом.

Причины неэффективности ПА: 1) неэффективность последовательности; 2) обмены между вычислителями; 3) доп. действия, связанные с параллелизацией.

При разработке ПА необходимо ***минимизировать*** все три фактора.

Основная характеристика алгоритма в рассматриваемом контексте – ***степень параллелизма d*** – число действий, которые могут выполняться одновременно, каждое на своем вычислителе.

Примеры: 1) $c_i=a_i+b_i$ ($i=1,\ldots,N$): степень параллелизма $d = N$; 2) $c_0=0$, $c_i=c_{i-1}+a_i$ ($i=1,\ldots,N$): степень параллелизма $d = 1$. Степень параллелизма необходимо учитывать при разработке ПА.

Степень параллелизма не зависит от числа вычислителей, это внутреннее свойство алгоритма.

Ускорить алгоритм более чем в d раз нельзя без изменения его структуры. Вывод: $p <= d$.

Уточнение понятия масштабируемости ПА.

Масштабируемость параллельного алгоритма – зависимость времени решения задачи от количества используемых вычислителей.

Сильная масштабируемость ПА – зависимость времени решения задачи от количества используемых вычислителей при фиксированном объеме операций, приходящемся ***на все*** вычислительные устройства.

Слабая масштабируемость ПА – зависимость времени решения задачи от количества используемых вычислителей при фиксированном объеме операций, приходящемся ***на каждое*** вычислительное устройство.

Алгоритмов с бесконечной сильной масштабируемостью не существует – 2й класс.

Алгоритмы с бесконечной слабой масштабируемостью существуют, например, построенные на принципе геометрического параллелизма – 3й класс.

4. Примеры оценки ускорения и эффективности ПА.

Пример 1: вычисление суммы чисел $s = a_1+\ldots+a_N$;

A_1: $s=a_1$, $s=s+a_i$, $i=2,\ldots,N$. – последовательный алгоритм. Нужен параллельный вариант.

1) ПА по методу сдваивания: $s=((a_1+a_2)+(a_3+a_4))+((a_5+a_6)+(a_7+a_8))$.

Если $N=2^q$ => $q=\log_2 N$ этапов, степень параллелизма $d_k = N/2^k$ на каждом этапе сложения. Пусть число процессоров $p=N/2$ (равно макс. степени параллелизма алгоритма сдваивания). Пусть t – время сложения и инициализации s, αt – время обмена. Посчитаем: $T_1=tN$, $T_p=2t*\log_2 N+2\alpha t*(\log_2 N-1)$, $S_p = T_1/T_p = tN / [2t*\log_2 N+2\alpha t*(\log_2 N-1)] = p / [(1+\alpha)*\log_2 p+1]$, $E_p = S_p / p = 1 / [(1+\alpha)*\log_2 p+1]$.

Почему: на каждом этапе каждый вычислитель выполняет: инициализацию и сложение => один из процессоров выполнит $2tq=2t\log_2 N$ действий. Передача двух чисел – $2\alpha t$, время всех обменов $2\alpha t*(\log_2 N-1)$.

Анализ: если α велико, то никакой эффективности, если α=0, то эффективность не более 50% (достигается на 2 проц.). Почему? Плохой метод в основе ПА.

2) ПА на основе геометрического параллелизма. Разобъем массив на p частей. На каждом вычислителе выполним сложение $m=N/p$ чисел обычным образом, а затем сложим p чисел на одном из вычислителей: $s=(a_1+a_2)+(a_3+a_4)+(a_5+a_6)+(a_7+a_8)$. Посчитаем: $T_1=tN$, $T_p=tN/p+tp+\alpha tp$, $S_p= T_1/T_p = tN / [tN/p+tp+\alpha tp] = p/[1+(1+\alpha)p^2/N]$, $E_p = S_p / p = 1/[1+(1+\alpha)p^2/N]$.

Анализ: Можно достичь высокой эффективности $p<<N$. С ростом числа вычислителей эффективность падает, т.к. размер задачи фиксирован.

Вывод: последовательности ПА могут быть разные. Часто хороший последовательный алгоритм имеет малую степень параллелизма и приводит к плохому ПА. И наоборот, плохой последовательный алгоритм хорошо "параллелится". Поэтому вводят понятия: ***ускорение и эффективность ПА по сравнению с наилучшим последовательным***. Можно говорить также о наилучшей последовательности параллельных алгоритмов и методах построения. Для оценки конкретной последовательности алгоритмов будем использовать 2-ой закон Амдала в сделанных предположениях:

$S_p= T_1/T_p=T_1/[\beta T_1+(1-\beta)T_1/p+T_d]=1/[\beta+(1-\beta)/p+T_d/T_1]$, β- доля непараллельных операций, $T_d=T_d(p)$ – время на обмены и дополнительные действия. Характерные ситуации:

1) $\beta=0$, $T_d = 0$ => $S_p=p$ - идеальный случай;

2) $\beta>0$, $T_d = 0$ => $S_p=p/[1+\beta(p-1)]<p$ ($\beta=1/2$ => $S_p<2$) – ограниченность ускорения;

3) $\beta>0$, $T_d > 0$, $T_d>>T_1$ => $S_p<1$ – при больших накладных расходах не имеет смысла параллелить.

Пример 2: вычисление $e^x \sim s(x)=1+x/1!+x^2/2!+\ldots+x^N/N!$, $N>1$.

1) ***Прямая схема*** - N сложений, $N-1$ делений, $N*(N-2)+1$ умножений ($x^k = x*x*\ldots*x$ -> $k-1$, $k!=1*2*\ldots*k$ -> $k-2$);

main: s = 1+x; for (k=2; k<=N; k++){p=f(x,k); s=s+p}; f(x,k): q=x/2; for (i=3;i<=k;i++){q=q*x/i}

2) ***Модиф. схема*** (накопление члена последовательности) - ***N*** сложений, ***N-1*** делений, ***N-1*** умножений;
main: s = 1+x; p = x; for (k=2; k<=N; k++){p=p*x/k; s=s+p}
3) ***Схема Горнера***: $e^x=1+x*(1+x/2*(1+x/3*(\ldots(1+x/(N-1)*(1+x/N))\ldots)))$ ***–N*** сложений, ***N–2*** делений, ***N-1*** умножений.
main: s = 1; for (k=N; k>1; k--){s=1+s*x/k}; s=1+s*x;
Анализ:
1) плохая точность, избыточная арифметическая сложность (АС) алгоритма, параллелится хорошо геометр. методом, но с некоторым дисбалансом загрузки процессоров;
2) средняя точность, оптимальная АС, параллелится плохо;
3) хорошая точность, оптимальная АС, не параллелится совсем.
Задача: предложить оптимальную параллельную последовательность. Посчитать ускорение и эффективность, учитывая, что t – время на сложение, αt – на умножение, βt – на деление, γt – на пересылку одного числа; учесть операции на увеличение счетчика цикла и проверки на конец цикла (время сравнения = времени сложения). Что изменится, если считать сумму с проверкой условия на точность: $|e^x-s(x)|$<= eps? ***Ответ:*** $\mathbf{A_1}$ – схема Горнера, $\mathbf{A_2}$ – Табл. 1, $\mathbf{A_3}$ – Табл. 2, $\mathbf{A_k}$ – см. ниже.

Таблица 1. ***Решение для двух процессоров:***

Вариант 1			***Вариант 2***		
1-ый проц.	***Обмен***	***2-ой проц.***	***1-ый проц.***	***Обмен***	***2-ой проц.***
p=x		***s=1, k=2***	***s1=1, k=2***		***s2=0, p=x, k=1***
	<-- k --; -- p -->			***<-- p --***	
p=p*x/k		***if(k<N):s=s+p, k=k+1***	***p=p*x/k***		***if(k<N):s2=s2+p, k=k+2***
	<-- k --; -- p -->			***-- p -->***	
p=p*x/k		***if(k<N):s=s+p, k=k+1***	***if(k<N):s1=s1+p, k=k+2***		***p=p*x/k***
	<-- k --; -- p -->			***<-- p --***	
…			***…***		
p=p*x/k		***if(k<N):s=s+p, k=k+1***	***p=p*x/k***		***else: s2=s2+p***
	<-- k --; -- p -->			***-- p -->***	
		else:s=s+p, stop	***else:s1=s1+p***		
				<-- s2 --	
			s=s1+s2, stop		

Таблица 2. ***Решение для трех процессоров:***

1-ый проц.	***Обмен***	***2-ой проц.***	***Обмен***	***3-ий проц.***
p=x				***s=1, k=1***
	-- p -->			
	<-- p --		***-- p -->***	
p=p*x				***if(k<N):s=s+p, k=k+1***
	-- p -->		***<-- k --***	
		p=p/k		
	<-- p --		***-- p -->***	
p=p*x				***if(k<N):s=s+p, k=k+1***
…				
p=p*x				***if(k<N):s=s+p, k=k+1***
	-- p -->		***<-- k --***	
		p=p/k		
	<-- p --		***-- p -->***	
				else:s=s+p, stop

Решение для произвольного числа процессоров (модифицированная схема):

Пусть N – точное число членов: $S_N(x) = 1 + x + x^2/2! + \ldots + x^{N-1}/(N-1)!$; p – число процессоров, и $k = N/p$ (дел. нацело).

На 0-м проц.: $S_0 = 1+x+x^2/2!+\ldots+x^{k-1}/(k-1)! = 1*[1+x+x^2/2!+\ldots+x^{k-1}/(k-1)!] = A_0*B_0$

На 1-м проц.: $S_1 = x^k/k!+x^{k+1}/(k+1)!+\ldots+x^{2k-1}/(2k-1)! = [x^k/k!]*[1+x/(k+1)+\ldots+x^{k-1}/((k+1)\ldots(2k-1))] = A_1*B_1$

На 2-м проц.: $S_2 = x^{2k}/(2k)!+x^{2k+1}/(2k+1)!+\ldots+x^{3k-1}/(3k-1)! = [x^{2k}/(2k)!]*[1+x/(2k+1)+\ldots+x^{k-1}/((2k+1)\ldots(3k-1))] = A_2*B_2$

И т.д. до $(p-1)$.

Схема параллельного счета:

Этап 1. Вычисление B_m (m-номер процессора) и накопление величины $C_m=x^{k-1}/(mk+1)\ldots((m+1)k-1))$ к концу этапа.

Этап 2. Передача C_m и вычисление A_m: $0 \text{ -- } C_0 \text{ --> } 1$, $A_1=C_0*x/k$, $C_1=C_1*A_1$, $1 \text{ -- } C_1 \text{ --> } 2$, $A_2=C_1*x/(2k)$, $C_2 = C_2*A_2$, $2 \text{ -- } C_2 \text{ --> } 3$, …

Этап 3. Вычисление S_m

Этап 4. Вычисление S методом сдваивания.

Лекция 6. Организация параллельных процессов.

1. Организация параллельных процессов.

В общем случае организация параллельных вычислений подразумевает организацию параллельных процессов на некой сети вычислителей. Собственно организация параллельных процессов включает:
1) Создание нескольких последовательных процессов.
2) Организацию обмена данными между процессами.
3) Синхронизацию работы созданных последовательных процессов.
4) Использование процессами разделяемых ресурсов.

2. Создание нескольких последовательных процессов.

Существуют три способа создания нескольких последовательных процессов:
а) Запуск пользователем нескольких программных кодов на некотором количестве вычислителей. На нескольких ВС запускаются пользователем вручную заранее подготовленные коды – не всегда одинаковые и совместимые, но всегда машиннозависимые.
Примеры: 1) comp1>main1.px; … comp10>main10.px; 2) server>mpirun –np 8 main.px (сокеты TCP/IP)
б) Запуск из головной программы нескольких новых процессов. В головной программе используются стандартные функции ОС UNIX для создания и работы с дочерними процессами. Среди основных функций используются ***fork*** (создание полной копии процесса), ***execl*** и ***execv*** (запуск исполняемого кода из файла), ***system*** (запуск команды shell), ***wait*** (ожидание завершения работы потомков). Процесс в UNIX – самостоятельная исполняющаяся в данный момент программа. Все процессы в UNIX уникальны и имеют свой PID. Исходный родитель – процесс с PID=0 – запускает все остальные служебные процессы с помощью функций fork, exec, system. Такая же возможность предоставляется и пользователю.
в) Создание ветвей последовательного процесса. Создание ветвей одного и того же процесса с помощью библиотеки threads (задач, нитей), содержащей функции pthread_create (создание новой ветви), pthread_join (ожидание завершения ветви) и др.
г) Гибридный способ, сочетающий несколько предыдущих.

3. Основные способы обмена данными между ними.

1) обмен сигналами (функции kill – посылка сигнала процессу, signal – вызов функции пользователя при получении сигнала с заданным номером);
2) обмен с помощью разделяемых файлов (функция блокировки записи lockf());
3) обмен с помощью каналов (функции pipe, read, write; два файловых дескриптора: для чтения и для записи);
4) обмен с помощью очередей сообщений (функции msgget, msgsnd, msgrcv, msgctl – порции данных одинаковой длины);
5) разделяемые сегменты общей памяти (функции shmget - создание на сервере, shmat - присоединение, shmdt - отсоединение, shmctl - удаление);
6) сокеты TCP/IP и их аналоги.

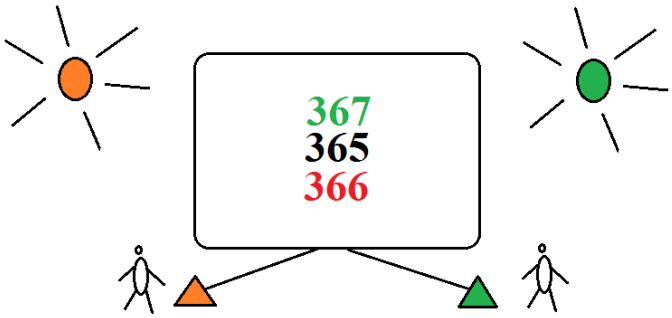
4. Синхронизация работы последовательных процессов.

При организации параллельных процессов (например, решении задачи на нескольких взаимодействующих вычислителях) существуют две фундаментальные проблемы:
1) возникновение тупиков; 2) недетерменированность параллельных вычислений.
Пример – ***задача Дейкстры "Обедающие философы":***
В парке гуляют философы. Там же стоит беседка с одним входом. В нее философы заходят время от времени поесть. При входе дворецкий регулирует проход. В беседке - стол, накрытый на пять персон. На столе - блюдо с макаронами, 5 тарелок и 5 вилок.
Общая схема питания (Схема 1): сесть за стол, двумя вилками положить макароны, затем одной есть и выйти. Второй вилкой можно пользоваться, если кто-то не ест. ***Трудности:*** Пока философов меньше 5-ти, все ОК (все поедят). Если же одновременно придут все 5 – ***тупик*** (никто не может начать есть).
Схема 2. Учтем конечность времени пользования вилками.
Вар. 1. Дворецкий не пускает к столу одновременно более 4-х человек. Тупика не будет, но есть неудобство – очередь перед столовой. Философ, стоящий в конце очереди рискует не поесть никогда. Например, если очередь типа стэка (первый пришел – последний ушел) => ***первый голодный***. Если первый пришел – первый ушел => есть ***проблема кормления привелегированных особ***. В обоих случаях нет гарантии обслуживания абсолютно всех философов.

Вар. 2. Добавим конечность времени обслуживания одного философа с помощью правила: если все вилки заняты и никто не ест, то все вилки кладем на стол и ждем некоторое время. Затем повторяем попытку. В этом случае будет либо ***драка***, либо опять ***тупик***.
Вар. 3. Если у всех по одной вилке, а в тарелке пусто, то тот, кто сидит на 1-ом месте кладет свою вилку на некоторое время. Тогда 1-ый вилку положил, 2-ой взял, положил макароны, вилку вернул, поел и ушел. На место 2-го пришел новый философ. Если опять все взяли вилки одновременно, то ситуация повторяется. => ***Первый не поест никогда***.
Вар. 4. Передвигать маркер места соседу. Тогда ситуация улучшится. Есть и другие варианты.
Иллюстрацией недетерменированности параллельных процессов здесь служит то, что мы не знаем, в каком порядке обслуживают философов.
Вывод: важнейший вопрос организации параллельных процессов – синхронизация их взаимодействия, особенно при пользовании общими ресурсами. Как это конкретно делается? См. ниже.
Существуют ***2 способа синхронизации*** последовательных процессов:
а) передача сообщений;
б) использование общих данных.
Следует также учесть: при использовании нескольких процессов нет единого “глобального” времени, т.е. нельзя сказать, что событие А на процессоре К произошло раньше события Б на процессоре М.
Поэтому алгоритм пишется в предположении, что ***различные процессы работают с различной не известной априори и меняющейся со временем скоростью.***

5. Общие разделяемые ресурсы и методы их совместного использования.
Разделяемые ресурсы – те, что доступны к использованию нескольким процессам. Например, это общая память, каналы связи, файлы и др.
Примеры часто встречающихся ошибок, связанных с разделяемыми ресурсами:
1) одновременная запись и чтение в общую память несколькими процессами;
2) использование асинхронно получаемых данных до их реального получения;
3) модификация асинхронно передаваемых данных до их реальной отправки;
4) одновременная запись в файл;
5) вызов из нескольких процессов одной и той же “нереентерабельной” функции, т.е. модифицирующей свои внутренние статические данные (например, printf).
Эти проблемы решаются при использовании ***низкоуровневых средств синхронизации – синхронных каналов связи и семафоров***.

6. Решение проблемы синхронизации и разделения ресурсов.
Проблема разделения ресурсов (пример). Подсчет событий двумя наблюдателями:
лампа 1 – человек 1 – доска (365) – человек 2 – лампа 2
Одновременно: лев. лампа -> лев. наблюдатель -> 366; прав. лампа -> прав. наблюдатель -> 366. Правильно: 367.



Q: Reg+Read+Sum+Write

Почему так: процесс чтения числа, прибавления 1 и записи исправленного числа разделены по времени. Если эти действия объединить, то ошибки не будет.
Введем следующие понятия:
Изолированный последовательный процесс – последовательность действий, выполняемых независимо от окружения.
Слабо связанные последовательные процессы – изолированные процессы, которые не взаимодействуют между собой за исключением редких и коротких моментов времени.

Параллельный процесс – совокупность слабо связанных последовательных процессов, которые необходимо организовать для выполнения поставленной задачи.
Критический интервал (КИ) – последовательность действий, выполнение которой может быть приравнено к одной непрерываемой операции.
Необходимо защитить критический интервал от одновременного вхождения в него двух процессов.
Соглашения для решения этой задачи:
1) существуют 2 циклических процесса, выполняющих время от времени КИ;
2) внутри КИ каждый процесс находится произвольное конечное время;
3) вне КИ процессы могут работать бесконечно долго;
4) процессы могут иметь любое число общих переменных;
5) процессы могут читать и записывать значения любых, в том числе общих переменных в любое время без ограничений;
6) результат одновременной записи в одну ячейку: либо x, либо y (целостность данных);
7) результат чтения во время записи: либо старый x, либо новый y (целостность данных);
8) если процессу необходимо войти в КИ, то он должен получить возможность входа за конечное время.
В итоге: неделимы остаются только чтение и запись в память.

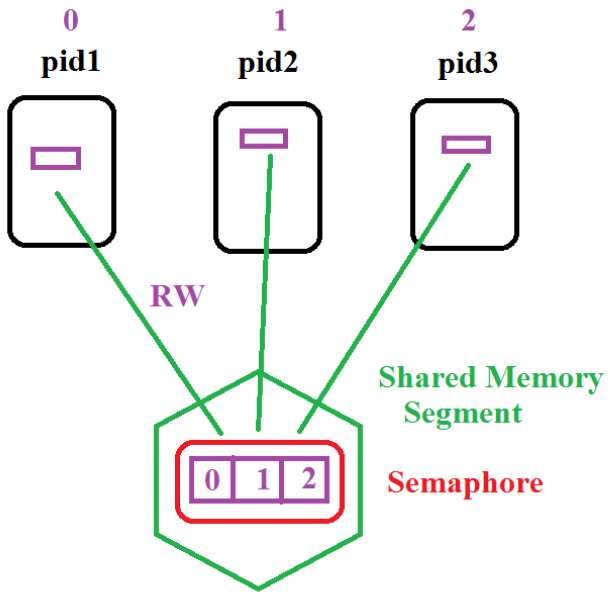
Вариант 1 решения задачи – безопасный (без взаимной блокировки), детермин. послед. вхождения в КИ: 1,2,1,2,...	
1-ый процесс	***2-ой процесс***
int next=1;	
while(1){	while(1){
<any actions>	<any actions>
while(next==2);	while(next==1);
{КИ}	{КИ}
next=2;	next=1;
}	}
Вариант 2 (Деккер, 1968 г.) – громоздкий, код неодинаковый, неизвестно, как обобщить на p процессов	
1-ый процесс	***2-ой процесс***
int next=1, C1=1, C2=1;	
while(1){	while(1){
<any actions>	<any actions>
A1: C1=0;	A2: C2=0;
L1: if (C2==0){	L2: if (C1==0){
if (next==1) goto L1;	if (next==2) goto L2;
C1=1;	C2=1;
while (next==2);	while (next==1);
goto A1;	goto A2;
}	}
{КИ}	{КИ}
next=2;	next=1;
C1=1;	C2=1;
}	}

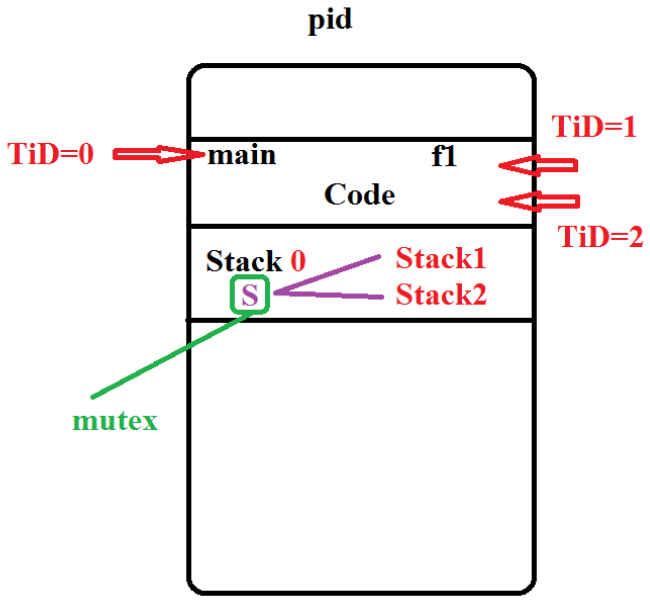
Вариант 3 (Дейкстра, 1965) – использование семафора.
Идея: объединение в одну операцию чтения переменной специального вида, ее модификации и передачи управления в зависимости от ее нового значения. Позволяет иметь единый компактный код и работать с несколькими процессами.
Семафор – неотрицательная целая переменная, доступ к которой возможен только с помощью двух неделимых операций Signal и Wait. Результат Signal(S) – увеличение S на 1 (не равен S=S+1). Если S=5, то результат Signal(S) в двух процессах гарантированно дает 7. Результат Wait(S) – уменьшение S на 1 (если S>0), или останов процесса (если S=0). Останов сбрасывается, как только кто-то выполнит функцию Signal(S), и приостановленный процесс (любой) выполнит в свою очередь уменьшение S на 1.
Решение на обоих процессорах:

```
Semaphore S;
while(1){
<any actions>  Wait(S);  {КИ}  Signal(S);
}
```

Реализация семафоров:

1) тяжелые семафоры – механизм доступа к общим объектам ОС (функции semget, semop, semctl);
2) легкие семафоры – mutex, механизм доступа к общим переменным нашей программы.





Лекция 7. Разработка параллельных программ. Проблемы балансировки загрузки.

1. Языки и средства параллельного программирования.

А) Языки программирования делятся на 3 группы:

1) стандартные последовательные языки, поддерживающие специальные библиотеки функций для организации параллельных вычислений (Fortran 77, Pascal, C/C++/C#);

2) специализированные языки, включающие операторы или директивы скалярной, векторной, матричной и параллельной обработки (Occam, Fortran 90/95, HPF, MPF, MPC, MC#, TC#);

3) шейдерные языки программирования для спецвычислителей (например, графических процессоров и ПЛИС'ов, CUDA, OpenCL и др.).

Б) Среды разработки параллельных программ: Intel Parallel Studio, CUDA Toolkit и NVIDIA Nsight, ROCm (AMD), MS Visual Studio, Eclipse, Qt Designer, Boost и др.

В) Системные средства ОС – каналы межпроцессорного обмена, сокеты, средства порождения конкурентных или параллельных процессов, средства создания и использования разделяемых ресурсов.

Г) Официальные стандарты организации параллельных программ: POSIX, PVM, MPI, OpenMP.

Модели, библиотеки и средства – ShMem (Cray, глобальная адресация памяти), NUMA (англ. Non-Uniform Memory Access – неравномерный доступ к памяти) и ccNUMA (когерентный кэш памяти), CUDA (Nvidia, программирование ГПУ), MapReduce (Google). ПО ИПМ для автоматизации распараллеливания – DVM (distributed virtual machine), DVMH (hybrid DVM), NORMA, SAPFOR.

Д) Элементы параллельного программирования – идеи, методы, примеры алгоритмов и программ, в том числе: контексты и нити, синхронные и асинхронные каналы обмена данными, виртуальные объекты (в том числе: виртуальные процессоры, память, коммуникации, топологии, ресурсы).

2. Общие направления разработки параллельных программ.

1) выбор оптимальной последовательности алгоритмов $\{A_k\}$, критерии:

а) макс. быстродействие (минимальное время решения для каждого реализуемого k),

б) макс. масштабируемость (чем больше данных, тем больше вычислителей можно взять),

в) универсальность и однородность кода ($\forall k, p: \quad A_k = A_p$).

2) определение объема данных: N ~ p или N >>p;

3) определение модели памяти МВС: распределенная, разделяемая, гибридная (удаленные и локальные процессы), иерархическая;

4) выбор платформы: аппаратные средства, язык программирования, ***коммуникационная*** среда;

5) определение способов ввода-вывода исходных, промежуточных и результирующих данных;

6) выбор каналов связи и топологии обменов;

7) применение виртуальных процессоров (вычислителей) и виртуальных топологий обменов;

8) отладочная информация (протоколирование) и способы тестирования.

9) решение проблемы балансировки загрузки процессоров;

3. Параллельный алгоритм и структура программы.

Каков параллельный алгоритм – одноэтапный (а) или многоэтапный (б):

а) $Q_{посл} = Q_0 + Q_1 + \ldots + Q_n \le Q_0 + n \cdot q$, $\quad Q_{пар} = Q_0 + \max\limits_{k=1..n} Q_k \le Q_0 + q$;

б) $Q_{посл} = \sum\limits_m Q_{посл}^{(m)}, \quad Q_{посл}^{(m)} = Q_0^{(m)} + Q_1^{(m)} + \ldots + Q_{n^{(m)}}^{(m)} \le Q_0^{(m)} + n^{(m)} \cdot q^{(m)}, \quad Q_{пар} = \sum\limits_m Q_{пар}^{(m)}, \quad Q_{пар}^{(m)} \le Q_0^{(m)} + q^{(m)}$;

на каждом этапе своя степень параллелизма.

ПА должен быть наилучшим хотя бы по одному из указанных выше критериев.

Любая программа содержит три последовательные части:

1) ***инициализацию*** (настройка окружения, коммуникаций, ввода/вывода исходных данных);

2) ***расчетную часть*** (вычисления, обмены данными, ввод-вывод);

3) ***сохранение результатов и окончание работы***.

4. Выбор платформы.

Модель ВС – мультипроцессорная система, мультикомпьютерная система, смешанная система.

В каждом классе – однородная, неоднородная или гибридная архитектура.

Память: общая, распределенная, гибридная, иерархическая.

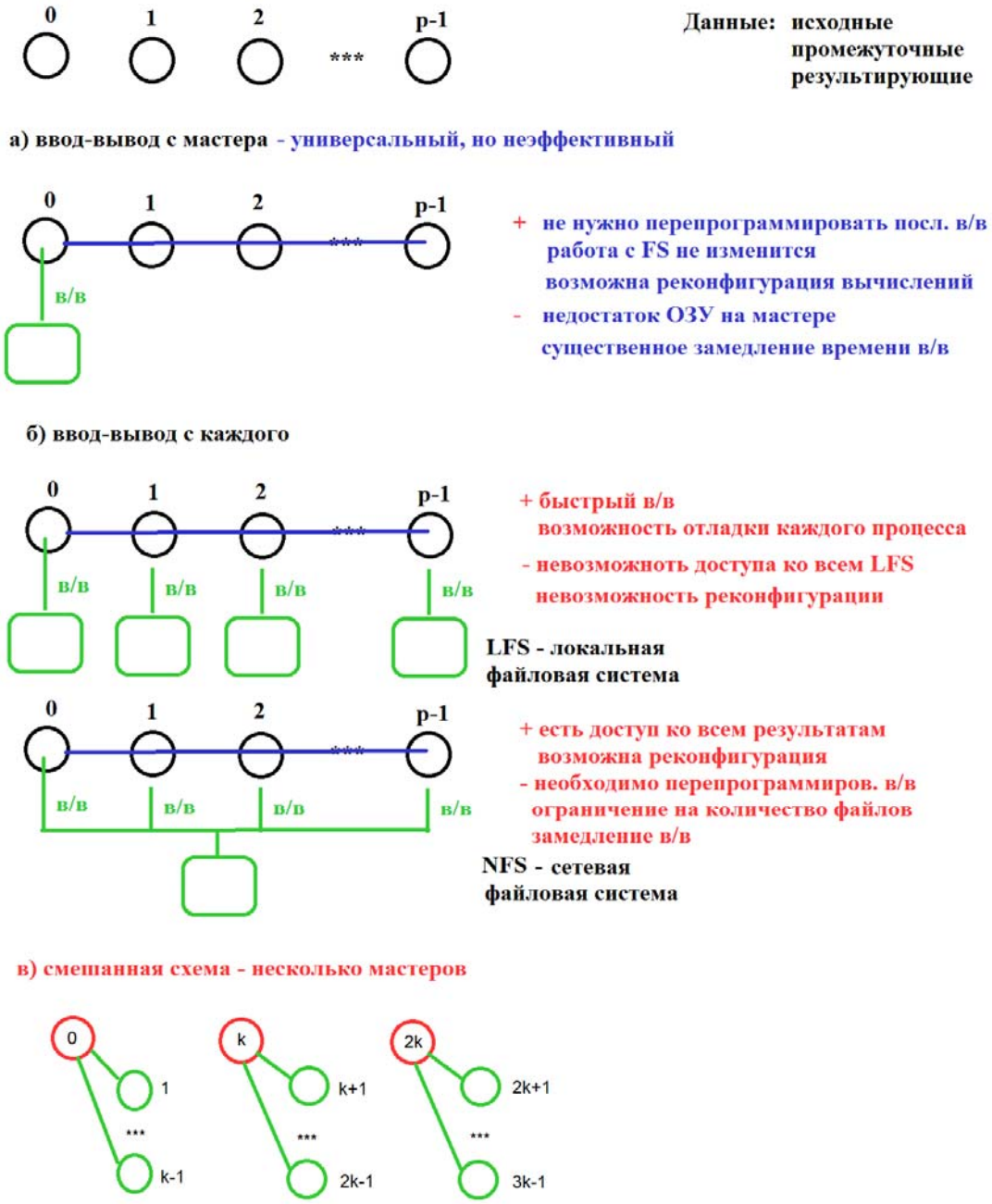
Проблема связи между различными типами памяти. Память ЦПУ, ГПУ. ПЛИСов.

Синхронизация и защита памяти.

Каналы связи - ***синхронные и асинхронные*** (последние бывают ***буферизованные и небуферизованные***). Топологии обменов бывают реальные и виртуальные. Пример: организация топологий в MPI.

5. Ввод-вывод данных.

а) ввод-вывод с мастера (для малого числа процессоров);
б) ввод-вывод с каждого процессора (большие поля данных);
в) смешанная схема ввода-вывода (несколько мастеров).



Системы хранения данных: LFS (локальные диски вычислителей), NFS (сетевая), PFS (параллельная).

Ввод-вывод с мастера – универсальный подход.

Применим даже там, где не все процессоры имеют доступ к диску.

Преимущество: единая мнемоника файловых имен (main.px, main.d, main.r, main.p, main.w, …), малое время записи данных на общий диск (например, из-за скорости сети).

Недостаток: если объем данных превышает размер ОЗУ одного процессора, то приходится организовывать последовательную схему записи и чтения.

Ввод-вывод с каждого процессора может осуществляться двумя способами:

1) запись-чтение всеми одного общего файла (конфликт по записи + дополнительное время определения своего фрагмента);

2) запись-чтение каждым своей группы файлов (более устойчивая схема, но нельзя изменить число процессоров при продолжении расчетов без доп. действий, сильно увеличивается нагрузка на файловую систему по количеству файлов и размеру директорий).

Смешанная схема ввода-вывода – несколько мастеров и несколько уровней специализации. Очень гибкая схема. В любом варианте необходимо обеспечивать масштабируемость.

6. Компиляция и запуск программ.

Компиляция:
1) прямое обращение к компилятору (cc, CC, gcc, icc, f77, ifc, …);
2) использование скриптов (mpicc, mpicxx, mpif77);
3) обращение через makefile: make –f main.mkc или nmake /f main.mak.

Компиляция на собственном или удаленном узле.

Запуск:
1) непосредственный: >main.px
2) с помощью среды: >mpirun –np 10 –machine hosts main.px

7. Отладка и протоколирование.

При отладке и в процессе массовых расчетов необходимо иметь контрольную информацию с каждого процессора, причем выводить ее не на экран, а в отдельный файл. Нужна система именования файлов протокола (main000.p, main001.p, …) и различная степень детализации информации (lp – параметр детализации информации). Основная проблема – емкость файловой системы => информация должна быть компактной и интегральной.

При тестировании программы в целом необходимо провести следующие тесты:
а) тест на точность решения в последовательном и параллельном вариантах;
б) тест на сильную масштабируемость (фиксирован объем задачи, приходящийся на все вычислители);
в) тест на слабую масштабируемость (фиксирован объем задачи, приходящийся на один вычислитель).

8. Проблемы балансировки загрузки.

Причины, приводящие к дисбалансу:
1) неоднородность алгоритма;
2) неоднородность ВС на аппаратном уровне;
3) неустойчивость работы АПС (процессоры, каналы связи, ОС и т.д.).

Что делать? Ответ: 1) установить наличие и причину дисбаланса; 2) устранить или ослабить дисбаланс. Для реализации п. 1) необходимо иметь механизм замера времени выполнения одинаковых фрагментов кода. Для реализации п. 2) необходимо использовать один из способов балансировки загрузки.

Виды балансировки: статическая и динамическая (централизованный и децентрализованный алгоритм).

Пример. Обработка данных в продолжительном цикле. На каждом шаге цикла с помощью p вычислителей обрабатывается N данных. Обозначим t_1, …, t_p – времена расчета шага цикла на соответствующем вычислителе, $t_s = (t_1+\ldots+t_p)/p$ – среднее время на шаг цикла, $\sigma=max|(t_k-t_s)/t_s|*100\%$ – дисперсия (разброс) времен. Задача балансировки состоит в минимизация дисперсии.

Решение с применением статической балансировки загрузки.

Можно заранее узнать производительности вычислителей $q_1, \ldots, q_p$. Тогда на первом шаге цикла можно распределить N данных в соответствии с производительностью вычислителей:

$N_1/N_2 = q_1/q_2, \ldots, N_{p-1}/N_p = q_{p-1}/q_p, N_1+\ldots+N_p = N \Rightarrow N_1=N*q_1/(q_1+\ldots+q_p), \ldots, N_p=N*q_p/(q_1+\ldots+q_p)$.

Это распределение оставить неизменным на последующих шагах цикла.

Решение с применением динамической балансировки загрузки.

Если информация заранее неизвестна или ситуация меняется, то производительности вычислителей можно определять на каждом шаге цикла: $q_1=N_1/t_1, \ldots, q_p=N_p/t_p$. Затем определять новые значения $N_1, \ldots, N_p$ для следующего шага цикла. При этом используются два алгоритма:

а) точный алгоритм: $N_k^{'} = Nq_k / \left(q_1 + \ldots + q_p\right)$;

б) приближенный диффузный алгоритм: $N_k^{'} = \left(1-\tau\right)N_k + \tau N q_k / \left(q_1 + \ldots + q_p\right), \ \tau \sim 0.05$.

Лекция 8. Концепция гибридной параллельной вычислительной платформы

1. Предварительные замечания

Особенностью современного этапа развития прикладных параллельных программных средств математического моделирования является переход к гибридным решениям. Причиной этому послужило широкое распространение гибридных компьютерных архитектур и развитие программных сред и языков программирования, ориентированных на специальные вычислители, среди которых в первую очередь следует отметить графические ускорители.

В настоящей работе сформулирована концепция гибридных параллельных вычислений, которую можно положить в основу программной части гибридной суперкомпьютерной платформы (технологии) – ГСКП (ГСКТ). Среди множества вопросов, примыкающих к этой проблеме, были рассмотрены два: алгоритмы решения задач и особенности их реализации на доступных широкому кругу пользователей вычислительных системах.

2. Особенности современных аппаратно-программных средств

Для достижения поставленных целей обратимся к обзору аппаратных и программных средств, имеющихся в наличии у большинства исследователей. В качестве аппаратных решений будем различать три класса вычислительных систем (ВС):

1) настольные или мобильные персональные компьютеры пользователей (ПКП),

2) бюджетные кластеры лабораторий (БКЛ) и небольших организаций,

3) суперкомпьютерные вычислительные системы (СКВС) крупных организаций и центров коллективного пользования (ЦКП).

По сути, эти решения отличаются количеством независимых вычислительных узлов и способами их интеграции в единую вычислительную систему. Архитектура современного вычислительного узла этих систем де-факто является гибридной. Рассмотрим её подробнее.

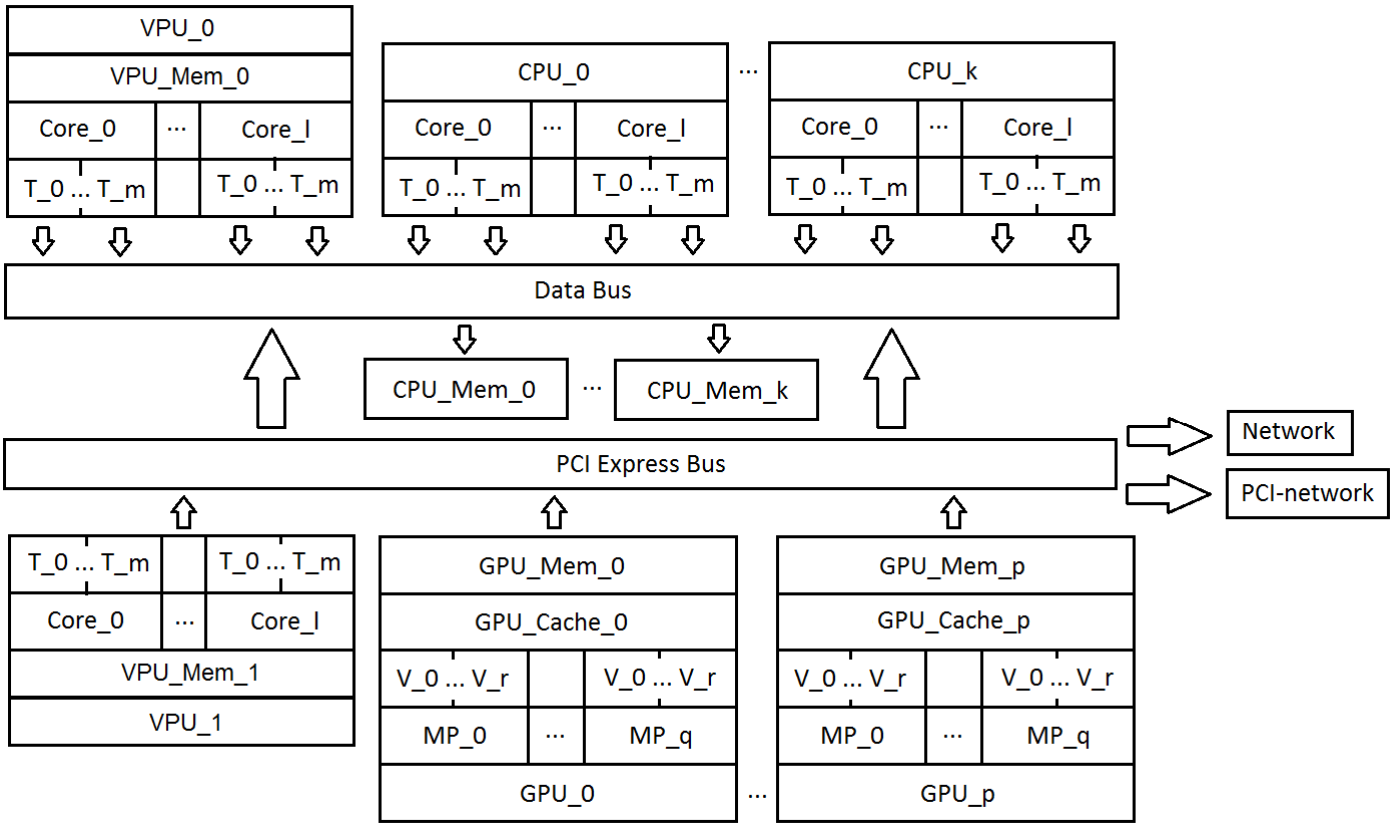


Рис.1. Гибридная архитектура современного вычислительного узла кластера или персонального компьютера.

На Рис. 1 показана гибридная архитектура современного вычислительного узла, характерная практически для любой современной компьютерной системы. Здесь предполагается, что узел имеет k центральных (ЦПУ/CPU) и p графических (ГПУ/GPU) процессоров, а также k банков общей оперативной памяти, тесно связанных каждый с одним ЦПУ, но доступных всем ЦПУ.

Каждый ЦПУ имеет l ядер, поддерживающих по m конвеерных устройств обработки данных, и доступ к любому банку оперативной памяти через общую шину данных.

Каждый ГПУ имеет q мультипроцессоров и $q*r*n$ ядер, сгруппированных в r блоков по n конвеерных устройств обработки данных, и доступ к собственной оперативной памяти (прямой доступ), к памяти других ГПУ (в том числе внешних) по шине PCI Express (NVlink в новых ВС), а также к оперативной памяти своего узла. В отличие от ЦПУ в графических ускорителях имеется также возможность управления кэш-памятью.

При использовании подобного вычислительного узла в настольном исполнении получаем мощную рабочую станцию, вполне пригодную для проведения научных расчётов среднего уровня. При объединении небольшого количества (порядка 10-50) узлов с подобной архитектурой с помощью среднескоростной сети (например, Gigabit Ethernet, в том числе сдвоенный или счетверенный) получаем бюджетный кластер террафлопного диапазона производительности. Если для объединения применяется одна или несколько высокоскоростных сетей (10/40/100-Gigabit Ethernet, 56/100/200-Gigabit InfiniBand, 50/100-Gigabit Omni Path и т.д.), а число узлов измеряется сотнями и более, получаем суперкомпьютерную систему вплоть до экзофлопсного уровня производительности.

С точки зрения управляющей программной среды гибридных вычислительных систем (ГВС) ситуация складывается следующим образом. В настольных и мобильных аппаратных решениях в основном используется 64х-битная операционная система Windows (далее ОС Windows), а также Mac OS X. Существенно реже встречается операционная система UNIX в версии Linux (далее ОС Linux). Для реализации кластерных решений используются в основном ОС Linux, существенно реже ОС Windows в версии Windows Server или Windows Cluster Server. Таким образом, можно отметить, что подавляющее число современных вычислительных установок управляется ОС Windows или ОС Linux.

3. Постановка задачи

Специфика разработки неспециальных приложений для вычислителей с универсальной центральной архитектурой отличается тем, что прикладному программисту фактически не требуется знать детальную структуру узла, а лишь использовать количество ядер l или параллельных потоков $l*m$, а также объём доступной оперативной памяти, которые легко определить во время исполнения кода. При этом инициализация многопоточного приложения внутри вычислительного узла производится за очень короткое время (порядка нескольких микросекунд) и может повторяться многократно в течение всего цикла решения задачи (так, например, происходит при использовании технологии OpenMP). Ввод-вывод с каждой нити приложения осуществляется также, как в однопоточном приложении, и не вызывает больших проблем.

Разработка любого приложения для вычислителей с гибридной архитектурой, напротив, требует детального знания об устройстве узлов. При этом только часть информации можно получить во время исполнения кода. В этой ситуации требуется изначально ориентироваться на конкретную аппаратную платформу. Время доступа к различным типам памяти гибридного узла сильно отличается. Количество одновременно исполняемого кода и обрабатываемого массива данных сильно ограничено. Ввод-вывод данных возможен напрямую только из тех нитей приложения, которые выполняются на ЦПУ. Следует также отметить, что время инициализации ГПУ достаточно велико (порядка нескольких секунд), поэтому инициализация этих устройств производится один раз в начале цикла вычислений. Эти и другие особенности гибридных вычислительных систем приходится учитывать при разработке конкретных приложений.

Для упрощения дальнейшего анализа была конкретизирована аппаратно-программная платформа. Ее основу составляет архитектура ЦПУ-ГПУ на базе центральных процессоров Intel (или AMD) и графических ускорителей NVIDIA (или AMD). Данное сужение класса вычислителей не является фатальным, поскольку они составляют более 90% используемого парка вычислительных систем и доступны практически любому пользователю. Однако это сужение позволяет большую часть анализа аппаратуры проводить во время исполнения программы и разработать общие подходы к разработке приложений.

Основная цель предлагаемого исследования – сформировать подход к разработке параллельных приложений для такого рода систем, который удовлетворял бы следующим требованиям:

1) разработанный параллельный код позволяет выполнять расчёты задачи на ГВС из указанных выше трёх классов: ПКП, БКЛ, СКВС;

2) разработанный параллельный код легко переносится с одной аппаратно-программной платформы на другую;

3) для разработки параллельного кода потребуется только свободно распространяемое или условно свободно распространяемое программное обеспечение (ПО).

4. Гибридная параллельная вычислительная платформа

Предлагаемая концепция параллельных вычислений на ГВС указанного класса выработана на основе анализа литературы и опыта авторов в разработке кросс-платформных приложений.

Сформулируем общие предложения концепции гибридных параллельных вычислений.

П1. В основе любого параллельного решателя, реализующего прикладную задачу, должно находиться простое MPI-приложение, ориентированное на распределённую модель вычислений. Распределённая модель вычислений выбрана в связи с тем, что требуется создавать приложения, ориентированные на произвольное сколь угодно большое количество вычислителей. К тому же поддержка MPI-программ реализована на большинстве аппаратно-программных платформ, и перенос таких программ с одной платформы на другую не вызывает особых затруднений.

П2. Основные языки программирования, которые обычно используются для реализации MPI-приложений: C, C++, Fortran. Естественно остановиться на этом множестве и в данном случае. Однако версии самих языков требуют уточнения.

В качестве единой свободно распространяемой версии компиляторов с языков C, C++, Fortran в случае 64-битной адресации можно использовать GNU C/C++/Fortran. Однако это решение подойдёт только для ОС Linux. Дело в том, что для 64-битной версии ОС Windows реализован только компилятор GNU C/C++ (в пакете MinGW64) и компилятора с языка Fortran в нём нет. Другой проблемой является то, что свободно распространяемый компилятор C/C++ для GPU (пакеты CUDA Toolkit или AMD ROCm) в ОС Linux является надстройкой над GNU C/C++. В ОС Windows пакет компиляторов CUDA Toolkit (AMD ROCm) базируется на полукоммерческой среде Microsoft Visual Studio и требует её наличия на компьютере. Поэтому в ОС Windows приходится использовать Microsoft Visual C/C++, а в ОС Linux можно использовать свободно распространяемый компилятор GNU C/C++. Свободно распространяемый 64-битный компилятор с языка Fortran для ОС Windows отсутствует, в ОС Linux это GNU Fortran. Заметим, что полное кроссплатформное решение имеется у Intel и IBM для трёх языков C/C++/Fortran, однако оно будет платным.

П3. Выбор реализации MPI зависит от двух параметров. Во-первых, от типа сетевого интерфейса между узлами МВС. Во-вторых, от типа операционной системы. В первом случае имеются следующие альтернативы: в качестве сетевого решения может использоваться Ethernet, InfiniBand, Myrinet, SCI, PCI Express, NVLINK, OMNI Path и другая аппаратура связи. Поскольку первые два типа интерфейсов захватили большую часть рынка коммуникаций, то можно ориентироваться только на них. При этом среди различных реализаций MPI выделяются две группы: реализации, поддерживающие только один тип коммуникаций (***гомогенная реализация***), и реализации, поддерживающие сразу несколько сетевых интерфейсов (***гетерогенная реализация***). В первую группу можно поместить MPICH, MPICH2 и LAMMPI (поддерживают коммуникации в сетях Ethernet), MVAPICH и MVAPICH2 (поддерживают коммуникации в сетях InfiniBand). Также сюда попадают реализации MPI от Intel, IBM, NVidia, Melanox. Во вторую группу попал OpenMPI. Он поддерживает множество способов коммуникаций, в том числе Ethernet и InfiniBand, а также гетерогенность.

Если далее рассмотреть фактор, связанный с типом ОС, то выяснится, что практически все реализации MPI имеют версии для ОС Linux, и только некоторые из них имеют версии для Windows. С точки зрения создания свободно распространяемого ПО следует использовать MPICH, который имеет реализации для ОС Windows и Linux. Однако это решение подойдёт только для сетей Ethernet. Условно бесплатной альтернативой MPICH под ОС Windows является MS MPI, входящий в пакет Microsoft HPC Pack (версии 2008/2012/2016/2019). В случае интерфейса InfiniBand для ОС Linux имеем 2 решения: MVAPICH и OpenMPI. Коммерческий вариант для обеих ОС и обоих типов сетевого интерфейса имеется только у Intel и IBM. Важно также отметить, что с точки зрения содержания MPI-программы ***конкретная реализация MPI не влияет на итоговый текст создаваемого параллельного приложения***.

П4. Поскольку каждый узел рассматриваемого семейства вычислительных систем имеет многоядерную архитектуру, то естественно использовать дальнейшее распараллеливание прикладной задачи по ядрам (нитям ядер – трэдам) CPU на общей памяти внутри узла. Реализацию этой идеи можно производить с помощью технологий OpenMP или PThreads. Оба эти средства имеют свои преимущества и недостатки. Однако общая их особенность состоит в том, что они встроены в язык программирования высокого уровня. При этом функции библиотеки PThreads являются неотъемлемой частью стандарта

языка ANSI C/C++, а интерфейс OpenMP включён как в ANSI C/C++, так и в Fortran. Комбинируя оба подхода, можно создавать весьма гибкие прикладные параллельные программы.

Версии интерфейсов OpenMP и PThreads зависят от версий компиляторов и runtime-библиотек ОС. В частности, библиотека процессов PThreads, является неотъемлемой частью ядра ОС Linux. При использовании ОС Windows имеется её аналог (условно назовём его WThreads), который отличается по интерфейсу, но не по функциональным возможностям. В итоге, текст параллельной программы в этом случае зависит только от сделанного выбора: OpenMP или PThreads.

П5. Во многих случаях производительность параллельного приложения можно существенно увеличить, если наиболее затратные части кода выполнять на ГПУ. Для реализации такой аппаратной возможности предлагается использовать технологию CUDA от NVidia или ROCm от AMD. Остановимся на первом варианте. Технология CUDA состоит из двух основных частей. Во-первых, это специальный драйвер устройства (в дальнейшем CUDA driver), обеспечивающий управление ГПУ со стороны ОС и других приложений (версии драйвера поставляются для большинства аппаратно-программных платформ). Во-вторых, это компилятор с языка C/C++, входящий в CUDA Toolkit, позволяющий писать либо отдельные программы для GPU, либо фрагменты кода для ГПУ, встраиваемые в код для ЦПУ. Использование технологии CUDA позволяет задействовать все ресурсы вычислительного узла, в том числе ресурсы всех ЦПУ и всех ГПУ одновременно.

Альтернативой технологии CUDA является технология OpenCL, позволяющая разрабатывать многопоточные приложения для вычислителей с различной архитектурой, в том числе, отличной от ЦПУ и ГПУ. В частности, в рамках этой технологии могут быть созданы программы для вычислений на графических картах компании AMD (ATI Radeon) и других производителей. Однако эта технология вцелом не является пока универсальной (конкурирующие версии производятся IBM, Intel, NVidia, AMD). В этой ситуации OpenCL, разработанный сторонними производителями, плохо реализуется на графических процессорах компании NVidia, которые наиболее распространены во всём мире. По этим причинам данная технология пока не рекомендуется к использованию в рамках создаваемой Гибридной параллельной вычислительной платформы.

П6. Реализация технологии CUDA накладывает определённые требования на разработку приложения.

Во-первых, компилятор CUDA C/C++ является надстройкой над Microsoft Visual C/C++ в ОС Windows и надстройкой над GNU C/C++ в ОС Linux. Поэтому предустановка этих компиляторов на вычислителях становится необходимой.

Во-вторых, использование нескольких ГПУ на узле требует создания соответствующего количества нитей ЦПУ, управляющих отдельными ГПУ. Тем самым использование технологии CUDA тесно связано с технологиями OpenMP или PThreads.

В-третьих, передача данных между отдельными ГПУ в рамках одного узла может осуществляться либо через основную память посредством ЦПУ, либо непосредственно по каналам прямого доступа шины PCI Express (NVlink). Реализация первого механизма не требует привлечения дополнительных средств программирования. Однако в этом случае имеется существенное снижение скорости передачи данных между различными ГПУ. Одна из реализаций второго механизма состоит в использовании библиотеки Shmem, с помощью которой можно организовать единое адресное пространство памяти всех ГПУ, в том числе располагающимися за пределами узла (в других узлах). В последних версиях CUDA Toolkit реализована более эффективная модель второго механизма обмена – GPUDirect.

В-четвёртых, при разработке MPI-приложений, использующих вычисления на ГПУ, компиляция функций для ГПУ и так называемых "обёрток" производится компилятором CUDA C/C++ (nvcc), а сборка приложения в целом производится средствами пакета MPI для ЦПУ. В последних версиях CUDA Toolkit всю сборку можно осуществить одним и тем же компилятором nvcc в связи с тем, что появилась совместная реализация MPI от Melanox и NVidia.

П7. При реализации параллельной программы решения какой-либо задачи на гибридном вычислителе следует, прежде всего, определить модель размещения и обработки основных массивов данных: 1) только в памяти ЦПУ; 2) только в памяти ГПУ; 3) смешанный способ хранения.

В первом случае подразумевается, что ГПУ используется только как сопроцессор, обрабатывающий небольшие но вычислительно ёмкие блоки данных.

Во втором случае подразумевается, что вся задача укладывается в память одного или нескольких ГПУ, а ЦПУ используется только для загрузки ГПУ и реализации операций ввода-вывода.

В третьем случае подразумевается, что задача настолько велика по объёму, что суммарной памяти ГПУ недостаточно для размещения всех обрабатываемых данных.

П8. Выбор модели размещения и обработки данных определяет два важных параметра параллельной программы.

Первый параметр – тип основного интерфейса передачи данных между вычислителями. В случае 1) – передача данных по сети, в случае 2) – передача данных по шине PCI Express (NVlink), в случае 3) – смешанная схема обменов.

Второй параметр – тип используемого средства программирования коммуникаций. В случае 1) это MPI, в случае 2) – Shmem или CUDA GPU Direct, в случае 3) – использование обоих средств.

П9. Итоговая реализация гибридного параллельного приложения выполняется с помощью одной из следующих параллельных технологий:

1) MPI (простое MPI-приложение, использующее только ЦПУ и MPI-коммуникации как между узлами, так и внутри каждого узла),

2) MPI+OpenMP (MPI-приложение, использующее только ЦПУ и MPI-коммуникации между узлами, а также распараллеливание на общей памяти внутри узлов с помощью технологии OpenMP),

3) MPI+PThreads (MPI-приложение, использующее только ЦПУ и MPI-коммуникации между узлами, а также распараллеливание на общей памяти внутри узлов с помощью технологии PThreads),

4) MPI+OpenMP+CUDA (MPI-приложение, использующее ЦПУ и ГПУ, MPI-коммуникации между узлами, распараллеливание на общей памяти ЦПУ внутри узлов с помощью технологии OpenMP и распараллеливание внутри ГПУ с помощью технологии CUDA),

5) MPI+PThreads+CUDA (MPI-приложение, использующее ЦПУ и ГПУ, MPI-коммуникации между узлами, распараллеливание на общей памяти ЦПУ внутри узлов с помощью технологии PThreads и распараллеливание внутри ГПУ с помощью технологии CUDA),

6) MPI+OpenMP+CUDA+Shmem или MPI+OpenMP+CUDA+GPUDirect (MPI-приложение, использующее ЦПУ и ГПУ, MPI-коммуникации между узлами, распараллеливание на общей памяти ЦПУ внутри узлов с помощью технологии OpenMP и распараллеливание внутри ГПУ с помощью технологии CUDA, коммуникации между ГПУ различных узлов с помощью функций библиотеки Shmem),

7) MPI+PThreads+CUDA+Shmem или MPI+PThreads+CUDA+GPUDirect (MPI-приложение, использующее ЦПУ и ГПУ, MPI-коммуникации между узлами, распараллеливание на общей памяти ЦПУ внутри узлов с помощью технологии OpenMP и распараллеливание внутри ГПУ с помощью технологии CUDA, коммуникации между ГПУ различных узлов с помощью функций библиотеки Shmem).

8) MPI+CUDA+GPUDirect или CUDA+GPUDirect – современные технологии.

Замечание 1. Вместо технологии CUDA от NVidia во многих случаях уже можно с успехом использовать технологию ROCm от AMD.

Замечание 2. Наряду с векторными операциями ЦПУ и ГПУ возможно использовать режим вычислений TensorFlow. Он аппаратно поддерживается в разных вариантах на платформах Intel, IBM, Nvidia, AMD. TensorFlow – открытая программная библиотека для машинного обучения, разработанная компанией Google для решения задач построения и тренировки нейронных сетей. Библиотека TensorFlow также часто применяется для решения прикладных задач оптимизации и распознавания образов.

Литература

1. Message Passing Interface (MPI) Forum Home Page. http://www.mpi-forum.org/
2. OpenMP Home Page. http://openmp.org/wp/
3. NVidia CUDA Toolkit Home Page. https://developer.nvidia.com/cuda-toolkit
4. AMD ROCm Home Page. https://rocm.docs.amd.com/en/latest/
5. Боресков А.В., Харламов А.А., Марковский Н.Д. и др. Параллельные вычисления на GPU. Архитектура и программная модель CUDA. – М., Изд-во Московского ун-та, 2012. – 336 с.
6. Сандерс Дж., Кэндрот Э. Технология CUDA в примерах: введение в программирование графических процессоров. М.: ДМК Пресс, 2011. - 232 с.
7. NVIDIA CUDA C Programming Guide. Version 4.0. Santa Clara (CA, USA): NVIDIA Corporation, 2011. - 187 p.
8. Боресков А.В., Харламов А.А. Основы работы с технологией CUDA. М.: ДМК Пресс, 2010. - 232 с.

9. Logan S. Cross-platform development in C++ : building Mac OS X, Linux, and Windows applications. Crawfordsville (Indiana, USA): RR Donnelly, 2007. - 575 p.
10. Стефенс Д.Р., Диггинс К., Турканис Д., Когсуэлл Д. C++. Сборник рецептов. М.: КУДИЦ-ПРЕСС, 2007. - 624 с.
11. Ортега Дж. Введение в параллельные и векторные методы решения линейных систем. М.: Мир, 1971. - 367 с.
12. Воеводин В.В., Воеводин Вл.В. Параллельные вычисления. СПб.: БХВ-Петербург, 2002. - 608 с.
13. Корнеев В.Д. Параллельное программирование в MPI. - Москва-Ижевск, Институт компьютерных исследований, 2003.
14. Dongarra J., Foster I., Fox J., et al. Sourcebook of Parallel Computing. San Francisco (CA, USA): Elsevier Science, 2003. - 852 p.
15. Гергель В.П. Теория и практика параллельных вычислений. СПб.: "Интернет-университет информационных технологий - ИНТУИТ.ру", "БИНОМ. Лаборатория знаний", 2007. - 424 с.

Лекция 9. Математическое моделирование и параллельные вычисления. Основные классы задач. Решение алгебраических уравнений и ОДУ.

1. Вычислительная и прикладная математика.

Вычислительная математика – один из разделов математики, использующий в качестве основного метода решения математических задач приближённые вычисления. Классы задач вычислительной математики фактически совпадают с классами задач аналитической математики:

а) дискретизация объектов, приближение функций (интерполяция, экстраполяция, аппроксимация);

б) дифференцирование и интегрирование функций;

в) решение уравнений различных типов (алгебраических, обыкновенных дифференциальных, в частных производных, интегральных, интегро-дифференциальных) и их систем;

г) нахождение минимумов и максимумов функций и функционалов;

д) решение задач оптимального управления;

е) решение спектральных задач;

ж) решение геометрических задач.

Вычислительная математика условно делится на две части: развитие аппарата численных методов и применение численных методов к решению математических задач.

Прикладная математика – раздел математики, исследующий с помощью аналитических и численных методов математические задачи, возникающие в естественных и гуманитарных науках, а также в инженерных приложениях. В части решения математических задач с помощью численных методов вычислительная и прикладная математика совпадают. Далее будем рассматривать в основном именно эту общую часть.

2. Методология математического моделирования и параллельные вычисления.

Математическое моделирование – современная научная методология исследования объектов, процессов и явлений как окружающего нас мира, так и различных виртуальных абстракций. Основной принцип ММ – замена исходного объекта исследования математической моделью. Более точно:

Математическое моделирование – современная методология познания, использующая математический аппарат и предполагающая разработку, реализацию, проверку и применение математических моделей.

ММ ***базируется на методах прикладной математики***, однако заключает в себе и другие методики, например, сбор и анализ информации об объектах исследования, развитие аппарата математических описаний и математического анализа конкретных задач, разработку алгоритмов и программ для вычислительных систем, компьютерную обработку данных.

Современная реализация математического моделирования с помощью вычислительной техники есть ***вычислительный (компьютерный) эксперимент***. Последний заключает в себя бесконечно повторяющуюся последовательность следующих этапов исследования объектов (процессов, явлений):

1) сбор информации об объекте;

2) формулирование математических описаний (моделей) объекта;

3) анализ математических описаний объекта;

4) сравнение полученных данных с оригиналом.

Поскольку в настоящее время ни один из этих этапов не обходится без применения компьютерной обработки данных, то и ***параллельные вычисления*** в этом контексте являются одним из этапов и методов математического моделирования.

3. Задачи, требующие применения параллельных вычислений.

Если проанализировать основные классы задач вычислительной и прикладной математики, то окажется, что большинство из них сводится к ***решению уравнений различных типов*** (алгебраических, дифференциальных, интегральных, функциональных, смешанных), а также их систем. При этом в качестве области задания уравнений могут выступать как континуальные, так и дискретные множества и пространства. Наиболее часто при моделировании используются ***уравнения математической физики***.

В узком смысле предмет математической физики – анализ моделей на основе уравнений в частных производных. Чаще всего это уравнения в частных производных 2-го порядка, разделяющиеся на три класса: а) гиперболические; б) параболические; в) эллиптические. ***Почему***: они хорошо описывают большинство процессов и явлений, исследующихся математическими методами.

В широком смысле математическая физика имеет дело ***со всеми математическими задачами***, потому что они описывают тот или иной физический процесс или явление.

Параллельные вычисления возникают тогда, когда задачи МФ решаются численно в дискретных пространствах большой размерности. Поэтому предметом дальнейшего рассмотрения будут параллельные алгоритмы и их реализация при численном решении задач МФ или связанных с ними других математических задач.

4. Этапы вычислительного эксперимента применительно к задачам математической физики.

1) дискретизация непрерывного объекта;
2) построение численной схемы;
3) реализация численной схемы (алгоритм решения);
4) параллельный алгоритм решения;
5) параллельная программа;
6) расчеты на МВС;
7) обработка результатов вычислений.

5. Решение алгебраических уравнений.

5.1. Случай одного уравнения.

Поиск корней уравнения $f(x)=0$ сопровождается множественным вычислением функции $f(x)$.

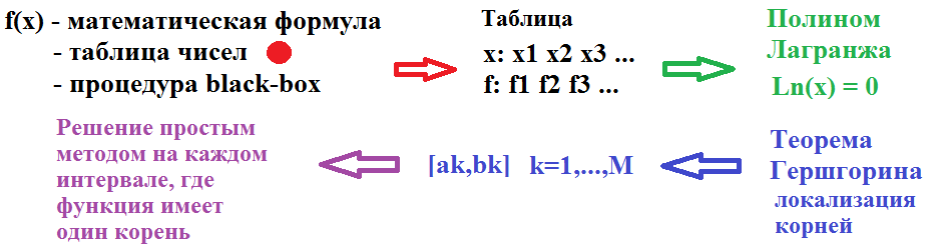
Наиболее общая постановка задачи: $f(x)=0, \quad x\in(-\infty,+\infty)$.

Проблема бесконечного интервала решается преобразованием Римана:

$$(-\infty,+\infty)=(-\infty,-A)\cup[-A,+A]\cup(+A,+\infty);$$

$$(-\infty,-A):\quad x'=-1/x,\quad g(x')=f(-1/x),\quad x'\in(0,1/A);$$

$$(+A,+\infty):\quad x'=+1/x,\quad h(x')=f(+1/x),\quad x'\in(0,1/A).$$



Вывод: во многих случаях задачу можно разделить на фиксированное число независимых ветвей (функциональный параллелизм). Кроме того, можно одновременно вычислять несколько значений $f(x)$ в методах высокого порядка точности. Если корней у уравнения больше одного, то каждый из них можно вычислять независимо.

Ресурс параллелизма – каждое независимое вычисление $f(x)$, сложность вычислений функции, вычисления коэффициентов и полинома Лагранжа, число изолированных корней.

5.2. Случай системы уравнений.

$\mathbf{F}(\mathbf{x})=0 \quad\Leftrightarrow\quad f_k(x_1,\dots,x_m)=0,\quad k=1,\dots,m;\quad \mathbf{x},\mathbf{F}\in\mathbb{R}^m.$

Система может быть линейная или нелинейная.

5.2.1. Методы решения линейных систем.

$\mathrm{A}\mathbf{x}=\mathbf{b},\quad \mathbf{x},\mathbf{b}\in\mathbb{R}^m,\quad \mathrm{A}\in\mathbb{R}^{m\times m}.$

В общем случае матрица системы является полностью заполненной. Для обращения матрицы можно использовать прямые методы и итерационные. Пример прямого метода – метод Гаусса. Пример итерационного метода – метод неполного разложения Холецкого.

5.2.2. Методы решения нелинейных систем.

Методы решения нелинейных систем являются итерационными. Они подразделяются на четыре основных класса: градиентные, координатные, стохастические, гибридные. Ресурс параллелизма – количество независимых вычислений в алгоритмах методов.

Градиентный метод: $\mathbf{x}_0$ – начальное приближение, $\mathbf{x}_{s+1}=\mathbf{x}_s-\alpha\left[\nabla\mathbf{F}(\mathbf{x}_s)\right]^{-1}\mathbf{F}(\mathbf{x}_s),\quad s=0,1,2,\dots$.

Координатный метод: $\begin{cases} f_k\left(x_1^*,\dots,x_k,\dots,x_m^*\right)=\varphi_k(x_k)=0,\quad k=1,\dots,m;\quad\Rightarrow\quad \mathbf{x}_s; \\ \left\|\mathbf{F}(\mathbf{x}_s)\right\|<\varepsilon\quad ?\quad (\textit{нет})\Rightarrow \textit{повтор процедуры.}\end{cases}$

$\mathbf{x}_1,...,\mathbf{x}_N$ – *последовательность независимых случайных векторов в* $D \subset \mathbb{R}^m$;

Стохастический метод: $\mathbf{x}_* = \mathbf{x}_{s_0}: \quad \left\|\mathbf{F}(\mathbf{x}_*)\right\| = \min\limits_{1 \le s \le N} \left\|\mathbf{F}(\mathbf{x}_s)\right\| \le \dfrac{C}{\sqrt{N}}.$

Гибридные: метод случайного градиента, градиентный метод покоординатного спуска и др.

Ресурс параллелизма – размерность пространства, сложность вычислений функции, порядок метода.

6. Решение ОДУ.

6.1. Постановки задач. Любое количество ОДУ разного порядка можно свести к системе уравнений первого порядка:

$\mathbf{F}(\dot{\mathbf{x}}, \mathbf{x}, t) = 0,$ или $\dot{\mathbf{x}} = \mathbf{F}(\mathbf{x}, t), \quad 0 < t < 1, \quad \mathbf{x}, \mathbf{F} \in \mathbb{R}^m.$

Покомпонентная формулировка:

$f_i\left(\dot{x}_1,...,\dot{x}_m, x_1,...,x_m, t\right) = 0, \quad i = 1,...,m.$

Задачи решения ОДУ бывают двух типов: начальные и краевые.

Начальная задача (задача Коши): $\mathbf{x}(0) = \mathbf{x}_0$.

Краевая задача: $\mathbf{A}_0\mathbf{x}(0) = \mathbf{x}_0, \quad \mathbf{A}_1\mathbf{x}(1) = \mathbf{x}_1, \quad \mathbf{A}_0, \mathbf{A}_1 \in \mathbb{R}^{m \times m}, \quad rank(\mathbf{A}_0) + rank(\mathbf{A}_1) = m.$

6.2. Методы решения задачи Коши.

Методы решения разделяются на аналитические, полуаналитические и приближенные.

Аналитические (теоретические методы) – поиск траектории $\mathbf{x}(t) = (x_1(t),...,x_m(t))$ в явном виде или в виде квадратур.

Полуаналитические методы – разложение траектории $\mathbf{x}(t)$ по базисным функциям $\xi_k(t)$, при котором коэффициенты разложения определяются приближенно.

Приближенные (численные) методы позволяют найти конечное множество точек траектории:

$\mathbf{x}(t), \quad t \in [0,1] \quad \to \quad \left\{\mathbf{x}(t_k), \quad k = 0,...,N\right\}.$

Методы с постоянным шагом: $t_k = \tau \cdot k, \quad k = 0,...,N, \quad \tau = 1/N.$

Формула Коши: $\mathbf{x}\left(t+\tau\right) = \mathbf{x}\left(t\right) + \int\limits_t^{t+\tau} \mathbf{F}(x(s), s)ds.$

Одношаговые методы.

Схема Эйлера: $\mathbf{x}(t+\tau) = \mathbf{x}(t) + \tau\mathbf{F}(\mathbf{x}(t), t)$ – явная схема 1-го порядка (формула левых прямоугольников).

Схема Рунге-Кутты: $\mathbf{x}(t+\tau) = \mathbf{x}(t) + \tau\mathbf{F}(\mathbf{x}(t) + 0.5\tau\mathbf{F}(\mathbf{x}(t), t), t + 0.5\tau)$ – явная схема 2-го порядка (формула средних прямоугольников).

Схема Адамса: $\mathbf{x}(t+\tau) = \mathbf{x}(t) + 0.5\tau\left(\mathbf{F}(\mathbf{x}(t), t) + \mathbf{F}(\mathbf{x}(t+\tau), t+\tau)\right)$ – симметричная неявная схема 2-го порядка (формула трапеций).

Многошаговые методы (явные и неявные).

Явный двухшаговый метод Адамса:

$\mathbf{x}(t+\tau) = \mathbf{x}(t) + \tau\left[1.5\mathbf{F}(\mathbf{x}(t), t) - 0.5\mathbf{F}(\mathbf{x}(t-\tau), t-\tau)\right]$ – квадратура + экстраполяция.

Неявный трехшаговый метод Адамса:

$$\mathbf{x}(t+\tau) = \mathbf{x}(t) + \tau\left[\frac{5}{12}\mathbf{F}(\mathbf{x}(t+\tau), t+\tau) + \frac{8}{12}\mathbf{F}(\mathbf{x}(t), t) - \frac{1}{12}\mathbf{F}(\mathbf{x}(t-\tau), t-\tau)\right].$$

Схемы с переменным шагом: методы Гира, Башфорта и др.

Ресурс параллелизма – размерность вектора $\mathbf{x}$, явность или неявность алгоритма, количество независимых вычислений вектора $\mathbf{F}$.

6.3. Методы решения краевых задач.

Метод стрельбы. Решаем начальную задачу с условиями:

$\mathbf{A}_0\mathbf{x}(0) = \mathbf{x}_0, \quad \mathbf{A}_1\mathbf{x}(1) = \mathbf{x}_1, \quad \mathbf{A}_0, \mathbf{A}_1 \in \mathbb{R}^{m \times m}, \quad rank(\mathbf{A}_0) + rank(\mathbf{A}_1) = m.$

Метод стрельбы состоит в том, что мы дополняем вектор $\mathbf{x}_0$ недостающими компонентами и решаем задачу Коши. Получаем траекторию $\mathbf{x}(t)$. Эта траектория должна удовлетворять второму краевому условию: $\mathbf{A}_1\mathbf{x}(1) = \mathbf{x}_1$. Это фактически нелинейное уравнение, заданное неявным образом. Его решаем с помощью итераций.

Вариант метода. Введем новый вектор параметров $\boldsymbol{\delta} = (\delta_1, ..., \delta_m)$. Переупорядочим вектор траектории $\mathbf{x}(t)$ и переформулируем граничные условия так, чтобы они превратились в равенства, и слева первыми компонентами были значения из вектора $\mathbf{x}_0$ (в количестве *k*), а справа последними компонентами были значения из вектора $\mathbf{x}_1$ (в количестве *m-k*). В качестве недостающих компонент возьмем компоненты вектора $\boldsymbol{\delta}$:

$$\mathbf{x}(0) = (x_{1,0}, ..., x_{k,0}, \delta_{k+1,0}, ..., \delta_{m,0}),$$

$$\mathbf{x}(1) = (\delta_{1,N}, ..., \delta_{k,N}, x_{k+1,N}, ..., x_{m,N}).$$

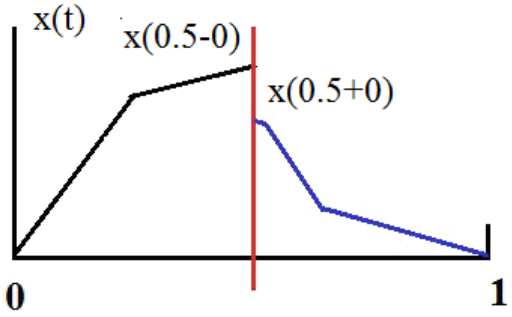


Рис. 1. Вид пробных траекторий.

Задав начальное состояние вектора $\boldsymbol{\delta}$, можем найти траекторию $\mathbf{x}(t)$, составленную из решения двух задач Коши – от 0 до 0.5 и от 1.0 до 0.5 (см. Рис. 1). Далее можно применить итерационный процесс по вектору $\boldsymbol{\delta}$, минимизирующий норму разности траекторий в средней точке временного интервала:

$$\Phi(\boldsymbol{\delta}) = \left\| \mathbf{x}(0.5+0;\boldsymbol{\delta}) - \mathbf{x}(0.5-0;\boldsymbol{\delta}) \right\|^2 \to \min.$$

В качестве таких итераций можно использовать метод Ньютона с неявным вычислением градиента функционала $\Phi(\boldsymbol{\delta})$: $\overset{s+1}{\boldsymbol{\delta}} = \overset{s}{\boldsymbol{\delta}} - \alpha \left(\frac{\partial \Phi}{\partial \delta} \left(\overset{s}{\boldsymbol{\delta}} \right) \right)^{-1} \left(\overset{s+1}{\boldsymbol{\delta}} - \overset{s}{\boldsymbol{\delta}} \right)$. В качестве начального можно взять нулевой вектор.

Комментарии. Можно стрелять и слева, и справа, и навстречу. Можно также применять ***метод параллельной стрельбы*** (разбиение отрезка на *N* частей, ввод базисных функций на каждом подотрезке, а далее сшивка решения). Это способ повышения устойчивости алгоритма.
Применяется также ***метод случайной стрельбы***.

Линейная краевая задача:

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{f}, \quad 0 < t < 1, \quad \mathbf{A}_0\mathbf{x}(0) = \mathbf{x}_0, \quad \mathbf{A}_1\mathbf{x}(1) = \mathbf{x}_1.$$

Метод построения фундаментальных решений. Смысл его в том, что в линейном случае можно найти фундаментальную систему решений и подбирать коэффициенты разложения решения по этому базису.

$\mathbf{x}(t) = \mathbf{x}^{(0)}(t) + \sum_{k=1}^{m} \alpha_k \mathbf{x}^{(k)}(t)$, где $\mathbf{x}^{(k)}(t)$ – решения однородных систем вида

$$\dot{\mathbf{x}}^{(k)} = \mathbf{A}\mathbf{x}^{(k)}, \quad 0 < t < 1, \quad x_i^{(k)}(0) = \delta_{ik},$$

$\mathbf{x}^{(0)}(t)$ – решение неоднородной системы

$$\dot{\mathbf{x}}^{(0)} = \mathbf{A}\mathbf{x}^{(0)} + \mathbf{f}, \quad 0 < t < 1, \quad x_i^{(0)}(0) = 0.$$

Коэффициенты α_k определяем из условий:

$$\mathbf{A}_0\mathbf{x}(0) = \mathbf{A}_0 \left[\sum_{k=1}^{m} \alpha_k \mathbf{x}^{(k)}(0) \right] = \mathbf{x}_0, \quad \mathbf{A}_1 \left[\mathbf{x}^{(0)}(1) + \sum_{k=1}^{m} \alpha_k \mathbf{x}^{(k)}(1) \right] = \mathbf{x}_1.$$

Получаем СЛАУ размерности *m*. Метод хорош, но не устойчив, если матрица СЛАУ плохо обусловлена. Ресурс параллелизма – размерность векторов и сложность вычислений правой части.

Метод дифференциальной прогонки для систем из двух уравнений:

$$\dot{u} = av + f, \quad \dot{v} = bu + g, \quad u(0) = u_0, \quad v(1) = v_1.$$

Решение представим в виде $u(t) = \alpha(t)v(t) + \beta(t)$. С помощью элементарных преобразований можно получить следующие формулы для определения решения:

$$\dot{\alpha} + b\alpha^2 - a = 0, \quad \alpha(0) = 0; \quad \dot{\beta} + \alpha b\beta + \alpha g - f = 0, \quad \beta(0) = u_0;$$

$$\dot{v} - \alpha bv - b\beta - g = 0, \quad v(1) = v_1; \quad u = \alpha v + \beta.$$

Решение каждой задачи Коши можно реализовать численно, в том числе используя параллельный алгоритм.

Лекция 10. Решение пространственно одномерных краевых задач.

1. Постановки краевых задач

$$Lu \equiv \frac{d}{dx}\left(k\frac{du}{dx}+r_0u\right)+r_1\frac{du}{dx}-qu=-f, \quad 0<x<1. \tag{1}$$

Линейный случай: $k=k(x)\ge k_0>0,\ r_i=r_i(x),\ q=q(x),\ f=f(x)$ – кусочно-непрерывны на (0,1).

Обобщенная краевая задача для уравнения (1):

$$\eta_m u(x_m)+\theta_m u'(x_m)=\mu_m,\ \eta_m^2+\theta_m^2\ne 0,\ m=0,\ 1, \tag{2_1}$$

$$u(0)=u(1),\ u'(0)=u'(1), \tag{2_2}$$

$$\Phi_0(u)=c_0,\ \Phi_1(u)=c_1. \tag{2_3}$$

Здесь $x_0\equiv 0$, $x_1\equiv 1$, η_m, θ_m, μ_m, c_m – константы ($m=0,1$). (2_1) включают условия либо 1-го, либо 2-го, либо 3-го рода, а также могут быть смешанными. (2_2) соответствуют периодической задаче (коэффициенты в (1) должны быть периодичны). В (2_3): $\Phi_m(u)$ – линейные функционалы u:

$$\Phi_m(u)=\int_0^1 u(x')\rho_m(x')dx' \quad (m=0,1).$$

Квазилинейный случай:

$k=k(x,u)\ge k_0>0,\ r_i=r_i(x,u),\ q=q(x,u),\ f=f(x,u)$, $\eta_m=\eta_m(u)$, $\theta_m=\theta_m(u)$, $\mu_m=\mu_m(u)$ – нелинейные функции u, а $\Phi_m(u)$ – нелинейные функционалы, например, $\int_0^1 |u(x')|^2\rho_0(x')dx'$, $\int_0^1 |u'(x')|^2\rho_1(x')dx'$ и др..

Нестационарный случай (параболический вариант):

$$\frac{\partial u}{\partial t}=\frac{\partial}{\partial x}\left(k\frac{\partial u}{\partial x}+r_0u\right)+r_1\frac{\partial u}{\partial x}-qu+f,\ 0<x<1,\ t>0, \tag{3}$$

$$u(x,0)=g(x), \tag{4}$$

+ условия вида (2). В линейном случае коэффициенты зависят от x, t, в квазилинейном – от x, t, u.

Во всех случаях предполагаем существование единственного решения.

Четыре модели:

1) $r_0\equiv 0,\ r_1\equiv 0$ – уравнение диффузии;
2) $r_0\equiv 0,\ r_1\ne 0$ – уравнение конвекции-диффузии;
3) $r_0\ne 0,\ r_1\equiv 0$ – уравнение внутренней конвекции-диффузии;
4) $r_0\ne 0,\ r_1\ne 0$ – обобщенное уравнение конвекции-диффузии.

Это разделение обусловлено как свойствами решения, так и выбором численного метода.

2. Базовый численный алгоритм

Для численного решения используем разновидность сеточных методов – метод конечных разностей. Способ построения разностных схем – интегро-интерполяционный метод.

Сетки: $\bar\omega_x=\{x=x_i,\ i=0,...,N_x\}$, $\bar\omega_t=\{t=t_j=\tau\cdot j,\ j=0,...,N_t,\ \tau=L_t/N_t\}$.

Преобразование: $Lu=\dfrac{dw}{dx}+\dfrac{r_1}{k}w-\tilde q u,\quad w=k\dfrac{du}{dx}+r_0u=\dfrac{k}{e}\dfrac{d}{dx}(eu),\quad e=\exp\left\{\int_0^x\dfrac{r_0}{k}dx'\right\},\quad \tilde q=q+\dfrac{r_0r_1}{k}.$

Схема в стационарном случае в операторном виде: $\Lambda_h y_h=-\varphi_h$. (1_h)

Здесь y_h – сеточный аналог u, φ_h – сеточный аналог f, а Λ_h – разностный оператор, сеточный аналог L.

Введем обозначения:

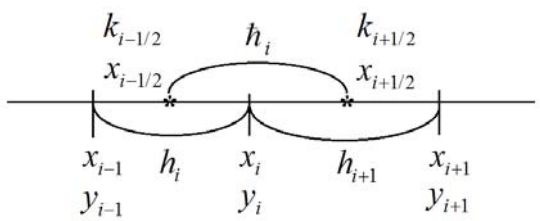
$x_{i-1},x_i,x_{i+1},x_{i-1/2},x_{i+1/2}$ – узлы сетки,

h_i, h_{i+1}, $\hbar_i=0.5\cdot(h_i+h_{i+1})$ – шаги сетки,

$y_i\approx u(x_i)$ – приближенные значения искомой функции,

$e_i,\varphi_i,\tilde Q_i$ – значения (аппроксимации) заданных сеточных функций в целых узлах сетки,

$\tilde k_{i\pm 1/2}$ – значения (аппроксимации) заданных сеточных функций в полуцелых узлах.



$\hbar_0=0.5\cdot h_1,\quad \hbar_{N_x}=0.5\cdot h_{N_x}.$

Тогда схема (1_h) в индексной форме будет иметь вид:

$$\Lambda_h y_i \equiv \frac{1}{\hbar_i}\left(\tilde{k}_{i+1/2}\frac{y_{i+1}e_{i+1}-y_i e_i}{e_{i+1/2}h_{i+1}}-\tilde{k}_{i-1/2}\frac{y_i e_i - y_{i-1}e_{i-1}}{e_{i-1/2}h_i}\right)-\tilde{Q}_i y_i = -\varphi_i, \quad 0<i<N_x. \tag{1_i}$$

Здесь $\tilde{k}_{i\pm1/2} = k_{i\pm1/2}\cdot\left(\chi_i \pm \hbar_i \tilde{r}^{\pm}_{1,i}\right)$, $k_{i\pm1/2}$ – точные значения или аппроксимации $\left[\frac{1}{h_{i+1}}\int\limits_{x_i}^{x_{i+1}}\frac{dx'}{k(x')}\right]^{-1}$ и $\left[\frac{1}{h_i}\int\limits_{x_{i-1}}^{x_i}\frac{dx'}{k(x')}\right]^{-1}$,

e_i – точные значения или аппроксимации $\exp\left\{\int\limits_0^{x_i}\frac{r_0(x')}{k(x')}dx'\right\}$, $\tilde{Q}_i$ – точные значения или аппроксимации $\frac{1}{\hbar_i}\int\limits_{x_{i-1/2}}^{x_{i+1/2}}\tilde{q}(x)dx$, φ_i – точные или приближенные значения $\frac{1}{\hbar_i}\int\limits_{x_{i-1/2}}^{x_{i+1/2}}f(x)dx$, $\chi_i = \frac{1}{1+R_i}$, $R_i = 0.5(\tilde{r}^{+}_{1,i}h_{i+1}-\tilde{r}^{-}_{1,i}h_i)$,

$\tilde{r}^{\pm}_{1,i} = 0.5\frac{r_{1,i}\pm|r_{1,i}|}{k_i}$, $q_i = q(x_i)$, $k_i = k(x_i)$, $r_{m,i} = r_m(x_i)$ $(m=0,\ 1)$.

Если во всех членах используются точные значения интегралов, получаем ***точную схему***, иначе ***приближенную***. В последнем случае используем следующие ***аппроксимации коэффициентов*:**

$$e_i = \exp\left\{\int\limits_0^{x_i}\frac{r_0(x)}{k(x)}dx\right\} \approx \exp\left\{\sum_{j=0}^{i}\frac{r_0(x_j)}{k(x_j)}\hbar_j\right\}, \quad \tilde{Q}_i = \frac{1}{\hbar_i}\int\limits_{x_{i-1/2}}^{x_{i+1/2}}\tilde{q}(x)dx \approx \tilde{q}(x_i), \quad \varphi_i = \frac{1}{\hbar_i}\int\limits_{x_{i-1/2}}^{x_{i+1/2}}f(x)dx \approx f\left(x_i\right),$$

$$k_{i+1/2} = \left[\frac{1}{h_{i+1}}\int\limits_{x_i}^{x_{i+1}}\frac{dx}{k(x)}\right]^{-1} \approx k\left(x_{i+1/2}\right) \vee \frac{1}{2}\left(k\left(x_i\right)+k\left(x_{i+1}\right)\right) \vee \frac{2k\left(x_i\right)k\left(x_{i+1}\right)}{k\left(x_i\right)+k\left(x_{i+1}\right)}.$$

Условия границе:

Для первой краевой задачи они очевидны: $y_0 = \mu_0, \quad y_{N_x} = \mu_1$. ($2_{1h}$)

Для краевых задач 2-го и 3-го рода, в граничных узлах записываются уравнения (1_h) с учетом потоков:

$$\frac{1}{\hbar_0}\left(\tilde{k}_{1/2}\frac{y_1 e_1 - y_0 e_0}{e_{1/2}h_1}-w_0\right)-\tilde{q}_0 y_0 = -\varphi_0, \quad \frac{1}{\hbar_{N_x}}\left(w_{N_x}-\tilde{k}_{N_x-1/2}\frac{y_{N_x}e_{N_x}-y_{N_x-1}e_{N_x-1}}{e_{N_x-1/2}h_{N_x}}\right)-\tilde{q}_{N_x}y_{N_x} = -\varphi_{N_x} \tag{2_{2h}}$$

Для 2-ой краевой задачи (условия Неймана $u'(0)=u'(1)=0$) потоки w_0, w_{N_x} равны 0.

Для 3-ей краевой задачи $w_0 = \tilde{\eta}_0\left(y_0-\mu_0\right)$, $w_{N_x} = -\tilde{\eta}_1\left(y_{N_x}-\mu_1\right)$.

Для периодической задачи добавляется одно граничное уравнение (1_i), например, в узле $i=N_x$ и учитываются равенства $y_0 = y_{N_x}$, $y_{N_x+1} = y_1$. При этом $\hbar_{N_x} = 0.5\cdot(h_{N_x}+h_1)$.

В случае условий (2_3) используются сеточные аналоги интегралов (формула трапеций):

$$\sum_{i=0}^{N_x} y_i \rho_{m,i}\hbar_i = c_m \quad (m=0,\ 1). \tag{2_{3h}}$$

Анализ схем. Схема (1_h)-(2_h) в случае

$r_0 \equiv 0,\ r_1 \equiv 0$ – ***однородная схема Самарского***;

$r_0 \equiv 0,\ r_1 \neq 0$ – ***регуляризованная однородная схема Самарского***;

$r_0 \neq 0,\ r_1 \equiv 0$ – ***схема Кареткиной-Костомарова***;

$r_0 \neq 0,\ r_1 \neq 0$ – ***обобщение*** двух предыдущих схем.

Квазилинейный случай. Схема (1_h)-(2_h) справедлива. Однако коэффициенты зависят от y_h. Существенный момент – локальный или нелокальный тип нелинейности коэффициентов.

Нестационарный случай.

Схема с весами: $\frac{\hat{y}_h - y_h}{\tau} = \sigma\hat{\Lambda}_h\hat{y}_h + (1-\sigma)\Lambda_h y_h + \sigma\hat{f}_h + (1-\sigma)f_h$, ($3_h$)

$y_h\big|_{t=0} = g_h$, (4_h)

где $\hat{y}_h$ и y_h относятся к верхнему и нижнему слоям по времени ($t+\tau$ и t). Вес σ = 0, 1 и 0.5 – явная, неявная и симметричная схемы. Коэффициенты в обеих схемах выбирают так, чтобы $\Psi = O(\hbar^2)$ и

$O(\hbar^2+\tau)$ или $O(\hbar^2+\tau^2)$. Для этого коэффициенты уравнений и граничных условий, содержащие интегралы, вычисляются точно или аппроксимируются со вторым порядком.

3. Система алгебраических уравнений и последовательные алгоритмы прогонки

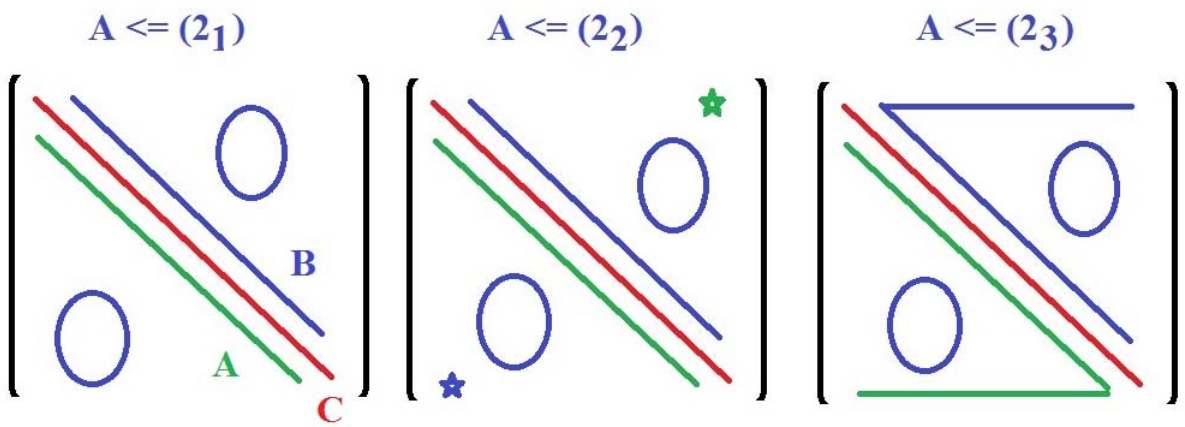
Разностная схема (1_h), (2_h) или (3_h), (4_h), (2_h) эквивалентна некоторой алгебраической задаче вида:

$$AY=F,\quad Y=\left(y_0,y_1,...,y_{N_x}\right),\quad F=\left(\varphi_0,\varphi_1,...,\varphi_{N_x}\right),\quad A=\begin{pmatrix} a_{00} & ... & a_{0N_x} \\ ... & ... & ... \\ a_{N_x0} & ... & a_{N_xN_x} \end{pmatrix}. \qquad (*)$$

Эта задача может быть линейной или нелинейной ($A=A(Y), F=F(Y)$). Рассмотрим первый случай.

Алгебраическая задача возникает после преобразования разностной схемы к специальному виду. Она дает разностное решение (стационарное, либо относящееся к очередному слою по времени).

Преобразование схемы к виду (*) состоит в умножении уравнений (1_i) на величину $\left(-\hbar_i h_m\right)$, h_m – минимальный шаг среди всех h_i. В результате матрица имеет свойства: $A=A^*\geq 0$. Проблема нулевого собственного значения в задаче Неймана ($u'(0)=u'(1)=0$) решается заданием $y_{N_x}=1$ или переходом к матрице $\tilde{A}=A+\delta I$, где I – единичная матрица, $\delta\approx N_x^{-1}$ – параметр. Обсудим далее портрет матрицы:



Для удобства решения система линейных уравнений с трехдиагональной матрицей, или близкая к ней, записывается в ***каноническом*** виде:

$$-A_iy_{i-1}+C_iy_i-B_iy_{i+1}=F_i,\quad 1\leq i\leq N-1. \qquad (5)$$

На границе в случае (2_1) получаем: $C_0y_0-B_0y_1=F_0,\ C_Ny_N-A_Ny_{N-1}=F_N$. ($6_1$)

В случае (2_2) уравнения (5) в узлах $i=0$ и $i=N$ имеем

$$-A_0y_{N-1}+C_0y_0-B_0y_1=F_0,\ -A_Ny_{N-1}+C_Ny_N-B_Ny_1=F_N. \qquad (6_2)$$

В случае (2_3) получаем:

$$C_0y_0+\sum_{i=1}^{N}B_{0,i}y_i=F_0,\qquad \sum_{i=0}^{N-1}A_{N,i}y_i+C_Ny_N=F_N. \qquad (6_3)$$

Замечание: в последнем случае можно оставить условия в виде (2_{3h}).

Свойства коэффициентов:

$$A_i,B_i\geq 0,\ C_i>0,\ i=0,...,N, \qquad (7_1)$$

либо $\quad C_i=A_i+B_i+D_i,\ D_i>0,\ i=0,...,N, \qquad (7_2)$

либо $\quad C_i=A_{i+1}+B_{i-1}+D_i,\ D_i>0,\ i=1,...,N-1. \qquad (7_3)$

Свойство (7_3) вместо (7_2) возникает в случае немонотонного оператора ($r_0\neq 0$) во внутренних узлах сетки. На границе в любом случае должны быть выполнены условия (7_2).

Если задача нелокальная, должны выполняться ***условия ее разрешимости***.

В качестве алгоритма решения алгебраической задачи (5), (6) используем ***метод прогонки*** – компактный метод Гаусса для систем с трехдиагональной матрицей.

Варианты метода прогонки.

Правая прогонка:

$$\alpha_0 = \frac{B_0}{C_0}, \quad \beta_0 = \frac{F_0}{C_0}, \quad \alpha_i = \frac{B_i}{C_i - A_i\alpha_{i-1}}, \quad \beta_i = \frac{F_i + A_i\beta_{i-1}}{C_i - A_i\alpha_{i-1}}, \quad i = 1,...,N;$$

$$y_N = \beta_N, \quad y_i = \alpha_i y_{i+1} + \beta_i, \quad i = N-1,...,0.$$

Левая прогонка:

$$\alpha_N = \frac{A_N}{C_N}, \quad \beta_N = \frac{F_N}{C_N}, \quad \alpha_i = \frac{A_i}{C_i - B_i\alpha_{i+1}}, \quad \beta_i = \frac{F_i + B_i\beta_{i+1}}{C_i - B_i\alpha_{i+1}}, \quad i = N-1,...,0;$$

$$y_0 = \beta_0, \quad y_i = \alpha_i y_{i-1} + \beta_i, \quad i = 1,...,N.$$

Встречная прогонка:

$$\alpha_0 = \frac{B_0}{C_0}, \quad \beta_0 = \frac{F_0}{C_0}, \quad \alpha_i = \frac{B_i}{C_i - A_i\alpha_{i-1}}, \quad \beta_i = \frac{F_i + A_i\beta_{i-1}}{C_i - A_i\alpha_{i-1}}, \quad i = 1,...,M;$$

$$\alpha_N = \frac{A_N}{C_N}, \quad \beta_N = \frac{F_N}{C_N}, \quad \alpha_i = \frac{A_i}{C_i - B_i\alpha_{i+1}}, \quad \beta_i = \frac{F_i + B_i\beta_{i+1}}{C_i - B_i\alpha_{i+1}}, \quad i = N-1,...,M+1;$$

$$y_M = \frac{\beta_M + \alpha_M\beta_{M+1}}{1 - \alpha_M\alpha_{M+1}}, \quad y_i = \alpha_i y_{i+1} + \beta_i, \quad i = M-1,...,0, \quad y_i = \alpha_i y_{i-1} + \beta_i, \quad i = M+1,...,N.$$

Правая циклическая прогонка:

$$\alpha_0 = 0, \quad \beta_0 = 0, \quad \gamma_0 = 1,$$

$$\alpha_i = \frac{B_i}{C_i - A_i\alpha_{i-1}}, \quad \beta_i = \frac{F_i + A_i\beta_{i-1}}{C_i - A_i\alpha_{i-1}}, \quad \gamma_i = \frac{A_i\gamma_{i-1}}{C_i - A_i\alpha_{i-1}}, \quad i = 1,...,N;$$

$$p_0 = p_N = 0, \quad q_0 = q_N = 1, \quad p_i = \alpha_i p_{i+1} + \beta_i, \quad q_i = \alpha_i q_{i+1} + \gamma_i, \quad i = N-1,...,1;$$

$$y_N = \frac{F_N + A_N p_{N-1} + B_N p_1}{C_N - A_N q_{N-1} - B_N q_1}, \quad y_i = p_i + y_N q_i, \quad i = N-1,...,0, \quad y_0 = y_N.$$

Левая циклическая прогонка:

$$\alpha_N = 0, \quad \beta_N = 0, \quad \gamma_N = 1,$$

$$\alpha_i = \frac{A_i}{C_i - B_i\alpha_{i+1}}, \quad \beta_i = \frac{F_i + B_i\beta_{i+1}}{C_i - B_i\alpha_{i+1}}, \quad \gamma_i = \frac{B_i}{C_i - B_i\alpha_{i+1}}, \quad i = N-1,...,0;$$

$$p_0 = p_N = 0, \quad q_0 = q_N = 1, \quad p_i = \alpha_i p_{i-1} + \beta_i, \quad q_i = \alpha_i q_{i-1} + \gamma_i, \quad i = 1,...,N-1;$$

$$y_0 = \frac{F_0 + A_0 p_{N-1} + B_0 p_1}{C_0 - A_0 q_{N-1} - B_0 q_1}, \quad y_i = p_i + y_0 q_i, \quad i = 1,...,N-1, \quad y_N = y_0.$$

Интегральная прогонка:

Представим решение в виде: $y_i = y_i^{(I)} + y_0 y_i^{(III)} + y_N y_i^{(II)}, \quad i = 0,...,N$.

Здесь $y_i^{(I)}, y_i^{(II)}, y_i^{(III)}$ – базисные функции, решения следующих краевых задач:

$$(I): \quad -A_i y_{i-1} + C_i y_i - B_i y_{i+1} = F_i, \ 1 \le i \le N-1, \quad y_0 = 0, \quad y_N = 0;$$

$$(II): \quad -A_i y_{i-1} + C_i y_i - B_i y_{i+1} = 0, \ 1 \le i \le N-1, \quad y_0 = 0, \quad y_N = 1;$$

$$(III): \quad -A_i y_{i-1} + C_i y_i - B_i y_{i+1} = 0, \ 1 \le i \le N-1, \quad y_0 = 1, \quad y_N = 0.$$

Из условий (2_{3h}) найдем по правилам Крамера граничные значения y_0, y_N :

$$\alpha_{11} = \sum_{i=0}^{N} y_i^{(I)} \rho_{0,i} \hbar_i, \quad \alpha_{12} = \sum_{i=0}^{N} y_i^{(II)} \rho_{0,i} \hbar_i, \quad \alpha_{13} = \sum_{i=0}^{N} y_i^{(III)} \rho_{0,i} \hbar_i,$$

$$\alpha_{21} = \sum_{i=0}^{N} y_i^{(I)} \rho_{1,i} \hbar_i, \quad \alpha_{22} = \sum_{i=0}^{N} y_i^{(II)} \rho_{1,i} \hbar_i, \quad \alpha_{23} = \sum_{i=0}^{N} y_i^{(III)} \rho_{1,i} \hbar_i,$$

$$y_0 = \frac{\alpha_{12}(c_1 - \alpha_{21}) - \alpha_{22}(c_0 - \alpha_{11})}{\alpha_{12}\alpha_{23} - \alpha_{22}\alpha_{13}}, \quad y_N = \frac{\alpha_{23}(c_0 - \alpha_{11}) - \alpha_{13}(c_1 - \alpha_{21})}{\alpha_{12}\alpha_{23} - \alpha_{22}\alpha_{13}}.$$

Немонотонная прогонка

$$(!) \quad C_i = A_{i+1} + B_{i-1} + D_i, \quad i = 1,...,N-1; \quad A_i: \ i = 1,...,N; \quad B_i: \ i = 0,...,N-1;$$

$$(!) \quad C_0 y_0 - \tilde{B}_0 y_1 \equiv C_0 y_0 - A_0 y_1 = F_0, \quad C_N y_N - \tilde{A}_N y_{N-1} \equiv C_N y_N - B_N y_{N-1} = F_N.$$

Правая немонотонная прогонка (без перемены знака, r_0<=0):

$$\alpha_0 = \frac{A_0 A_1}{B_0 C_0}, \quad \beta_0 = \frac{F_0}{C_0}, \quad \alpha_i = \frac{A_{i+1}}{C_i - B_{i-1}\alpha_{i-1}}, \quad \beta_i = \frac{F_i + A_i \beta_{i-1}}{C_i - B_{i-1}\alpha_{i-1}}, \quad i = 1,...,N-1;$$

$$y_N = \frac{F_N + B_N \beta_{N-1}}{C_N - \left(B_N / A_N\right) B_{N-1}\alpha_{N-1}}, \quad y_i = \frac{B_i}{A_{i+1}}\alpha_i y_{i+1} + \beta_i, \quad i = N-1,...,0.$$

Левая немонотонная прогонка (без перемены знака, r_0>=0):

$$\alpha_N = \frac{B_{N-1} B_N}{A_N C_N}, \quad \beta_N = \frac{F_N}{C_N}, \quad \alpha_i = \frac{B_{i-1}}{C_i - A_{i+1}\alpha_{i+1}}, \quad \beta_i = \frac{F_i + B_i \beta_{i+1}}{C_i - A_{i+1}\alpha_{i+1}}, \quad i = N-1,...,1;$$

$$y_0 = \frac{F_0 + A_0 \beta_1}{C_0 - \left(A_0 / B_0\right) A_1 \alpha_1}, \quad y_i = \frac{A_i}{B_{i-1}}\alpha_i y_{i-1} + \beta_i, \quad i = 1,...,N.$$

Встречная правая немонотонная прогонка (одна перемена знака, слева r_0<=0, справа r_0>=0):

$$0 \le \frac{B_i}{A_{i+1}} \le 1, \quad i = 0,...,M-1; \quad 0 \le \frac{A_i}{B_{i-1}} \le 1, \quad i = M+1,...,N;$$

$$\alpha_0 = \frac{A_0 A_1}{B_0 C_0}, \quad \beta_0 = \frac{F_0}{C_0}, \quad \alpha_i = \frac{A_{i+1}}{C_i - B_{i-1}\alpha_{i-1}}, \quad \beta_i = \frac{F_i + A_i \beta_{i-1}}{C_i - B_{i-1}\alpha_{i-1}}, \quad i = 1,...,M-1;$$

$$\alpha_N = \frac{B_{N-1} B_N}{A_N C_N}, \quad \beta_N = \frac{F_N}{C_N}, \quad \alpha_i = \frac{B_{i-1}}{C_i - A_{i+1}\alpha_{i+1}}, \quad \beta_i = \frac{F_i + B_i \beta_{i+1}}{C_i - A_{i+1}\alpha_{i+1}}, \quad i = N-1,...,M+1;$$

$$y_M = \frac{F_M + A_M \beta_{M-1} + B_M \beta_{M+1}}{C_M - B_{M-1}\alpha_{M-1} - A_{M+1}\alpha_{M+1}} = \frac{F_M + A_M \beta_{M-1} + B_M \beta_{M+1}}{B_{M-1}\left(1 - \alpha_{M-1}\right) + A_{M+1}\left(1 - \alpha_{M+1}\right) + D_M},$$

$$y_i = \frac{B_i}{A_{i+1}}\alpha_i y_{i+1} + \beta_i, \quad i = M-1,...,0, \quad y_i = \frac{A_i}{B_{i-1}}\alpha_i y_{i-1} + \beta_i, \quad i = M+1,...,N.$$

Встречная левая немонотонная прогонка (одна перемена знака, слева r_0>=0, справа r_0<=0):

$$0 \le \frac{A_i}{B_{i-1}} \le 1, \quad i = 1,...,M; \quad 0 \le \frac{B_i}{A_{i+1}} \le 1, \quad i = M,...,N-1;$$

$$y_i = y_i^{(I)} + y_M y_i^{(II)}, \quad i = 0,...,M;$$

$$(I): \quad C_0 y_0^{(I)} - \tilde{B}_0 y_1^{(I)} = F_0, \quad -A_i y_{i-1}^{(I)} + C_i y_i^{(I)} - B_i y_{i+1}^{(I)} = F_i, \quad 1 \le i \le M-1, \quad y_M^{(I)} = 0;$$

$$(II): \quad C_0 y_0^{(II)} - \tilde{B}_0 y_1^{(II)} = 0, \quad -A_i y_{i-1}^{(II)} + C_i y_i^{(II)} - B_i y_{i+1}^{(II)} = 0_i, \quad 1 \le i \le M-1, \quad y_M^{(II)} = 1;$$

$$y_i = y_i^{(I)} + y_M y_i^{(III)}, \quad i = M,...,N;$$

$$(I): \quad y_M^{(I)} = 0, \quad -A_i y_{i-1}^{(I)} + C_i y_i^{(I)} - B_i y_{i+1}^{(I)} = F_i, \quad M+1 \le i \le N-1, \quad C_N y_N^{(I)} - \tilde{A}_N y_{N-1}^{(I)} = F_N;$$

$$(III): \quad y_M^{(III)} = 1, \quad -A_i y_{i-1}^{(III)} + C_i y_i^{(III)} - B_i y_{i+1}^{(III)} = 0, \quad M+1 \le i \le N-1, \quad C_N y_N^{(III)} - \tilde{A}_N y_{N-1}^{(III)} = 0.$$

Условие сопряжения:

$$-A_M\left(y_{M-1}^{(I)} + y_M y_{M-1}^{(II)}\right) + C_M y_M - B_M\left(y_{M+1}^{(I)} + y_M y_{M+1}^{(III)}\right) = F_M.$$

Решение:

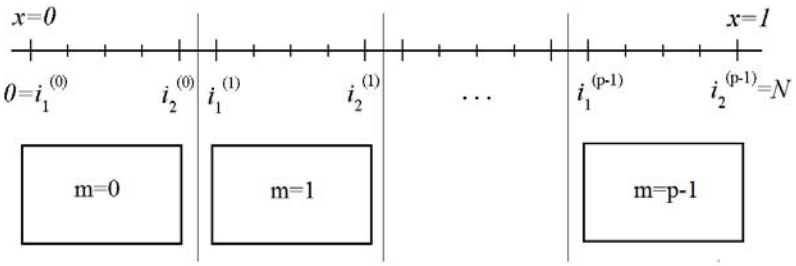
$$y_M = \frac{F_M + A_M y_{M-1}^{(I)} + B_M y_{M+1}^{(I)}}{\left(C_M - A_M y_{M-1}^{(II)} - B_M y_{M+1}^{(III)}\right)} = \frac{F_M + A_M y_{M-1}^{(I)} + B_M y_{M+1}^{(I)}}{\left(B_{M-1}\left(1 - \frac{A_M}{B_{M-1}} y_{M-1}^{(II)}\right) + A_{M+1}\left(1 - \frac{B_M}{A_{M+1}} y_{M+1}^{(III)}\right) + D_M\right)}.$$

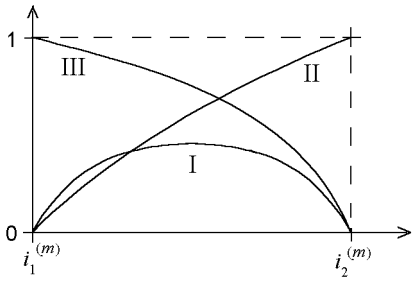
4. Параллельный алгоритм решения

В качестве параллельного алгоритма решения алгебраической задачи также используем метод прогонки. Однако имеется альтернатива – использовать либо метод конвеерной прогонки, либо метод параллельной прогонки.

Метод конвеерной прогонки состоит в разбиении всех операций алгоритма прогонки на элементарные преобразования величин отдельными ступенями конвеера. Учитывая, что современные процессоры самостоятельно выполняют такие операции, применение метода конвеерной прогонки в программе пользователя не ускорит процесс вычислений. Поэтому применение метода конвеерной прогонки может быть целесообразным лишь при решении алгебраической задачи (5), (6) с матричными коэффициентами большой размерности. В этом случае говорят о применении алгоритма ***матричной прогонки***.

Параллельный алгоритм прогонки сформулируем на примере решения системы (5), (6_1). Будем считать, что алгоритм предназначен для МВС с p одинаковыми универсальными процессорами (вычислителями), имеющими собственную оперативную память и объединенными с помощью высокоскоростной коммуникационной среды. Введем равномерное линейное разбиение множества номеров узлов сетки $\Omega = \{0,1,...,N\}$ на связные подмножества $\Omega_m = \{i_1^{(m)},...,i_2^{(m)}\}$ ($m = 0,...,p-1$), соответствующие разбиению вектора неизвестных по вычислителям:





В результате такого разбиения вычислитель с номером m будет обрабатывать $(i_2^{(m)} - i_1^{(m)} + 1)$ точек. Представим решение на каждом вычислителе с номером $m = 1,...,p-2$, в виде:

$$y_i \equiv y_i^{(m)} = y_i^{(I,m)} + y_{i_1^{(m)}} y_i^{(III,m)} + y_{i_2^{(m)}} y_i^{(II,m)}, \qquad (8_1)$$

где $y_i^{(\alpha,m)}$ ($\alpha = I, II, III$) определены на Ω_m и играют роль базиса, а значения функции на границе Ω_m – $y_{i_1^{(m)}}$ и $y_{i_2^{(m)}}$ – пока не известны. Во внутренних узлах Ω_m функция $y_i^{(I,m)}$ находится из уравнений (5), а функции $y_i^{(II,m)}$, $y_i^{(III,m)}$ из уравнений (5) с нулевой правой частью. Граничные условия для $y_i^{(\alpha,m)}$:

$$y_{i_1^{(m)}}^{(I,m)} = 0,\ y_{i_2^{(m)}}^{(I,m)} = 0;\ y_{i_1^{(m)}}^{(II,m)} = 0,\ y_{i_2^{(m)}}^{(II,m)} = 1;\ y_{i_1^{(m)}}^{(III,m)} = 1,\ y_{i_2^{(m)}}^{(III,m)} = 0. \qquad (9_1)$$

На нулевом и последнем вычислителях:

$$y_i^{(0)} = y_i^{(I,0)} + y_{i_2^{(0)}} y_i^{(II,0)}, \quad y_i^{(p-1)} = y_i^{(I,p-1)} + y_{i_1^{(p-1)}} y_i^{(III,p-1)}. \qquad (8_2)$$

Граничные условия:

$$\begin{aligned} &C_0 y_0^{(I,0)} - B_0 y_1^{(I,0)} = F_0,\ y_{i_2^{(0)}}^{(I,0)} = 0; \quad y_{i_1^{(p-1)}}^{(I,p-1)} = 0,\ C_N y_N^{(I,p-1)} - A_N y_{N-1}^{(I,p-1)} = F_N; \\ &C_0 y_0^{(II,0)} - B_0 y_1^{(II,0)} = 0,\ y_{i_2^{(0)}}^{(II,0)} = 1. \quad y_{i_1^{(p-1)}}^{(III,p-1)} = 1,\ C_N y_N^{(III,p-1)} - A_N y_{N-1}^{(III,p-1)} = 0. \end{aligned} \qquad (9_2)$$

Нахождение значений в граничных узлах подобластей производится с помощью так называемой ***короткой задачи.*** Ее составляют следующие уравнения из (5):

$$-A_{i_2^{(m)}} y_{i_2^{(m)}-1} + C_{i_2^{(m)}} y_{i_2^{(m)}} - B_{i_2^{(m)}} y_{i_2^{(m)}+1} = F_{i_2^{(m)}}, \quad -A_{i_1^{(m+1)}} y_{i_1^{(m+1)}-1} + C_{i_1^{(m+1)}} y_{i_1^{(m+1)}} - B_{i_1^{(m+1)}} y_{i_1^{(m+1)}+1} = F_{i_1^{(m+1)}}. \qquad (10)$$

Если учесть в этих уравнениях очевидные связи:

$$y_{i_2^{(m)}-1} = y_{i_2^{(m)}-1}^{(I,m)} + y_{i_1^{(m)}} y_{i_2^{(m)}-1}^{(III,m)} + y_{i_2^{(m)}} y_{i_2^{(m)}-1}^{(II,m)},\ y_{i_2^{(m)}+1} = y_{i_1^{(m+1)}},$$

$$y_{i_1^{(m+1)}-1} = y_{i_2^{(m)}},\ y_{i_1^{(m+1)}+1} = y_{i_1^{(m+1)}+1}^{(I,m)} + y_{i_1^{(m+1)}} y_{i_1^{(m+1)}+1}^{(III,m)} + y_{i_2^{(m+1)}} y_{i_1^{(m+1)}+1}^{(II,m)},$$

то получим:

$$-\tilde{A}_{i_2^{(m)}} y_{i_1^{(m)}} + \tilde{C}_{i_2^{(m)}} y_{i_2^{(m)}} - \tilde{B}_{i_2^{(m)}} y_{i_1^{(m+1)}} = \tilde{F}_{i_2^{(m)}}, \quad -\tilde{A}_{i_1^{(m+1)}} y_{i_2^{(m)}} + \tilde{C}_{i_1^{(m+1)}} y_{i_1^{(m+1)}} - \tilde{B}_{i_1^{(m+1)}} y_{i_2^{(m+1)}} = \tilde{F}_{i_1^{(m+1)}}, \qquad (11)$$

с коэффициентами

$$\tilde{A}_{i_2^{(m)}} = A_{i_2^{(m)}} y_{i_2^{(m)}-1}^{(III,m)}, \quad \tilde{B}_{i_2^{(m)}} = B_{i_2^{(m)}}, \quad \tilde{C}_{i_2^{(m)}} = C_{i_2^{(m)}} - A_{i_2^{(m)}} y_{i_2^{(m)}-1}^{(II,m)}, \quad \tilde{F}_{i_2^{(m)}} = F_{i_2^{(m)}} + A_{i_2^{(m)}} y_{i_2^{(m)}-1}^{(I,m)},$$

$$\tilde{A}_{i_1^{(m+1)}} = A_{i_1^{(m+1)}}, \quad \tilde{B}_{i_1^{(m+1)}} = B_{i_1^{(m+1)}} y_{i_1^{(m+1)}+1}^{(II,m+1)}, \quad \tilde{C}_{i_1^{(m+1)}} = C_{i_1^{(m+1)}} - B_{i_1^{(m+1)}} y_{i_1^{(m+1)}+1}^{(III,m+1)}, \ \tilde{F}_{i_1^{(m+1)}} = F_{i_1^{(m+1)}} + B_{i_1^{(m+1)}} y_{i_1^{(m+1)}+1}^{(I,m+1)}.$$

В итоге получим следующую ***короткую систему*** из $2p-2$ уравнений для $2p-2$ неизвестных

$$-\tilde{A}_i y_{i-1} + \tilde{C}_i y_i - \tilde{B}_i y_{i+1} = \tilde{F}_i,\ i \in \tilde{\Omega} = \{i_2^{(0)}, i_1^{(1)}, i_2^{(1)}, ..., i_1^{(p-1)}\}, \qquad (12)$$

где под индексом $i \pm 1$ понимается переход к соответствующему соседнему элементу из множества $\tilde{\Omega}$. Аналогично можно показать, что в граничных узлах $i_2^{(0)}$ и $i_1^{(p-1)}$ уравнения (12) принимают вид (6_1).

Свойства базисных функций:

$$\left\| y^{(I,m)} \right\|_C \le \left\| D^{-1} F \right\|_C,\ 0 \le y^{(II,m)} \le 1,\ 0 \le y^{(III,m)} \le 1,\ m = 0,...,p-1, \quad 0 \le y_i^{(II,m)} + y_i^{(III,m)} \le 1, \text{ для всех } i, m \qquad (13)$$

Эти свойства обеспечивают устойчивость вычислений по формулам (8).

В силу свойств базисных функций (13) коэффициенты ***короткой системы*** уравнений также удовлетворяют условиям принципа максимума => решение системы (12) ***существует и единственно***. Определив это решение последовательным методом прогонки, с помощью формул (8) можно вычислить решение исходной задачи в параллельном варианте.

Последовательность действий параллельного алгоритма прогонки.

1. Каждый вычислитель с помощью алгоритма последовательной прогонки решает три (или две) задачи для нахождения базисных функций $y^{(\alpha,\, m)}$.
2. Каждый вычислитель находит свою часть коэффициентов ***короткой системы*** относительно неизвестных $y_{i_1}^{(m)},\ y_{i_2}^{(m)}$ ($m = 0,...,p-1$).
3. Все вычислители осуществляют ***коллективный обмен*** коэффициентами ***короткой системы***.
4. Каждый вычислитель решает короткую систему и выбирает нужные ему значения $y_{i_1}^{(m)}$ и $y_{i_2}^{(m)}$.
5. Каждый вычислитель использует значения $y_{i_1}^{(m)}$ и $y_{i_2}^{(m)}$ для вычисления своей части решения по формулам (8).

Теоретические ускорение и эффективность алгоритма параллельной прогонки.

Каждый ПА оценивается по ускорению S_p и эффективности E_p, которые определяются по формулам $S_p = t_1 / t_p,\ E_p = (S_p / p)\cdot 100\%$, где t_1 – оценка времени решения исходной задачи на одном вычислителе, t_p – оценка времени решения исходной задачи на p вычислителях. В теоретических исследованиях часто не принимают во внимание накладные расходы, связанные с временем обменов, поскольку последнее является неконтролируемым фактором и сильно зависит от конкретной МВС. Оценки времён t_1 и t_p часто заменяют на оценки числа обобщенных арифметических операций Q_1 и Q_p. Последние оценивают по количеству элементарных объектов вычислений, например, как в нашем случае, по числу узлов расчетной сетки *N*. Воспользуемся этим же подходом.

Алгоритм ***последовательной прогонки*** имеет оценку $Q_1 = C_1 N$, где $C_1 \approx 8$ – константа, не зависящая от числа узлов сетки и не включающая операции присваивания. Алгоритм ***параллельной прогонки*** имеет оценку $Q_p \approx C_2 N / p + 2C_1 p$ действий (здесь $C_2 = 3C_1 + 4 = 28$ и также не учитывает операции присваивания). Последнее выражение получено следующим образом. На каждом вычислителе, кроме нулевого и последнего, решаются три задачи размерности N / p. Каждый вычислитель также решает короткую задачу размерности $2p-2$. Сумма этих двух этапов вычислений и составляет величину Q_p. Константы C_1, C_2 очевидно не зависят от *N* и *p*.

Теперь оценим величины ускорения и эффективности параллельного алгоритма прогонки:

$$S_p = \frac{Q_1}{Q_p} = \frac{p}{C_2 / C_1 + 2p^2 / N}, \quad E_p = \frac{S_p}{p}\cdot 100\% = \frac{1}{C_2 / C_1 + 2p^2 / N}\cdot 100\%.$$

Анализ полученных выражений показывает, что при условии $p << \sqrt{N}$ ускорение составляет $S_p \approx p / 3.5$, а эффективность $E_p \approx 28.6\%$. Эти оценки можно уточнить, если рассмотреть структуру вычислений и учесть соотношения между различными видами арифметических операций. Однако полученная асимптотика не сильно изменится. Примем минимальную теоретическую оценку эффективности параллельного алгоритма прогонки на уровне 30%.

Ускорение и эффективность параллельного алгоритма решения краевой задачи в целом.

В последовательном варианте алгоритма величина $Q_1 = C_0 N + C_1 N$, где первое слагаемое связано с вычислением коэффициентов алгебраической задачи. В случае использования нескольких вычислителей получим соответственно $Q_p \approx C_0 N / p + C_2 N / p + 2C_1 p$. Поэтому

$$S_p = \frac{p}{(C_0 + C_2)/(C_0 + C_1) + 2(p^2 / N)C_1 / (C_0 + C_1)} \to \frac{C_0 + C_1}{C_0 + C_2} p, \quad при \quad p << \sqrt{N};$$

$$E_p = \frac{1}{(C_0 + C_2)/(C_0 + C_1) + 2(p^2 / N)C_1 / (C_0 + C_1)}\cdot 100\% \to \frac{C_0 + C_1}{C_0 + C_2}\cdot 100\%, \quad при \quad p << \sqrt{N}.$$

При подсчете константы C_0 учтем, что минимум $2N$ операций приходится на вычисление узлов сетки (например, по формуле $x_i = a + h_x \cdot i$), еще $8N$ операций получим при вычислении коэффициентов алгебраической задачи в каноническом виде:

$$A_i = 0.5\left(k_i + k_{i-1}\right), \quad B_i = 0.5\left(k_i + k_{i+1}\right), \quad C_i = A_i + B_i + h_x^2 q_i, \quad F_i = h_x^2 f_i .$$

Сложность вычислений функций k, q, f зависит от типа их зависимости от аргумента x (см. Табл. 1). Поэтому для каждого варианта зависимостей получим различные асимптотические оценки ускорения и эффективности. Они показывают, что чем сложнее вычисления коэффициентов задачи, тем выше ускорение и эффективность итогового параллельного алгоритма решения.

Таблица 1. Варианты задачи, приводящие к конкретным значениями величины C_0.

№	Тип зависимости функций k, q, f от аргумента x	Величина C_0	Ускорение	Эффективность, %
1.	Постоянная: $y = a$	10	$\frac{18}{38}p$	47.4
2.	Линейная: $y = a \cdot x + b$	10+3*2=16	$\frac{24}{44}p$	54.5
3.	Квадратичная: $y = a \cdot x^2 + b \cdot x + c$	10+3*5=25	$\frac{33}{53}p$	62.3
4.	Экспоненциальная*): $y = a + b \cdot \exp[\alpha \cdot x]$	10 + 3*(3+10)=49	$\frac{57}{77}p$	74.0
5.	Тригонометрическая*): $a \cdot \sin\left(\alpha \cdot x\right) + b \cdot \cos\left(\beta \cdot x\right)$	10+3*(5+20)=85	$\frac{93}{113}p$	82.3

*) Здесь принята оценка сложности вычисления экспоненциальных и тригонометрических функций равная 10 по сравнению с элементарными операциями с вещественными числами.

Следствия и обобщения

5.1. Циклическая прогонка.

В случае решения периодической краевой задачи в последовательном варианте используется алгоритм циклической прогонки. В нем фактически вводится аналогичный базис из двух функций на всем отрезке интегрирования, а короткая система содержит одно неизвестное значение, которое легко находится из соответствующего граничного уравнения. В параллельном варианте алгоритма, на каждом вычислителе вводится базис из трех функций. Получаемая короткая система содержит теперь $2p-1$ неизвестных в силу периодичности. Эта система решается с помощью последовательного алгоритма циклической прогонки. Минимальная теоретическая эффективность параллельного алгоритма циклической прогонки составляет ***50%*** ввиду того, что в оценке для Q_1 константа $C_1 = 14$, а в оценке Q_p константа $C_2 = 28$ (базисные функции на каждом вычислителе находятся обычным, а не циклическим, алгоритмом прогонки).

5.2. Интегральная прогонка.

Случай интегральных граничных условий аналогичен периодической задаче. Отличием является то, что короткая система уравнений в параллельном алгоритме содержит $2p$ неизвестных и может решаться с помощью любого последовательного алгоритма прогонки. Минимальная теоретическая эффективность параллельного варианта интегральной прогонки достигает ***100%***, ввиду того, что константы $C_1 = C_2 = 36$.

5.3. Немонотонная прогонка.

В случае немонотонного оператора ($r_0 \neq 0$) используется алгоритм немонотонной прогонки, который в последовательном случае требует введения аналогичных базисных функций. При этом аналогом разбиения по вычислителям здесь является разбиение исходной области точками смены знака функции r_0. Если k – число смен знака, то короткая система уравнений содержит в последовательном алгоритме $2k$, а в параллельном – не более $2p+2k$ неизвестных. В обоих случаях она также решается одним из вариантов метода немонотонной прогонки, используемого для знакопостоянной функции r_0. Эффективность параллельного варианта может достигать ***100%***, поскольку последовательный вариант более трудоемок по сравнению с алгоритмом обычной (монотонной) прогонки.

Если функция r_0 не меняет знак, то можно использовать левую или правую (в зависимости от знака r_0) немонотонную прогонку. Поэтому минимальная теоретическая эффективность параллельного варианта снизится до ***30%***. При этом время решения в последовательном варианте уменьшится в 3 раза, и в параллельном варианте будет сложнее достичь выполнения асимптотического поведения ускорения.

5.4. Квазилинейный случай.

В квазилинейной постановке задачи для реализации разностной схемы необходим итерационный процесс. Это либо метод простой итерации (МПИ), либо метод Ньютона (МН). В МН при условии локальной нелинейности (ньютоновский оператор перехода не выходит за рамки шаблона разностной схемы, за исключением, быть может, граничных узлов) на каждой итерации получаем задачу типа (5), (6). Приведем основные формулы алгоритмов вышеупомянутых методов.

Зададим начальное приближение к искомому решению – вектор $\overset{0}{Y} = (\overset{0}{y}_0, ..., \overset{0}{y}_N)$, и используем один из следующих алгоритмов.

Алгоритм метода простой итерации (МПИ):

$$-A_i(\overset{s}{Y})\overset{s+1}{y}_{i-1} + C_i(\overset{s}{Y})\overset{s+1}{y}_i - B_i(\overset{s}{Y})\overset{s+1}{y}_{i+1} = F_i(\overset{s}{Y}), \quad 1 \le i \le N-1,$$

$$C_0(\overset{s}{Y})\overset{s+1}{y}_0 - B_0(\overset{s}{Y})\overset{s+1}{y}_1 = F_0(\overset{s}{Y}), \quad C_N(\overset{s}{Y})\overset{s+1}{y}_N - A_N(\overset{s}{Y})\overset{s+1}{y}_{N-1} = F_N(\overset{s}{Y}),$$

$$\overset{s}{Y} = (\overset{s}{y}_0, ..., \overset{s}{y}_N), \quad s = 0, 1, ...$$

Алгоритм метода Ньютона (МН):

$$-A_i(\overset{s}{Y})\overset{s+1}{y}_{i-1} + C_i(\overset{s}{Y})\overset{s+1}{y}_i - B_i(\overset{s}{Y})\overset{s+1}{y}_{i+1} + \theta\sum_{j=0}^{N}\left[-\frac{\partial A_i}{\partial y_j}\left(\overset{s}{Y}\right)\overset{s}{y}_{i-1} + \frac{\partial C_i}{\partial y_j}\left(\overset{s}{Y}\right)\overset{s}{y}_i - \frac{\partial B_i}{\partial y_j}\left(\overset{s}{Y}\right)\overset{s}{y}_{i+1}\right]\left(\overset{s+1}{y}_j - \overset{s}{y}_j\right) =$$

$$= F_i(\overset{s}{Y}) + \theta\sum_{j=0}^{N}\frac{\partial F_i}{\partial y_j}\left(\overset{s}{Y}\right)\left(\overset{s+1}{y}_j - \overset{s}{y}_j\right), \quad 1 \le i \le N-1,$$

$$C_0(\overset{s}{Y})\overset{s+1}{y}_0 - B_0(\overset{s}{Y})\overset{s+1}{y}_1 + \theta\sum_{j=0}^{N}\left[\frac{\partial C_i}{\partial y_j}\left(\overset{s}{Y}\right)\overset{s}{y}_0 - \frac{\partial B_i}{\partial y_j}\left(\overset{s}{Y}\right)\overset{s}{y}_1\right]\left(\overset{s+1}{y}_j - \overset{s}{y}_j\right) = F_0(\overset{s}{Y}) + \theta\sum_{j=0}^{N}\frac{\partial F_0}{\partial y_j}\left(\overset{s}{Y}\right)\left(\overset{s+1}{y}_j - \overset{s}{y}_j\right),$$

$$C_N(\overset{s}{Y})\overset{s+1}{y}_N - A_N(\overset{s}{Y})\overset{s+1}{y}_{N-1} + \theta\sum_{j=0}^{N}\left[-\frac{\partial A_i}{\partial y_j}\left(\overset{s}{Y}\right)\overset{s}{y}_{N-1} + \frac{\partial C_i}{\partial y_j}\left(\overset{s}{Y}\right)\overset{s}{y}_N\right]\left(\overset{s+1}{y}_j - \overset{s}{y}_j\right) = F_N(\overset{s}{Y}) + \theta\sum_{j=0}^{N}\frac{\partial F_N}{\partial y_j}\left(\overset{s}{Y}\right)\left(\overset{s+1}{y}_j - \overset{s}{y}_j\right).$$

Здесь $\theta \in [0,1]$ – параметр регуляризации метода. При $\theta = 1$, получается классический метод Ньютона, при $\theta = 0$ – метод простой итерации.

Решение на каждой итерации в вышеуказанных алгоритмах определяется методом прогонки. При получении очередного приближения используется ***критерий сходимости*** по норме невязки или по норме разности соседних итераций.

При параллельной реализации методов используется соответствующий параллельный вариант прогонки. При этом минимальная теоретическая эффективность общего алгоритма решения будет зависеть от сложности вычисления коэффициентов краевой задачи и выбранного варианта метода прогонки. Путем повышения эффективности распараллеливания в рамках алгоритма метода Ньютона может быть фиксация второй и третьей базисных функций на первой итерации. Это соответствует так называемому ***стационарному алгоритму*** метода Ньютона. В этом случае можно получить минимальную теоретическую эффективность параллельного алгоритма порядка ***100%***.

5.5. Нестационарная задача.

При решении нестационарной задачи по неявной или симметричной схемам имеется аналогия с решением квазилинейной задачи. Если функции k, r_i, q, определяющие оператор перехода с одного слоя по времени на другой, постоянны или зависят только от *x*, то базисные функции в параллельном алгоритме прогонки $y^{(II)}, y^{(III)}$ можно вычислить только один раз (на первом временном слое). Тогда на каждом последующем слое по времени пересчитывается только базисная функция $y^{(I)}$. Этот прием может увеличить теоретическую эффективность до ***100%***. Если функции k, r_i, q слабо зависят от *t*, то по аналогии со стационарным алгоритмом методом Ньютона их можно вычислять не на каждом слое по времени. В этом случае также можно получить высокую эффективность параллельного алгоритма.

5.6. Многомерный случай.

В случае решения многомерных краевых задач в областях прямоугольной формы возможно эффективное использование параллельного алгоритма прогонки на этапах реализации итерационных или временных схем с факторизованным оператором. В качестве примеров можно указать решение многомерного уравнения Пуассона методом переменных направлений или решение многомерного

уравнения теплопроводности с помощью локально-одномерных разностных схем. Эффективность получаемых здесь параллельных реализаций также может достигать ***100%***.

5.7. Матричная прогонка.

В случае решения систем уравнений вида (7)-(8) с матричными коэффициентами можно воспользоваться алгоритмом параллельной матричной прогонки, который строится на изложенных выше принципах. Очевидно, что во многих случаях и здесь удастся получить высокую эффективность распараллеливания.

Заключение:

1) Алгоритм параллельной прогонки обладает асимптотическим свойством, которое позволяет эффективно использовать его, как непосредственно, так и в комбинациях с другими методами.

2) Минимальная теоретическая эффективность параллельного алгоритма прогонки составляет от 30 до 100% в зависимости от вычислительной сложности исходной задачи и способа агрегирования параллельной прогонки с другими алгоритмами.

Лекция 11. Параллельные алгоритмы решения уравнения теплопроводности.

1. Постановка модельной трёхмерной начально-краевой задачи

Рассмотрим проблему решения трехмерного линейного уравнения теплопроводности:

$$\frac{\partial u}{\partial t}=\sum_{\alpha=1}^{3}\frac{\partial}{\partial x_\alpha}\left(k_\alpha(\vec{x},t)\frac{\partial u}{\partial x_\alpha}\right)+f(\vec{x},t),\ \ \vec{x}\in D,\ \ t>0. \tag{1}$$

Здесь $u=u(\vec{x},t)$ – температура, функция пространственных координат $\vec{x}=(x_1,x_2,x_3)$ и времени t, $k_\alpha=k_\alpha(\vec{x},t)>0$ – коэффициенты теплопроводности, $f(\vec{x},t)$ – плотность мощности источников/стоков тепла, D – некоторая ограниченная прямоугольная область трехмерного пространства. Уравнение (1) замыкается добавлением некоторых начальных и граничных условий. В качестве начального возьмем однородное распределение температуры в объеме

$$u(\vec{x},0)=u_0=const>0. \tag{2}$$

Граничные условия 3-го рода наиболее естественны для задач теплопроводности:

$$(k(\vec{x},t)\nabla u,\vec{\sigma})=-\eta(u-u_0),\ \ \vec{x}\in S. \tag{3}$$

Здесь $\vec{\sigma}$ – вектор внешней нормали к S, η – коэффициент теплопередачи через поверхность S.

В дальнейшем D – трехмерный единичный куб, интервал по времени ограничен значением $T>0$. В этих предположениях граничные условия (3) можно переписать в виде

$$k_\alpha\frac{\partial u}{\partial x_\alpha}=+\eta(u-u_0),\ x_\alpha=0,\ k_\alpha\frac{\partial u}{\partial x_\alpha}=-\eta(u-u_0),\ x_\alpha=1,\ \alpha=1,2,3. \tag{4}$$

2. Методы численного решения уравнения теплопроводности

Рассмотрим возможные конечно-разностные методы решения задачи (1), (2), (4). Наша цель – предложить экономичный прямой метод или группу методов для решения задачи на МВС с распределенной памятью.

Представим уравнение (1) в следующем виде

$$\frac{\partial u}{\partial t}=Lu+f\ ,\ \ Lu\equiv\sum_{\alpha=1}^{3}L_\alpha u\ ,\ \ L_\alpha u\equiv\frac{\partial}{\partial x_\alpha}\left(k_\alpha\frac{\partial u}{\partial x_\alpha}\right). \tag{5}$$

Введем сетку $\Omega=\bar{\omega}_{x_1}\times\bar{\omega}_{x_2}\times\bar{\omega}_{x_3}\times\bar{\omega}_t$, в которой $\bar{\omega}_{x_\alpha}=\{x_{\alpha,i_\alpha}=i_\alpha h_\alpha,\ i_\alpha=0,1,...,N_\alpha,\ h_\alpha=1/N_\alpha\}$ ($\alpha=1,2,3$) и $\bar{\omega}_t=\{\ t=t_j=j\tau,\ j=0,1,...,N_t,\ \tau=t_{\max}/N_t\}$. На этой сетке определим разностный аналог искомой функции u_h. Для нахождения функции u_h с помощью интегро-интерполяционного подхода построим семейство разностных схем, называемое схемой с весами:

$$\frac{\hat{y}_h-y_h}{\tau}=\sigma\left[\ \hat{\Lambda}_h\hat{y}+\hat{\varphi}_h\right]+(1-\sigma)\left[\Lambda_h y_h+\varphi_h\right]. \tag{6}$$

Здесь использованы общепринятые обозначения. Аппроксимация Λ_h оператора L имеет вид

$$\Lambda_h y_h=\sum_{\alpha=1}^{3}\Lambda_{h,\alpha}y_h\ ,\ \ \Lambda_{h,\alpha}y_h=\left(\bar{k}_\alpha y_{h,x_\alpha}\right)_{\bar{x}_\alpha}.$$

В семействе схем (6) выделяют обычно явную ($\sigma=0$) и неявную ($\sigma=1$) разностные схемы, имеющие первый порядок точности по времени и второй – по пространству, а также симметричную схему Кранка-Николсон ($\sigma=0.5$), имеющую второй порядок точности и по времени, и по пространству.

Явная схема ($\sigma=0$) наиболее проста в реализации и экономична. Это означает, что при использовании явной схемы число арифметических действий Q необходимое для нахождения решения на очередном временном слое пропорционально количеству узлов пространственной сетки $N=\prod_{\alpha=1}^{3}\left(N_\alpha+1\right)$. Явная схема не является абсолютно устойчивой. При ее использовании необходимо соблюдать условие

$$\tau<0.5\hbar^2/(3k_0),\ \ \hbar=\min_{\alpha=1,2,3}h_\alpha\ ,\ \ k_0=\max_{x\in D,t\in[0,T],\alpha=1,2,3}k_\alpha(\vec{x},t). \tag{7}$$

Неявная схема ($\sigma=1$) является более трудоемкой в реализации и в общем случае не экономична. Однако она абсолютно устойчива, то есть вычисления по ней будут корректными при любых $\tau>0$ и

$h_\alpha > 0$. Однако есть условие асимптотической устойчивости $\tau < 0.5\hbar / \sqrt{k_0}$. При нахождении решения на очередном временном слое в зависимости от выбранного метода (прямого или итерационного) и вида функции $k_\alpha(\vec{x},t)$ может потребоваться от $O(N \log_2 N)$ до $O(N^{7/3})$ арифметических действий. Отметим также, что при выполнении условия (7) точность решений, полученных по явной и неявной схемам с одинаковым шагом по времени, практически совпадает.

<u>Прямые методы:</u> одна (иногда очень сложная) процедура дает решение с машинной точностью. **<u>Варианты:</u>** метод Гаусса, ленточный метод Гаусса, метод Фурье (простое и быстрое преобразования Фурье), комбинированный метод: ленточный метод Гаусса по одной (прогонка) или двум координатам + метод Фурье по двум или одной координатам.

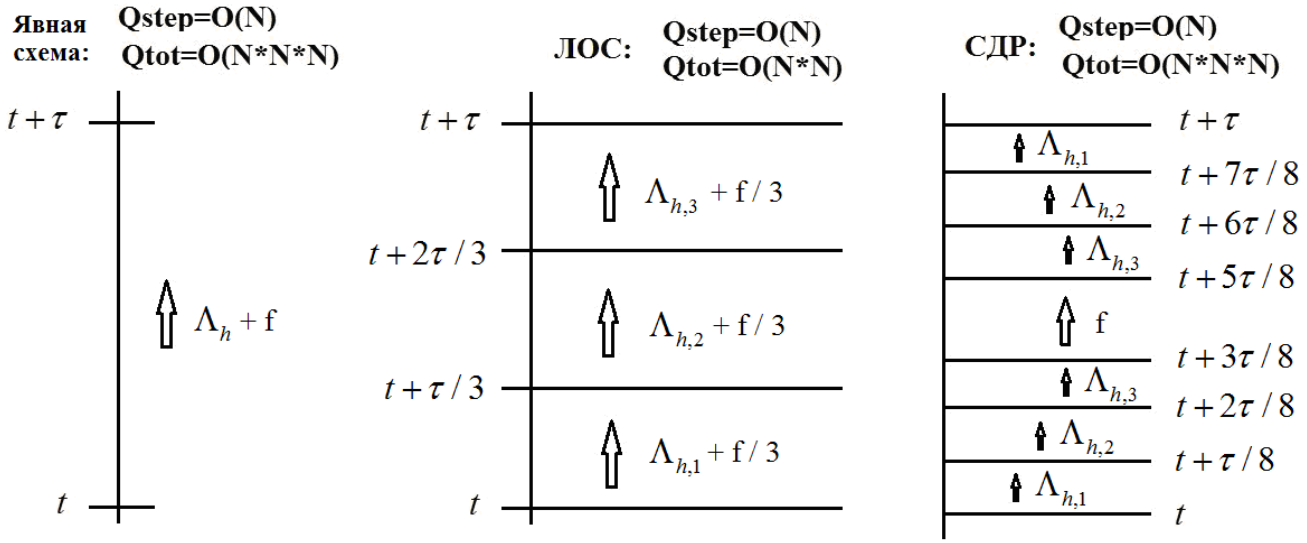
<u>Итерационные методы:</u> одна простая процедура применяется многократно и дает решение с заданной точностью. **<u>Варианты:</u>** двухслойные и многослойные, диагональные, треугольные, факторизованные (см. Лекцию 12). Итерационные методы редко используются для решения линейной задачи.

Симметричная схема ($\sigma = 0.5$) по трудоемкости реализации эквивалентна неявной схеме, а по условию устойчивости аналогична явной схеме: $\tau < \hbar^2 / (3k_0)$. Ее преимущество – второй порядок точности по времени.

На практике используются либо явная схема, как наиболее простая в реализации, либо ***экономичные неявные схемы***, основанные на расщеплении пространственного оператора по координатам x_α. Например, ***локально-одномерные схемы*** (ЛОС), предложенные А.А. Самарским. Особенность ЛОС – свойство суммарной аппроксимации уравнения исходной задачи и абсолютная устойчивость. Вариант ***неявной ЛОС***:

$$\frac{y_h^{j+\alpha/3} - y_h^{j+(\alpha-1)/3}}{\tau} = \Lambda_{h,\alpha}^{j+\alpha/3} y_h^{j+\alpha/3} + \frac{1}{3} f_h^{j+1}, \quad \alpha = 1,2,3. \tag{8}$$

Здесь введены неполные индексные обозначения и промежуточные слои по времени $t_{j+\alpha/3} = t_j + \alpha\tau/3$. Схема (8) имеет первый порядок точности по времени и второй по пространству. Она экономична и может быть реализована с помощью алгоритма прогонки на каждом промежуточном этапе по времени.



Если нужна повышенная точность по времени и экономичность, то используют ***неявные схемы двуциклического расщепления на основе схемы Кранка-Николсон***, которые также являются схемами суммарной аппроксимации, но имеют второй порядок точности по времени. Вариант такой схемы:

$$\begin{aligned}
&\frac{y_h^{j+\alpha/8} - y_h^{j+(\alpha-1)/8}}{\tau} = \frac{1}{4}\Lambda_{h,\alpha}^{j+1/2}\left(y_h^{j+\alpha/8} + y_h^{j+(\alpha-1)/8}\right), \quad \alpha = 1,2,3, \\
&\frac{y_h^{j+5/8} - y_h^{j+3/8}}{\tau} = f_h^{j+1/2}, \\
&\frac{y_h^{j+(9-\alpha)/8} - y_h^{j+(8-\alpha)/8}}{\tau} = \frac{1}{4}\Lambda_{h,\alpha}^{j+1/2}\left(y_h^{j+(9-\alpha)/8} + y_h^{j+(8-\alpha)/8}\right), \quad \alpha = 3,2,1.
\end{aligned} \tag{9}$$

Здесь $t_{j+\beta/8} = t_j + \beta\tau/8$, $\beta = 1,...,8$. Схема (9) абсолютно устойчива в норме L_2 (но не в норме C), экономична и реализуется методом прогонки.

Итог: имеем три схемы решения задачи на МВС: явная (6), неявная ЛОС (8) и схема двуциклического расщепления (9). Выбор обусловлен требованием экономичности вычислений, которое является основным при решении многомерных задач. Для неявной схемы (6) в случае переменных коэффициентов экономичный прямой или итерационный метод пока не найден.

Замечания:

1) Для расчета стационарных решений предпочтительна неявная ЛОС, в которой шаг по времени ограничен только условием асимптотической устойчивости ($\tau \sim \hbar$).

2) Для расчета псевдопериодических или существенно нестационарных решений подходит схема (9), как более точная и позволяющая также брать шаг $\tau \sim \hbar$. При этом условии схема (9) имеет точность $O(\hbar^2)$, а неявная ЛОС – $O(\hbar)$.

3) Явная схема наиболее проста в параллельной реализации и является эталоном эффективности.

3. Параллельные алгоритмы решения уравнения теплопроводности

Для реализации выбранных разностных схем на МВС необходимо применить один из подходов распараллеливания соответствующего “скалярного” (однопроцессорного) численного алгоритма. Наиболее эффективным для решения многомерных задач механики сплошной среды на МВС с распределенной памятью считается ***принцип геометрического параллелизма***, который предполагает проведение декомпозиции расчетной области на равные (по числу узлов сетки) подобласти соответственно числу процессоров (вычислителей).

В нашем случае, учитывая геометрические свойства (топологию) исходной расчетной области (а именно, ее прямоугольность), можно применить декомпозицию трех типов: ***линейную, квадратичную и кубическую***. Первая предполагает разбиение области по одной из трех координат, вторая – по двум координатам, третья – по трем (примеры различных типов разбиения показаны на рис. 1). В соответствии с типами декомпозиции возникают три различных алгоритма распараллеливания, а также три типа топологии межпроцессорных связей, которые условно называют “линейка”, “квадратная решетка” и “кубическая решетка” соответственно типу декомпозиции.

Параллельная реализация явной схемы (6). Рассмотрим сначала расчетные формулы в последовательном алгоритме реализации явной схемы:

$$y^{j+1}_{h,i_1,i_2,i_3} = y^{j}_{h,i_1,i_2,i_3} + \tau\left(\Lambda^{j}_{h,i_1,i_2,i_3} y^{j}_{h,i_1,i_2,i_3} + f^{j}_{h,i_1,i_2,i_3}\right), \quad (i_1,i_2,i_3) \in I\,. \tag{10}$$

Здесь $I = \{(i_1,i_2,i_3), i_1 = 0,...,N_1, i_2 = 0,...,N_2, i_3 = 0,...,N_3\} \equiv I_1 \times I_2 \times I_3$ – индексное множество узлов пространственной сетки, которое предлагается разбивать на подмножества (домены), обрабатываемые отдельными вычислителями.

При линейном разбиении области по одной из координат (например, по x_2, см. рис. 1а) k-ый процессор вычисляет решение на верхнем слое по времени в диапазоне узлов:

$$I^{(k)} = \{(i_1,i_2,i_3), i_1 = 0,...,N_1, i_2 = i^{(k)}_{20},...,i^{(k)}_{21}, i_3 = 0,...,N_3\} \equiv I_1 \times I^{(k)}_2 \times I_3\,.$$

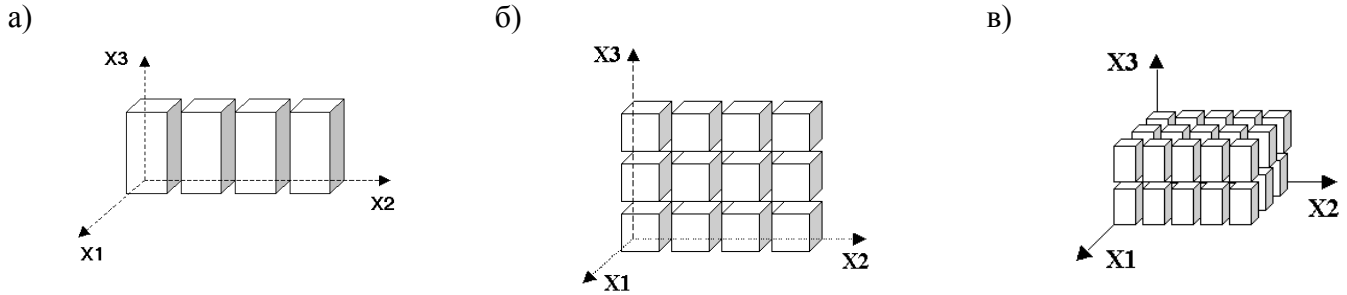


Рис. 1. Разбиение расчетной области по одной (а), двум (б) и трем координатам (в)**.**

Для получения равномерной загрузки процессоров количества точек $N^{(k)} = (N_1+1)(i^{(k)}_{21} - i^{(k)}_{20} + 1)(N_3+1)$ в диапазонах $I^{(k)}$ должны быть примерно одинаковы. Расчетные формулы для вычисления решения на верхнем слое:

$$y^{j+1}_{h,i_1,i_2,i_3} = y^{j}_{h,i_1,i_2,i_3} + \tau\left(\Lambda^{j}_{h,i_1,i_2,i_3} y^{j}_{h,i_1,i_2,i_3} + f^{j}_{h,i_1,i_2,i_3}\right), \quad (i_1,i_2,i_3) \in I^{(k)}. \tag{11}$$

В случае “***линейного***” разбиения области в формуле (11) для процессора k участвуют значения функции y_h в точках $\left(i_1, i^{(k)}_{20} - 1, i_3\right) \equiv \left(i_1, i^{(k-1)}_{21}, i_3\right)$ и $\left(i_1, i^{(k)}_{21} + 1, i_3\right) \equiv \left(i_1, i^{(k+1)}_{20}, i_3\right)$, лежащие в области данных

процессоров (k-1) и (k+1). Перед началом очередного шага вычислений по формулам (11) на процессоре k необходимо получить требующиеся значения y_h от соседних процессоров. Для этого лучше всего произвести один групповой обмен данными для всех индексов i_1, i_3. Обмен легко организуется в рамках линейной топологии. Количество передаваемых данных одним процессором равно $M^{(k)} = 4(N_1+1)(N_3+1)$ (обмен слева и справа в обе стороны) и не зависит от числа процессоров. В случае равного количества узлов сетки по различным направлениям ($N_1 = N_2 = N_3 = \sqrt[3]{N}$) величина $M^{(k)} = 4N^{2/3}$. Оценка ускорения:

$$Q_1 = C_1 N\ ,\ \ Q_p = C_1 \frac{N}{p} + C_0(p)M\ ,\ \ S_p = \frac{Q_1}{Q_p} = \frac{p}{1 + pC_0(p)M / C_1 N} = \frac{p}{1 + 4pC_0(p)N^{-1/3} / C_1}\ .$$

В случае "***квадратичного***" разбиения области, пример которого показан на рис. 1б, формулы (11) остаются в силе, но изменяются диапазон индексов $I^{(k)}$ и количество точек $N^{(k)}$, обрабатываемых процессором k:

$$I^{(k)} = \{(i_1, i_2, i_3), i_1 = 0,...,N_1, i_2 = i_{20}^{(k)},...,i_{21}^{(k)}, i_3 = i_{30}^{(k)},...,i_{31}^{(k)}\} \equiv I_1 \times I_2^{(k)} \times I_3^{(k)},$$

$$N^{(k)} = (N_1+1)\left(i_{21}^{(k)} - i_{20}^{(k)} + 1\right)\left(i_{31}^{(k)} - i_{30}^{(k)} + 1\right).$$

Соответственно изменяется и топология обменов и межпроцессорных связей, которые образуют теперь решетку. Для определения соседей процессора k, с которыми ему придется обмениваться данными, можно ввести двойную нумерацию процессоров и установить взаимно однозначное соответствие исходных и двойных процессорных номеров. В данном случае можно использовать соотношения:

$$k = p_3 k_2 + k_3\ ,\ \ k_2 = 0,..., p_2 - 1\ ,\ \ k_3 = 0,..., p_3 - 1\ ,\ \ k_2 = [k / p_3],\ \ k_3 = k \bmod p_3\ ,\ \ k = 0,..., p-1\ ,\ \ p = p_2 p_3\ .$$

Здесь p – общее число процессоров, образующих решетку $p_2 \times p_3$. В этих обозначениях процессор $k \leftrightarrow (k_2, k_3)$ перед началом очередного шага вычислений будет обмениваться недостающими данными с четырьмя соседями $k^{left} \leftrightarrow (k_2 - 1, k_3)$, $k^{right} \leftrightarrow (k_2 + 1, k_3)$, $k^{down} \leftrightarrow (k_2, k_3 - 1)$, $k^{up} \leftrightarrow (k_2, k_3 + 1)$ в рамках топологии "решетка". Количество передаваемых данных $M^{(k)} = 4(N_1+1)((N_2+1)/p_2 + (N_3+1)/p_3)$ теперь зависит от числа процессоров. При равном числе узлов сетки по различным направлениям и квадратной решетке процессоров ($p_2 = p_3 = \sqrt{p}$) величина $M^{(k)} = 8N^{2/3}p^{-1/2}$. Оценка ускорения:

$$S_p = \frac{p}{1 + pC_0(p)M / C_1 N} = \frac{p}{1 + 8p^{1/2}C_0(p)N^{-1/3} / C_1}\ .$$

В случае "***кубического***" разбиения области (см. рис. 1в) получим расчетные формулы (11), в которых

$$I^{(k)} = \{(i_1, i_2, i_3), i_1 = i_{10}^{(k)},...,i_{11}^{(k)}, i_2 = i_{20}^{(k)},...,i_{21}^{(k)}, i_3 = i_{30}^{(k)},...,i_{31}^{(k)}\} \equiv I_1^{(k)} \times I_2^{(k)} \times I_3^{(k)},$$

$$N^{(k)} = \left(i_{11}^{(k)} - i_{10}^{(k)} + 1\right)\left(i_{21}^{(k)} - i_{20}^{(k)} + 1\right)\left(i_{31}^{(k)} - i_{30}^{(k)} + 1\right).$$

Для определения соседей и обмена с ними данными вводится новая тройная нумерация процессоров:

$$k = p_2 p_3 k_1 + p_3 k_2 + k_3\ ,\ \ k_1 = 0,..., p_1 - 1\ ,\ \ k_2 = 0,..., p_2 - 1\ ,\ \ k_3 = 0,..., p_3 - 1\ ,$$

$$k_1 = [k / (p_2 p_3)],\ \ k_2 = [(k - p_2 p_3 k_1) / p_3],\ \ k_3 = (k - p_2 p_3 k_1) \bmod p_3\ ,\ \ k = 0,..., p-1\ ,\ \ p = p_1 p_2 p_3\ .$$

В соответствии с ней каждый процессор в кубической решетке имеет в общем случае 6 соседей, с которыми и обменивается необходимыми данными в начале каждого шага вычислений. Количество передаваемых данных равно

$$M^{(k)} = 4(N_1+1)(N_2+1)/(p_1 p_2) + 4(N_1+1)(N_3+1)/(p_1 p_3) + 4(N_2+1)(N_3+1)/(p_2 p_3)\ .$$

При $N_\alpha = \sqrt[3]{N}$ и $p_\alpha = \sqrt[3]{p}$ величина $M^{(k)} = 12N^{2/3}p^{-2/3}$, ускорение

$$S_p = \frac{p}{1 + pC_0(p)M / C_1 N} = \frac{p}{1 + 12p^{1/3}C_0(p)N^{-1/3} / C_1}\ .$$

Теоретическая эффективность рассмотренных трех алгоритмов равна 100%. Реальная эффективность алгоритма зависит от схемы обменов и числа процессоров. При прочих равных условиях использование линейной топологии при большом числе процессоров будет менее эффективно, чем использование топологий типа “решетка”, поскольку у последних размер передаваемых данных снижается с ростом числа процессоров.

Параллельная реализация неявной ЛОС (8). Для этого тоже можно использовать три типа разбиения расчетной области и три типа топологии межпроцессорных связей. При ***линейном разбиении*** области, например, по координате x_2, получается следующий параллельный алгоритм. Для вычисления решения на новом временном слое каждый процессор последовательно решает три задачи:

$$\left(E-\tau\,\Lambda_{h,\alpha}^{j+\alpha/3}\right)y_h^{j+\alpha/3}=y_h^{j+(\alpha-1)/3}+\frac{\tau}{3}f_h^{j+1},\quad \alpha=1,2,3, \tag{12}$$

в своем диапазоне индексов $I^{(k)}$. При этом первый и третий этапы ЛОС можно реализовать независимо от данных на других процессорах и использовать для этого алгоритм скалярной прогонки. На втором этапе ЛОС данные оказываются распределенными между процессорами. Поэтому необходимо использовать параллельный алгоритм, например, алгоритм параллельной прогонки. Он применяется по индексу i_2 для всех фиксированных значений i_1 и i_3. При этом возникают обмены данными, связанные с формированием массивов коэффициентов "коротких задач" (количество которых равно $(N_1+1)(N_3+1)$), а также при передаче значений решения этих задач на все процессоры. Обмены производятся либо в рамках топологии "звезда", либо в рамках топологии "клика". Линейная топология при такой организации обменов фактически не используется, однако она лежит в основе разбиения области. При использовании стандарта коммуникаций MPI лучше всего использовать функцию "коллективное суммирование" (MPI_Reduce или MPI_Allreduce) для сбора информации со всех процессоров. Максимальный объем передаваемых данных с одного процессора $M^{(k)}=(N_1+1)(N_3+1)4(2p-2)$ растет с увеличением p. При равном числе узлов сетки по различным направлениям $M^{(k)}=O\left(N^{2/3}p\right)$. Более точно: $M^{(k)}=8(p-1)l$, $l=1\div N_b$, $N_b=(N_1+1)(N_3+1)$. Здесь l – число задач, решаемых одновременно.

В случае ***"квадратичного"*** разбиения области, например, по координатам x_2, x_3, первый этап схемы реализуется с помощью скалярной прогонки, второй и третий – с помощью параллельной. Обмены данными между процессорами на каждом параллельном этапе организуются описанным выше способом. Максимальный объем передаваемых данных (максимум по двум параллельным этапам) равен $M^{(k)}=O\left((N_1+1)\times\max\left[(N_2+1)\frac{p_3}{p_2},(N_3+1)\frac{p_2}{p_3}\right]\right)$ или $M^{(k)}=O\left(N^{2/3}\right)$ – при равном количестве узлов сетки по различным направлениям и квадратной решетке процессоров.

В случае ***"кубического"*** разбиения области все этапы схемы реализуются с помощью параллельного алгоритма прогонки. Обмены данных производятся с помощью коллективных процедур на каждом этапе. Максимальный объем передаваемых данных с одного процессора равен

$$M^{(k)}=O\left(\max\left[(N_1+1)(N_2+1)\frac{p_3}{p_1p_2},(N_1+1)(N_3+1)\frac{p_2}{p_1p_3},(N_2+1)(N_3+1)\frac{p_1}{p_2p_3}\right]\right).$$

При равном количестве узлов сетки и количестве процессоров по различным направлениям $M^{(k)}=O\left(N^{2/3}p^{-1/3}\right)$ и убывает с увеличением p.

Эффективность алгоритмов. В скалярном варианте расчет по неявной ЛОС на каждом временном шаге требует $Q_1=3(C_1+C_2)N$ арифм. операций. Здесь C_1 – количество операций при вычислении коэффициентов уравнений в одном узле пространственной сетки, C_2 – количество операций при вычислениях по алгоритму прогонки на каждом этапе схемы, приходящееся на один узел сетки. На практике $C_1\gg C_2$. При параллельной реализации на системе с p процессорами с учетом коллективных обменов между процессами получаем оценки для $Q_p^{(\alpha)}$ ($\alpha=1,2,3$) рассмотренных выше алгоритмов:

$$Q_p^{(1)}=(3C_1+2C_2)\frac{N}{p}+N_1N_3\left(3C_2\frac{N_2}{p}+2C_2p\right)+C_0(p)M=\frac{N}{p}\left(3C_1+5C_2+\left(2C_2+8C_0(p)\right)\frac{p^2}{N_2}\right),$$

$$Q_p^{(2)}=\frac{N}{p}\left(3C_1+7C_2+\left(2C_2+8C_0(p)\right)\left(\frac{p_2^2}{N_2}+\frac{p_3^2}{N_3}\right)\right),$$

$$Q_p^{(3)} = \frac{N}{p}\left(3C_1 + 9C_2 + \left(2C_2 + 8C_0(p)\right)\left(\frac{p_1^2}{N_1} + \frac{p_2^2}{N_2} + \frac{p_3^2}{N_3}\right)\right).$$

Сравнивая теперь эти величины с Q_1, в предположениях $C_1 \sim C_2$ (наихудшая ситуация) и $p_\beta^2 \ll \sqrt[3]{N_\beta}$ ($\beta = 1,2,3$) получаем оценки для теоретического ускорения $S_p^{(\alpha)} = Q_1 / Q_p^{(\alpha)}$ и теоретической эффективности $E_p^{(\alpha)} = (S_p^{(\alpha)} / p)\cdot 100\%$ приближенно равные $\frac{6p}{8}, \frac{6p}{10}, \frac{6p}{12}$ и 75, 60, 50 % для $\alpha = 1,2,3$.

Параллельная реализация схемы двуциклического расщепления (9). В ее основе лежат те же принципы, что и в алгоритме для неявной ЛОС. Только теперь в скалярном и в параллельном случаях для поиска решения на новом временном слое мы решаем семь подзадач (семь этапов схемы (9)), шесть из которых решаются методом прогонки, а седьмой аналогичен этапу явной схемы:

$$\begin{aligned}
&\left(E - \frac{\tau}{4}\Lambda_{h,\alpha}^{j+1/2}\right) y_h^{j+\alpha/8} = 2y_h^{j+(\alpha-1)/8}, \quad y_h^{j+\alpha/8} = y_h^{j+\alpha/8} - y_h^{j+(\alpha-1)/8}, \quad \alpha = 1,2,3, \\
&y_h^{j+5/8} = y_h^{j+3/8} + \tau f_h^{j+1/2}, \\
&\left(E - \frac{\tau}{4}\Lambda_{h,\alpha}^{j+1/2}\right) y_h^{j+\beta/8} = 2y_h^{j+(\beta-1)/8}, \quad y_h^{j+\beta/8} = y_h^{j+\beta/8} - y_h^{j+(\beta-1)/8}, \quad \beta = 9-\alpha, \alpha = 3,2,1.
\end{aligned} \tag{13}$$

В параллельной реализации формулы (13) относятся к соответствующему индексному пространству $I^{(k)}$. Обмены данными организуются также как и в схеме (8). Для данной схемы $Q_1 = (4C_1 + 6C_2)N$. В параллельных реализациях:

$$Q_p^{(1)} = (4C_1 + 4C_2)\frac{N}{p} + 2C_2 N_1 N_3\left(3\frac{N_2}{p} + 2p\right) + C_0(p)M = \frac{N}{p}\left(4C_1 + 10C_2 + \left(4C_2 + 8C_0(p)\right)\frac{p^2}{N_2}\right),$$

$$Q_p^{(2)} = \frac{N}{p}\left(4C_1 + 14C_2 + \left(4C_2 + 8C_0(p)\right)\left(\frac{p_2^2}{N_2} + \frac{p_3^2}{N_3}\right)\right),$$

$$Q_p^{(3)} = \frac{N}{p}\left(4C_1 + 18C_2 + \left(4C_2 + 8C_0(p)\right)\left(\frac{p_1^2}{N_1} + \frac{p_2^2}{N_2} + \frac{p_3^2}{N_3}\right)\right).$$

Сравнивая эти величины с Q_1, в предположениях $C_1 \sim C_2$ и $p_\beta^2 \ll \sqrt[3]{N_\beta}$ ($\beta = 1,2,3$) получаем значения теоретических ускорения и эффективности приближенно равные $\frac{8p}{14}, \frac{8p}{18}, \frac{8p}{22}$ и 57,44,36 % для $\alpha = 1,2,3$.

4. Результаты тестовых расчетов

Расчеты проводились по явной схеме (6) и неявной ЛОС (8) на сетках 100^3, 200^3, 300^3. При этом использовались все три типа разбиения расчетной области. Расчеты проводились на однородной вычислительной системе МВС-1000М, имеющей 768 процессоров.

Расчеты по ***явной схеме*** выявили высокую эффективность распараллеливания и следующие тенденции.

1) Реальная эффективность тем выше, чем больше узлов расчетной сетки попадает на процессор, так как для подробной сетки загрузка процессоров получается более равномерной.

2) Эффективность при линейном разбиении ниже, чем на квадратной или кубической решетке, что определяется уменьшением накладных расходов при пересылках с ростом числа процессоров на конфигурациях типа “решетка”.

3) На сетке среднего объема преимущества кубической решетки по сравнению с квадратной проявляются при умеренном числе процессоров, так как количество обменов для кубической решетки больше, чем для квадратной, и при большом числе процессоров характеристики МВС заметно ухудшаются. На сетках очень большого объема ситуация меняется в пользу кубической решетки в силу уменьшения размера пересылаемых данных с ростом числа процессоров, которое происходит для этой топологии существенно быстрее.

4) Применение линейной топологии сильно ограничено числом точек по одному из направлений (на практике, число процессоров должно быть $\le N_\alpha / 2$). Более гибкими оказываются “квадратная” и “кубическая” решетки. Однако при их использовании величины N_α / p_α должны быть примерно

одинаковы. На сетке с одинаковым числом узлов по всем направлениям это требование интерпретируется как уменьшение коэффициента “неквадратности” решетки.

5) При проведении массовых расчетов на сетке фиксированного размера следует определить оптимальные топологию и число процессоров, при которых задача решается максимально быстро и эффективно. Например, для сеток 2 (3) оптимальной конфигурацией оказалась квадратная решетка процессоров 14x14=196 (19x19=361), дающая ускорение 167.6 (327.2) и эффективность 85.5 (90.6) %.

Расчеты по ***неявной ЛОС*** позволили сделать следующие выводы.

1) Эффективность расчетов с ростом числа процессоров здесь падает быстрее, чем в случае явной схемы. Это связано с тем, что накладные расходы (отношение суммарного времени обменов данными к общему времени расчета в %), связанные с обменами данных, здесь растут быстрее и могут быть близки к 100% при большом числе процессоров.

2) Ввиду более сложной и объемной процедуры обменов, преимущества кубической решетки по сравнению с “линейкой” и “квадратной” решеткой процессоров проявляются сразу во всем диапазоне процессорных конфигураций (от 1 до 500).

3) При малом и умеренном числе процессоров (1-100) очень сильно проявляется фактор “неквадратности” решетки, который следует учитывать в расчетах.

4) По сравнению с явной схемой эффективность расчетов по неявной ЛОС оказывается в 2-3 раза ниже. Учитывая это, а также различие расчетных формул, получаем, что при одинаковых: размере пространственной сетки, шаге по времени и числе процессоров неявный алгоритм работает в 1.5-3 раза медленнее. Однако если рассчитывается стационарное решение, то шаг в неявной схеме можно взять в N_α раз больший, что в десятки раз окупает ее более низкую эффективность распараллеливания.

При расчетах существенно нестационарных решений следует использовать ***схему двуциклического расщепления*** (9), где также можно увеличить шаг по времени на несколько порядков при той же точности вычислений, что и в явной схеме.

5. Выводы

1) При расчетах на МВС стационарных и медленно меняющихся со временем нестационарных решений краевых задач для параболических уравнений на грубых сетках следует использовать явные схемы. В этом случае условие устойчивости схемы не является обременительным, а эффективность ее параллельной реализации максимальна.

2) При расчетах на МВС стационарных и слабо нестационарных решений на подробной сетке достаточно использовать неявные ЛОС первого порядка точности по времени. При этом необходимо следить, чтобы выполнялось лишь условие их асимптотической устойчивости.

3) При расчетах на МВС существенно нестационарных решений следует использовать неявные схемы двуциклического расщепления или аналогичные им, имеющие второй порядок точности по времени. При этом шаг по времени должен определяться необходимой точностью решения и удовлетворять условию асимптотической устойчивости.

4) Выбор конкретного разбиения области, топологии и способа межпроцессорных обменов определяется соотношением размеров расчетной и “процессорной” сеток. При этом для фиксированной расчетной сетки всегда существуют оптимальные топология обменов и количество процессоров, при которых достигается скорейшее решение задачи в целом.

Лекция 12. Параллельные алгоритмы решения уравнения Пуассона.

1. Постановка модельной двумерной краевой задачи

$$\frac{\partial}{\partial x_1}\left(k_1\frac{\partial u}{\partial x_1}\right)+\frac{\partial}{\partial x_2}\left(k_2\frac{\partial u}{\partial x_2}\right)=-f\,,\quad \vec{x}\in D=\{(x_1,x_2):0<x_\alpha<1,\alpha=1,2\}\,, \tag{1}$$

Варианты граничных условий:

1) условия Дирихле: $u=g$, $\vec{x}\in\partial D$;
2) условия Неймана: $\partial u/\partial n=0$, $\vec{x}\in\partial D$;
3) условия третьего рода: $\partial u/\partial n=-\eta u$, $\vec{x}\in\partial D$;
4) условия смешанного типа: $g_1u+g_2\partial u/\partial n=g_3$, $\vec{x}\in\partial D$;
5) периодические условия;
6) интегральные условия.

Проблема решения задачи Неймана, регуляризация.

Координаты: декартовы, цилиндрические, сферические, обобщенные.

Варианты задачи: линейная, квазилинейная, нелинейная.

2. Численные алгоритмы на базе метода конечных разностей и конечных элементов.

Метод конечных разностей – дискретизация области и решения.

Схема получается путем интегрирования уравнения с весом 1:

$$\Lambda_h y_h=-\varphi_h\,,\quad \Lambda_h y_h\equiv\sum_{\alpha=1,2}\Lambda_{h,\alpha}y_h\,,\quad \Lambda_{h,\alpha}y_h=(\bar{k}_\alpha y_{x_\alpha})_{\bar{x}_\alpha}\,,\quad \vec{x}\in\Omega=\omega_{x_1}\times\omega_{x_2} \tag{1$_h$}$$

Метод конечных элементов – дискретизация только области.

Схема получается путем умножения уравнения на базисную функцию и интегрирования по всему пространству.

3. Свойства алгебраической задачи.

Численная схема (2) записывается с учетом граничных условий. Пример:

Равномерная сетка: $\Omega=\omega_{x_1}\times\omega_{x_2}$, $\omega_{x_\alpha}=\left\{x_{\alpha,i_\alpha}=i_\alpha h_\alpha, i_\alpha=0,...,N_\alpha, h_\alpha=1/N_\alpha\right\}$, $\alpha=1,2$.

Разностная схема с учетом граничных условий Дирихле в индексной форме:

$$\frac{1}{\hbar_1}\left\{k_{1,i_1+1/2,i_2}\frac{y_{i_1+1,i_2}-y_{i_1,i_2}}{h_1}-k_{1,i_1-1/2,i_2}\frac{y_{i_1,i_2}-y_{i_1-1,i_2}}{h_1}\right\}+$$

$$+\frac{1}{\hbar_2}\left\{k_{2,i_1,i_2+1/2}\frac{y_{i_1,i_2+1}-y_{i_1,i_2}}{h_2}-k_{2,i_1,i_2-1/2}\frac{y_{i_1,i_2}-y_{i_1,i_2-1}}{h_2}\right\}=-f_{i_1,i_2},\quad i_\alpha=1,...,N_\alpha-1, \tag{1$_i$}$$

$$y_{i_1,i_2}=g_{i_1,i_2},\quad i_\alpha=0,N_\alpha,\quad \alpha=1,2.$$

Далее мы преобразуем ее в алгебраическую задачу вида $AY=F$, где $Y,F\in\mathbb{R}^N$ ($N=(N_1+1)(N_2+1)$), $A\in\mathbb{R}^{N\times N}$, $A\geq 0$ – симметричная разреженная матрица, имеющая клеточную структуру. Естественное упорядочение: $Y=(y_{00},y_{10},...,y_{N_10},y_{01},y_{11},...,y_{N_11},...,y_{0N_2},y_{1N_2},...,y_{N_1N_2})$.

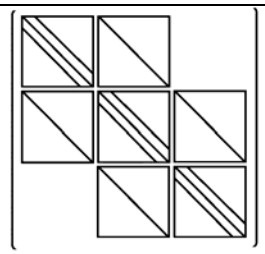
Задача Неймана: проблема вырожденности оператора, решается заданием доп. условия $y_{N_1N_2}=1$ или возмущением матрицы: $A\to A+\delta I$, δ – малое число.

4. Прямые методы решения линейной алгебраической задачи.

Линейные системы с полностью заполненной матрицей.

Метод Гаусса, Метод квадратного корня: арифметическая сложность – $O\left(N^3\right)$, N – число неизвестных.

Линейные системы с разреженной матрицей: максимальная арифметическая сложность – $O\left(N^2\right)$.

Универсальные: 1) Ленточный метод Гаусса (ЛМГ), 2) Ленточный метод квадратного корня (ЛМКК) или Метод Холецкого, ***Специальные:*** 3) Дискретное преобразование Фурье (ДПФ), 4) Быстрое дискретное преобразование Фурье (БДПФ), 5) БДПФ + метод прогонки (МП).	

Число операций

ЛМГ и ЛМКК: $Q_1 = O(Nl^2)$ (l – полуширина ленты – N_1 или N_2), для квадратной сетки $Q_1 = O(N^2)$.

ДПФ: $Q_1 = O(N^2)$. **БДПФ**: $Q_1 = O(N \log_2 N)$ – экономичный метод (ограничение по использованию).

БДПФ+МП: $Q_1 = O(N \log_2 \sqrt{N})$ – экономичный метод (ограничение по использованию).

Параллельные варианты: **ЛМГ** и **МКК** плохо параллелятся даже на МВС с общей памятью. **ДПФ** параллелится хорошо на МВС с общей и распределенной памятью. **БДПФ** практически не параллелится. **БДПФ+МП** параллелится хорошо на МВС с общей и распределенной памятью.

5. Итерационные методы решения линейной алгебраической задачи.

Двухслойная схема: $B_{s+1}\dfrac{Y^{s+1}-Y^s}{\tau_{s+1}} + AY^s = F$, $s = 0,1,2,\ldots$, Y^0 – начальное приближение.

Сходится, если $A = A^* > 0, B_{s+1} = B_{s+1}^* > 0, B_{s+1} - \dfrac{\tau_{s+1}}{2}A > 0$.

Количество операций необходимое для решения: $Q_1 = q_1 \cdot n(\varepsilon)$, число действий для реализации одной итерации, обычно $q_1 = O(N)$, $n(\varepsilon)$ – число итераций необходимое для достижения точности ε.

Методы с диагональной матрицей перехода ("диагональные" методы).

1) Метод простой итерации (**МПИ**): $B_s \equiv E, \tau_s \equiv \tau_0$. Число итераций $n(\varepsilon) = O(N \ln[2/\varepsilon])$. Диагональный, в целом не экономичный, нужны границы спектра, легко параллелится.

2) МПИ с чебышевским набором параметров (**ЧНП**): $n(\varepsilon) = O(\sqrt{N} \ln[2/\varepsilon])$. Диагональный, не экономичный, нужны границы спектра, легко параллелится.

3) Метод Якоби (**МЯ**): $B_s \equiv D, \tau_s \equiv 1$. Пример: $D = D_A = diag(A)$. $n(\varepsilon) = O(N \ln[2/\varepsilon])$. Диагональный, не экономичный, не нужны границы спектра, легко параллелится.

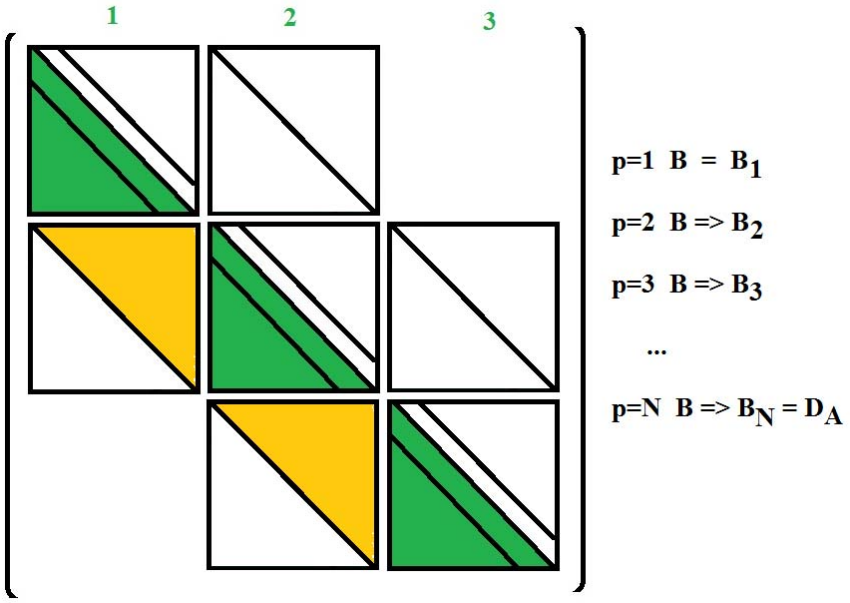
4) Метод Якоби с чебышевским набором параметров (**МЯ-ЧНП**) – МПИ с ЧНП для задачи $D_A^{-1}AY = D_A^{-1}F$. $n(\varepsilon) = O(\sqrt{N} \ln[2/\varepsilon])$. Диагональный, в целом не экономичный, нужны границы спектра, легко параллелится.

Методы с треугольной матрицей перехода ("треугольные" методы).

5) Метод Зейделя–Некрасова (**МЗН**): $B_s \equiv A^- + D, \tau_s \equiv 1$ ($A = A^- + D_A + A^+$), $n(\varepsilon) = O(\sqrt{N} \ln[2/\varepsilon])$. Треугольный, в целом не экономичный, не нужны границы спектра, трудно распараллелить.

Идея возможного распараллеливания состоит в коррекции матрицы перехода B (см. рис., зеленым и желтым цветом показаны блоки матрицы, лежащие в памяти разных вычислителей). Для каждого числа параллельных процессов p, строится свой оператор B_p, такой, обращение которого можно производить независимо. Для этого из исходной матрицы B удаляются "мешающие" блоки (на рис. выделены желтым цветом).

Результат – можно пользоваться, но появляется зависимость $n(\varepsilon, p)$. При выполнении условия $p \ll N$ зависимость не очень сильная.



6) Метод верхней релаксации (**МВР**): $B_s \equiv A^- + \dfrac{1}{\omega}D, \tau_s \equiv 1$, $n(\varepsilon) = O(\sqrt{N} \ln[2/\varepsilon])$. Треугольный, не экономичный, нужны границы спектра, трудно распараллелить.

7) Попеременно-треугольный метод (**ПТМ**): $B_s \equiv (E+\omega R_1)(E+\omega R_2),\ R = R_1 + R_2 = R^* > 0,\ R_1^* = R_2$.

Пример: $R_1 y = \frac{y_{\bar{x}_1}}{h_1} + \frac{y_{\bar{x}_2}}{h_2},\ R_2 y = -\frac{y_{x_1}}{h_1} - \frac{y_{x_2}}{h_2}$. $n(\varepsilon) = O(\sqrt[4]{N}\ln[2/\varepsilon])$. Треугольный, почти экономичный, нужны границы спектра, проблема с задачей Неймана, трудно распараллелить.

8) Модифицированный ПТМ (МПТМ): $B_s \equiv (D+\omega A_1)(D+\omega A_2),\ A = A_1 + A_2 = A^* > 0,\ A_1^* = A_2$, $D>0$ – диагональный. $n(\varepsilon) = O(\sqrt[4]{N}\ln[2/\varepsilon])$. Треугольный, почти экономичный, нужны границы спектра, проблема с задачей Неймана, трудно распараллелить.

Методы с факторизованной матрицей перехода (методы "факторизации").

9) Метод переменных направлений (**МПН**): $A = A_1 + A_2 = A^* > 0,\ A_1^* = A_1 > 0,\ A_2^* = A_2 > 0,\ A_1 A_2 = A_2 A_1$.

Аналог ЛОС: $(Y^{s+1/2} - Y^s)/\tau^{(1)} + A_1 Y^{s+1/2} + A_2 Y^s = F$, $(Y^{s+1} - Y^{s+1/2})/\tau^{(2)} + A_1 Y^{s+1/2} + A_2 Y^{s+1} = F$.

$n(\varepsilon) = O(\ln\sqrt{N}\cdot\ln[4/\varepsilon])$. Факторизованный, почти экономичный, не нужны границы спектра, проблема с задачей Неймана, легко распараллелить.

10) МПН2: $A = A_1 + A_2 = A^* > 0,\ A_1^* = A_1 > 0,\ A_2^* = A_2 > 0,\ A_1 A_2 \neq A_2 A_1$. Схема:

$(E+\omega_1 A_1)Y^{s+1/2} = (E-\omega_1 A_2)Y^s + \omega_1 F$, $(E+\omega_2 A_2)Y^{s+1} = (E-\omega_2 A_1)Y^{s+1/2} + \omega_2 F$. $n(\varepsilon) = O(\sqrt{N}\ln[1/\varepsilon])$.

Факторизованный, почти экономичный, не нужны границы спектра, проблема с задачей Неймана, легко распараллелить.

11) Альфа-Бета алгоритм

Данный алгоритм предложен Б.Н. Четверушкиным и является комбинацией прямых и итерационных методов. В его основе лежат метод прогонки и простой итерации по нелинейности (см. описание на сайте http://polyakov.imamod.ru/arc/stud/parallel/books/alpha-beta_algorithm.pdf).

Трехслойные итерационные методы

12) Трехслойный метод Чебышева

$$Y^1 = (E-\tau_0 A)Y^0 + \tau_0 F,\quad Y^{s+1} = \alpha_{s+1}(E-\tau_0 A)Y^s + (1-\alpha_{s+1})Y^{s-1} + \tau_0\alpha_{s+1}F,\quad s = 1,2,...,$$

$$\alpha_1 = 2,\quad \alpha_{s+1} = \frac{4}{4-\rho^2\alpha_s},\quad \rho = \frac{\mu-1}{\mu+1},\quad \mu = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}.$$

Можно объединить с методом Якоби: $D_A^{-1}AY = D_A^{-1}Y$. Число итераций $n(\varepsilon) = O(\sqrt{N}\ln[2/\varepsilon])$, нужно знать границы спектра, хорошо распараллеливается.

13) Схема сопряженных градиентов (**ССГ**):

$$R^0 = F - AY^0,\quad \forall s = 0,1,2,...:$$

$$B_s W^s = R^s;\quad \beta_0 = 0,\quad \beta_s = (W^s, R^s)/(W^{s-1}, R^{s-1});\quad G^0 = W^0,\quad G^s = W^s + \beta_s G^{s-1};$$

$$\alpha_s = (W^s, R^s)/(G^s, AG^s),\quad Y^{s+1} = Y^s + \alpha_s G^s,\quad R^{s+1} = F - AY^{s+1} = R^s - \alpha_s AG^s.$$

Если $B_s = B_s^* > 0$ – сходится за конечное число итераций при любом начальном приближении. Предлагается брать $B_s = (D^{-1} + A^-)D(D^{-1} + A^+)$. Тогда $n(\varepsilon) = O(\sqrt{N}\ln[2/\varepsilon])$ или $O(\sqrt[4]{N}\ln[2/\varepsilon])$. Треугольный, почти экономичный, не нужны границы спектра, трудно распараллелить.

***Современные методы на основе* ССГ**: различный выбор составляющих оператора B и упорядочения неизвестных.

(1) ССГ + МПТМ;
(2) ССГ + метод неполного разложения Холецкого (МНРХ) без заполнения => ICCG(0);
(3) ССГ + модифицированный МНРХ => MICCG(0);
(4) ССГ + метод симметричной верхней релаксации (МСВР) => SSORCG;
(5) ССГ + метод минимальных поправок;
(6) ССГ + метод минимальных невязок (GMRES).

Общая идея: A -> $A' = PAP^{-1}$ -> $B' = (D'^{-1} + A'_1)\ D'\ (D'^{-1} + A'_2)$ -> решаем задачу -> переупорядочиваем решение.

ICCG(0): $D = D_A$, $A_1 = A^-$, $A_2 = A^+$, $n(\varepsilon) = O(\sqrt{N}\ln[2/\varepsilon])$. В остальных методах $n(\varepsilon) = O(\sqrt[4]{N}\ln[2/\varepsilon])$.

MICCG(0): D выбираем из условия $Ae + \sigma D_A e = Be$, $e = (1,1,\ldots,1)^*$, $\sigma = O(N^{-1})$.

С ростом числа процессоров имеем асимптотику: $n(\varepsilon) = O(\sqrt{Np})$ или $O(\sqrt[4]{Np})$.

Альтернативный подход: Метод Шварца (МШ) разделения областей. Работает даже на дифференциальном уровне.

Задача $Au = f$ в $D = D_1 \bigcup D_2$. Тогда $u^0 = 0, \quad \forall k = 0,1,2,...$ решаем

$P_{11}(A)u_1^{k+1} + P_{12}(A)P_2(u^k) = P_1(f)$, в D_1, $\quad u_1^{k+1} = 0$ в D_2;

$P_{21}(A)P_1(u^k) + P_{22}(A)u_2^{k+1} = P_2(f)$, в D_2, $\quad u_2^{k+1} = 0$ в D_1;

$u^{k+1} = u_1^{k+1} + u_2^{k+1}$ в D.

Можно записать итерационную схему с параметрами, а области брать с налеганием. Число итераций $n(\varepsilon) = O(N \ln[2/\varepsilon])$. При увеличении области налегания скорость сходимости увеличивается.

6. Многомерное уравнение Пуассона

6.1. Прямые методы решения линейной алгебраической задачи.

Универсальные:

1) Ленточный метод Гаусса (ЛМГ),
2) Метод квадратного корня (МКК) или Метод Холецкого.

Специальные:

3) Дискретное преобразование Фурье (ДПФ),
4) Быстрое дискретное преобразование Фурье (БДПФ),
5) БДПФ по (m-1) направлениям + метод прогонки (МП) по одному из направлений.

Число операций: **ЛМГ, МКК**: $Q_1 = O(Nl^2)$, $N = \prod_{\alpha=1}^{m} N_\alpha$ – общее число неизвестных, $l = \prod_{\alpha=2}^{m} N_\alpha$ – полуширина ленты. **ДПФ**: $Q_1 = O(N^2)$. **БДПФ**: $Q_1 = O(N \log_2 N)$. **БДПФ+МП**: $Q_1 = O(N \log_2 N^{(m-1)/m})$.

Упорядочение: Джорж А., Лю Дж. Численное решение больших разреженных систем уравнений. М.: Мир, 1984; С. Писсанецки, Технология разреженных матриц. М.: Мир, 1988.

Параллельные варианты: **ЛМГ** и **МКК** – плохо распараллеливаются даже на МВС с общей памятью. **ДПФ** распараллеливается хорошо на МВС с общей памятью и похуже на МВС с распределенной памятью. **БДПФ** практически не распараллеливается. **БДПФ+МП** распараллеливается хорошо на МВС с общей и распределенной памятью за счет прогонки.

6.2. Итерационные методы решения линейной алгебраической задачи.

Многомерный МПН:

$B_s = \prod_{\alpha=1}^{m} (E + \tau_s^{(\alpha)} A_\alpha), \ \tau_s^{(\alpha)} > 0.$ Число итераций $n(\varepsilon) = O(\sqrt{h^{-1}} \ln[2/\varepsilon]) = O(\sqrt{N_\alpha} \ln[2/\varepsilon]) = O(\sqrt[2m]{N} \ln[2/\varepsilon])$ – с циклическим набором параметров.

Методы на основе ССГ: различный выбор составляющих оператора B и упорядочения неизвестных.

(1) ССГ + МПТМ;
(2) ССГ + метод неполного разложения Холецкого (МНРХ) без заполнения => ICCG(0);
(3) ССГ + модифицированный МНРХ => MICCG(0);
(4) ССГ + метод симметричной верхней релаксации (МСВР) => SSORCG;
(5) ССГ + метод минимальных поправок;
(6) ССГ + метод минимальных невязок (GMRES).

Все тяжело реализуются на МВС, если не применять приближенную факторизацию оператора B.

Число итераций: $n(\varepsilon) = O(\sqrt[\beta]{N} \ln[2/\varepsilon])$, $\beta = m, 2m$.

С ростом числа процессоров имеем асимптотику: $n(\varepsilon) = O\left(\sqrt[\beta]{Np}\right)$.

Метод Шварца: хорошо распараллеливается, хотя медленно сходится. Поэтому используется в комбинации с другими методами.

Лекция 13. Параллельные алгоритмы решения гиперболических уравнений.

1. Постановка начально-краевой задачи для одномерного уравнения переноса

$$\frac{\partial u}{\partial t}+a\frac{\partial u}{\partial x}=0,\quad t>0,\quad x>0,\quad a=const>0, \tag{1}$$

$$u(x,0)=u_0(x),\quad x\ge 0; \tag{2}$$

$$u(0,t)=\mu(t),\quad t>0. \tag{3}$$

Условие согласования данных: $u_0(0)=\mu(0)$, сами данные непрерывно-дифференцируемы.

Решение: $u(x,t)=u_0(x-at),\quad t\le x/a,\quad u(x,t)=\mu(t-x/a),\quad t\ge x/a.$

Зам. Если $x\in(-\infty,+\infty)=>u(x,t)=u_0(x-at)$ – решение задачи Коши.

2. Разностные схемы

В ограниченной расчетной области $D=\left[0,x_{\max}\right]\times\left[0,t_{\max}\right]$ вводим равномерную сетку $\Omega=\bar{\omega}_x\times\bar{\omega}_t$ с постоянными шагами h и τ. На этой сетке записываем разностную схему из следующего списка.

1) Схема с направленной разностью:

$y_t+ay_{\bar{x}}=0$ (операторная форма) или $\dfrac{y_i^{j+1}-y_i^j}{\tau}+a\dfrac{y_i^j-y_{i-1}^j}{h}=0$ (индексная форма),

погрешность аппроксимации схемы – $\Psi=O(\tau+h)$,

условие устойчивости (УУ) – $0<\gamma\le 1$, $\gamma=\dfrac{\tau a}{h}$ – число Куранта.

2) Схема с центральной разностью и коррекцией:

$y_t+ay_{\overset{\circ}{x}}-\dfrac{h^2}{2\tau}y_{\bar{x}x}=0$ или $\dfrac{y_i^{j+1}-y_i^j}{\tau}+a\dfrac{y_{i+1}^j-y_{i-1}^j}{2h}-\dfrac{h^2}{2\tau}\dfrac{y_{i+1}^j-2y_i^j+y_{i-1}^j}{h^2}=0$,

погрешность $\Psi=\begin{cases}O(\tau+h),\ если\ \tau=O(h),\\ \Psi=O(h^2),\ если\ \tau=h/|a|,\end{cases}$ УУ – $0<|\gamma|\le 1$.

3) Схема Лакса-Вендроффа второго порядка точности:

$y_t+ay_{\overset{\circ}{x}}-\dfrac{\tau a^2}{2}y_{\bar{x}x}=0$ или $\dfrac{y_i^{j+1}-y_i^j}{\tau}+a\dfrac{y_{i+1}^j-y_{i-1}^j}{2h}-\dfrac{\tau a^2}{2}\dfrac{y_{i+1}^j-2y_i^j+y_{i-1}^j}{h^2}=0$,

погрешность $\Psi=O(\tau^2+h^2)$, УУ $-0<|\gamma|\le 1$.

4) Схема с весами:

$y_t+a\left[\sigma\hat{y}_{\bar{x}}+(1-\sigma)y_{\bar{x}}\right]=0$ или $\dfrac{y_i^{j+1}-y_i^j}{\tau}+a\left[\sigma\dfrac{y_i^{j+1}-y_{i-1}^{j+1}}{h}+\left(1-\sigma\right)\dfrac{y_i^j-y_{i-1}^j}{h}\right]=0$, вес $\sigma\in\left[0,1\right]$.

Погрешность $\Psi=\begin{cases}O(\tau^2+h^2),\ если\ \sigma=\sigma_0=0.5\left(1-h/(\tau a)\right),\\ O(\tau+h),\ если\ \sigma\ne\sigma_0,\end{cases}$ УУ – $\sigma\ge\sigma_0$.

3. Общее пространственно одномерное линейное уравнение переноса

$$\frac{\partial u}{\partial t}+a(x,t)\frac{\partial u}{\partial x}=f(x,t),\quad t>0,\quad x>0. \tag{1'}$$

Рекомендуется использовать одну из следующих схем:

1) Схема Годунова (схема с направленными разностями):

$y_t+a^+y_{\bar{x}}+a^-y_x=f$ или $\dfrac{y_i^{j+1}-y_i^j}{\tau}+a_i^+\dfrac{y_i^j-y_{i-1}^j}{h}+a_i^-\dfrac{y_{i+1}^j-y_i^j}{h}=f_i^j$, где $a^{\pm}=0.5(a\pm|a|)$ – функции срезки.

Погрешность $\Psi=O(\tau+h)$, УУ – $0<\gamma_m\le 1$, $\gamma_m=\dfrac{\tau\cdot\max|a(x,t)|}{h}$.

2) Схема Лакса-Вендроффа:

$y_t+ay_{\overset{\circ}{x}}-\dfrac{\tau a^2}{2}y_{\bar{x}x}=f$ или $\dfrac{y_i^{j+1}-y_i^j}{\tau}+a\dfrac{y_{i+1}^j-y_{i-1}^j}{2h}-\dfrac{\tau a^2}{2}\dfrac{y_{i+1}^j-2y_i^j+y_{i-1}^j}{h^2}=f_i^j$.

Погрешность $\Psi=O(\tau^2+h^2)$, УУ – $0<\gamma_m\le 1$.

Два алгоритма решения и распараллеливания:

1) Решение задачи и деление расчетной области D и сетки Ω между параллельными процессами ***вдоль характеристик*** (рис. 1);

2) Решение задачи ***по слоям по времени*** и деление расчетной области D и сетки Ω между параллельными процессами по пространственной координате (рис. 2).

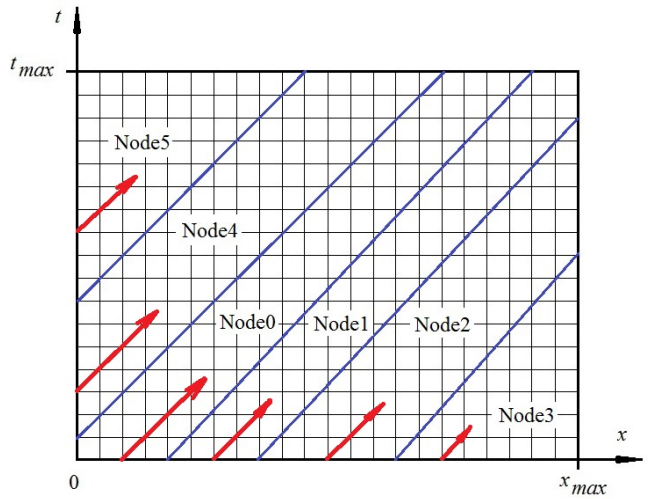


Рис. 1. Распределение узлов сетки в Алгоритме 1.

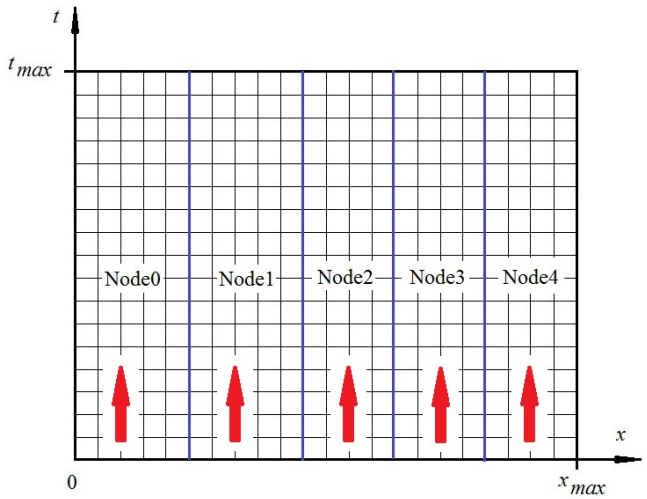


Рис. 2. Распределение узлов сетки в Алгоритме 2.

Алгоритм 1 подразумевает, что решение рассчитывается вдоль по каждой характеристики, начинающейся либо на оси x, либо на оси t. Поскольку алгоритм явный, порядок вычислений должен быть таким, чтобы для каждого нового искомого значения имелись все данные в рамках шаблона разностной схемы. Алгоритм 1 наиболее удобен для случая $a = const$ или малых изменений функции $a(x,t)$. В любом случае требуется, чтобы сетка была характеристической во всей области, а сами характеристики не пересекались. При этом криволинейность сетки допускается. Балансировка загрузки вычислителей достигается путем выполнения критерия примерного равенства количества узлов сетки, обрабатываемых каждым вычислителем.

Алгоритм 2 более универсален. Он не требует, чтобы сетка отслеживала какие-либо изменения функции $a(x,t)$. Достаточно, чтобы исходная задача была разрешима в D и погрешность аппроксимации отвечала заданной точности. При параллельных вычислениях их порядок должен быть таким, чтобы для каждого нового искомого значения также имелись все данные в рамках шаблона разностной схемы. Балансировка загрузки вычислителей также достигается путем выполнения критерия примерного равенства количества узлов сетки, обрабатываемых каждым вычислителем.

4. Многомерное линейное уравнение переноса

В многомерном случае чаще всего рассматривается следующий вариант уравнения переноса:

$$\frac{\partial u}{\partial t} + \left(\vec{a}(\vec{x},t), \nabla u\right) = f(\vec{x},t), \quad t > 0, \quad \vec{x} = (x_1, ..., x_m) \in \mathbb{R}^+. \tag{4}$$

Здесь $\mathbb{R}^+$ – положительное полупространство пространства $\mathbb{R}^m$. Начальные и граничные условия аналогичны (2), (3). Количество последних соответствует размерности пространства m. Как и выше, расчетная область выбирается ограниченной: $D = \left(\prod_{k=1}^{m}\left[0, x_{k,\max}\right]\right) \times \left[0, t_{\max}\right]$.

4.1. Случай постоянного вектора $\vec{a}$.

В этом случае задачу можно свести к поиску пучка локально-одномерных решений, распространяющихся параллельно в области D и берущих свое начало из какой либо точки начальных или граничных данных. Уравнение преобразуется в локально-одномерное:

$$\frac{\partial u}{\partial t} + \frac{\partial u}{\partial \xi} = f(\vec{x}(\xi), t), \quad t > 0, \quad \xi > 0; \quad \vec{x}(\xi) = \vec{x}(0) + \vec{a} \cdot \xi.$$

Здесь ξ – координата вдоль направления вектора $\vec{a}$, исходящего из некоторой граничной точки $M = (x_1, ..., x_m, t)$, в которой одна из координат или время равны 0. При параллельной реализации

используем обобщение Алгоритма 1 и распределяем между вычислителями пучок лучей, исходящих из разных точек M.

4.2. Случай переменного вектора $\vec{a}$.

В этом случае используются разностные схемы, обобщающие схемы Годунова и Лакса-Вендроффа.

1) Схема Годунова: $y_t + \sum_{k=1}^{m}\left(a_k^+ y_{\bar{x}_k} + a_k^- y_{x_k}\right) = f$.

Погрешность $\Psi = O(\tau + |\vec{h}|)$, $\vec{h} = (h_1,...,h_m)$ – вектор шагов.

Условие устойчивости – $0 < \gamma_m \le 1$, $\gamma_m = \dfrac{\tau \cdot \max|a_k(\vec{x},t)|}{|\vec{h}|}$.

2) Схема Лакса-Вендроффа: $y_t + \sum_{k=1}^{m}\left(a_k y_{\overset{\circ}{x}_k} - \dfrac{\tau a_k^2}{2} y_{\bar{x}_k x_k}\right) = f$.

Погрешность $\Psi = O(\tau^2 + |\vec{h}|^2)$, УУ – $0 < \gamma_m \le 1$.

5. Пространственно-одномерное уравнение колебаний струны

В начале рассмотрим простейший вариант уравнения:

$$\frac{\partial^2 u}{\partial t^2} = a^2 \frac{\partial^2 u}{\partial x^2} + f(x,t), \quad t > 0, \quad x \in (0,1); \tag{5}$$

Начальные условия:

$$u(x,0) = u_0(x), \quad u_t(x,0) = u_1(x), \quad x \in [0,1]; \tag{6}$$

Граничные условия можно выбирать любые, например:

$$u(0,t) = \mu_0(t), \quad u(1,t) = \mu_1(t), \quad t > 0. \tag{7}$$

Для решения задачи методом конечных разностей вводим в области $D = [0,1] \times [0, t_{\max}]$ сетку $\Omega = \bar{\omega}_x \times \bar{\omega}_t$ для примера с постоянными шагами h и τ, количествами узлов сетки N_x и N_t.

На Ω запишем разностную схему с весами:

$$y_{\bar{t}t} = a^2 \Lambda_h\left(\sigma \hat{y} + (1-2\sigma) y + \sigma \check{y}\right) + f, \quad \Lambda_h y \equiv y_{\bar{x}x}, \text{ или}$$

$$\frac{y_i^{j+1} - 2y_i^j + y_i^{j-1}}{\tau^2} = \sigma\left[a^2 \Lambda_h y_i^{j+1} + f_i^{j+1}\right] + (1-2\sigma)\left[a^2 \Lambda_h y_i^j + f_i^j\right] +$$

$$+\sigma\left[a^2 \Lambda_h y_i^{j-1} + f_i^{j-1}\right], \quad \Lambda_h y_i^j \equiv \frac{y_{i+1}^j - 2y_i^j + y_{i-1}^j}{h^2}, \quad i = 1,...,N_x, \quad j = 2,...,N_t; \tag{5h}$$

$$y(x,0) = u_0(x), \quad y_t(x,0) = u_1(x), \text{ или } y_i^0 = u_0(x_i), \quad \frac{y_i^1 - y_i^0}{\tau} = u_1(x_i), \quad i = 0,...,N_x; \tag{6h}$$

$$y(0,t) = \mu_0(t), \quad y(1,t) = \mu_1(t), \text{ или } y_0^j = \mu_0(t_j), \quad y_{N_x}^j = \mu_1(t_j), \quad j = 1,...,N_t. \tag{7h}$$

Вес $\sigma \in [0,1]$. Чаще всего рассматриваются явная ($\sigma = 0$), неявная ($\sigma = 1$) и симметричная ($\sigma = 0.5$) схемы. Погрешность аппроксимации схемы $\Psi = O(\tau^2 + h^2)$ при любом значении σ.

Условия устойчивости:

1) $\sigma = 0$: $0 < \gamma \le 1$, $\gamma = \dfrac{\tau a}{h}$;

2) $\sigma \ne 0$: $\sigma \ge \sigma_\varepsilon - 1/4\gamma^2$, $\sigma_\varepsilon = 1/4(1-\varepsilon)$, $\varepsilon > 0$.

Явная схема реализуется послойно по времени с помощью пересчета решения на новом слое через значения двух предыдущих слоев. Ее параллельная реализация основана на разбиении пространственной сетки на смежные подмножества одинаковой мощности и применении Алгоритма 2.

Остальные схемы ($\sigma \ne 0$) реализуются с помощью алгоритма прогонки, параллельная реализация основана на алгоритме параллельной прогонки (см. Лекцию 10).

6. Случай произвольного пространственного оператора

Обобщение уравнения (5) происходит при использовании произвольного пространственного оператора:

$$\frac{\partial^2 u}{\partial t^2} = Lu + f, \quad Lu = \frac{\partial}{\partial x}\left(k\frac{\partial u}{\partial x} + r_0 u\right) + r_1\frac{\partial u}{\partial x} - qu, \quad t > 0, \quad x \in (0,1). \tag{5'}$$

Это уравнение также дополняется начальными условиями (6) и граничными условиями (7), которые могут быть трех основных типов (см. Лекцию 10):

$$\eta_m u(x_m) + \theta_m \frac{\partial u}{\partial x}(x_m) = \mu_m,\ \eta_m^2 + \theta_m^2 \neq 0,\ m = 0,\ 1, \tag{7_1}$$

$$u(0) = u(1),\ \frac{\partial u}{\partial x}(0) = \frac{\partial u}{\partial x}(1), \tag{7_2}$$

$$\Phi_0(u) = c_0,\ \Phi_1(u) = c_1. \tag{7_3}$$

Для решения задачи методом конечных разностей используется схема с весами, аналогичная (5_h).

Погрешность аппроксимации $\Psi = O(\tau^2 + h^2)$ при любом значении $\sigma \in [0,1]$.

Условия устойчивости можно получить, если привести схему (5_h) к виду: $By_{\overset{\circ}{t}} + \tau^2 R y_{\bar{t}t} + Ay = f$.

Тогда, если будут выполнены условия: $A = A^* > 0, R = R^* > 0, B \geq 0, R > A/4$, схема будет устойчива.

В упрощенном виде эти условия приведут к условию Куранта: $0 < \frac{\tau a}{h} \leq 1$ с некоторой константой a.

Параллельная реализация схемы аналогична указанной в п. 5.

7. Многомерное уравнение гиперболического типа

$$\frac{\partial^2 u}{\partial t^2} = Lu + f, \quad Lu \equiv \sum_{\alpha=1}^{m}\left[\frac{\partial}{\partial x_\alpha}\left(\sum_{\beta=1}^{m} k_{\alpha\beta}\frac{\partial u}{\partial x_\beta} + r_{0,\alpha}u\right) + r_{1,\alpha}\frac{\partial u}{\partial x_\alpha}\right] - qu, \quad t > 0, \quad \vec{x} \in D \subset \mathbb{R}^m. \tag{5''}$$

Начальные условия аналогичны (6). Граничные условия аналогичны (7_1), (7_2), (7_3).

Для решения многомерной задачи можем использовать:

1) схему с весами и прямые или итерационные методы для расчета решения на каждом новом слое по времени (однако чаще всего здесь используют явную схему по времени, так как условие устойчивости будет типа условия Куранта: $0 < \frac{\tau a}{|\vec{h}|} \leq 1$);

2) схемы расщепления, например, локально-одномерные схемы и комбинацию прогонок для расчета решения на каждом новом слое по времени.

По сути этот случай аналогичен по методике решению многомерного уравнения теплопроводности. Поэтому методики распараллеливания те же (см. Лекцию 11).

Лекция 14. Решение многомерных уравнений в частных производных общего вида.

1. Постановки модельных задач.

$$Lu \equiv \sum_{\alpha=1}^{m} \frac{\partial}{\partial x_\alpha}\left(\sum_{\beta=1}^{m}\left(k_{\alpha\beta}\frac{\partial u}{\partial x_\beta} + r_{0,\alpha\beta}u\right)\right) + \sum_{\alpha=1}^{m}\left(r_{1,\alpha}\frac{\partial u}{\partial x_\alpha}\right) - qu\,, \quad \vec{x} \in D\,, \tag{1}$$

$$\frac{\partial^k u}{\partial t^k} = Lu + f \quad (k = 0,1,2), \quad \vec{x} \in D\,, \quad t > 0\,, \tag{2}$$

Начальные условия:

$$u|_{t=0} = u_0(\vec{x}), \quad \vec{x} \in D. \tag{3}$$

$$u|_{t=0} = u_0(\vec{x}), \quad \left.\frac{\partial u}{\partial t}\right|_{t=0} = u_1(\vec{x}), \quad \vec{x} \in D. \tag{3'}$$

Варианты граничных условий:

1) условия Дирихле: $u = g$, $\vec{x} \in \partial D$;
2) условия Неймана: $\partial u / \partial n = 0$, $\vec{x} \in \partial D$;
3) условия третьего рода: $\partial u / \partial n = -\eta u$, $\vec{x} \in \partial D$;
4) условия смешанного типа: $g_1 u + g_2 \partial u / \partial n = g_3$, $\vec{x} \in \partial D$;
5) периодические условия;
6) интегральные условия.

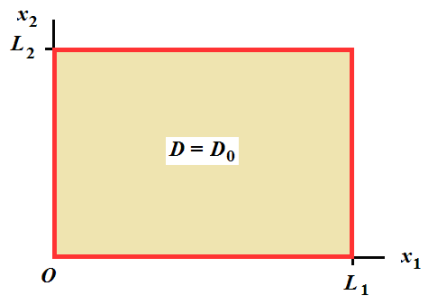
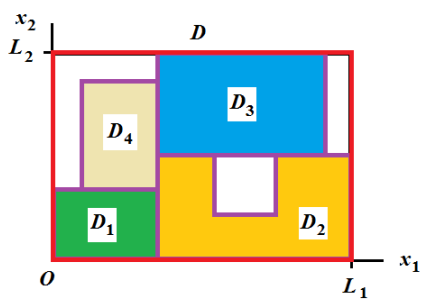
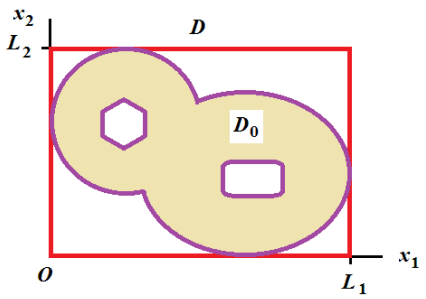
Проблема решения задачи Неймана в стационарном случае.

Координаты: декартовы, цилиндрические, сферические, обобщенные.

Варианты задачи: линейная, квазилинейная, нелинейная.

2. Численные методы

Любой численный алгоритм существенно зависит от формы области, в которой решается задача.

		
Прямоугольная область	Область, составленная из прямоугольников	Область произвольной криволинейной формы

1) Численные методы в случае прямоугольной области:

Сеточные методы: метод конечных разностей (МКР) и метод конечных элементов (МКЭ).

$$\Lambda_h y_h = -\varphi_h\,, \quad \Lambda_h y_h \equiv \sum_{\alpha=1}^{m} \Lambda_{h,\alpha} y_h\,, \quad \Lambda_{h,\alpha} y_h = (\bar{k}_\alpha y_{x_\alpha})_{\bar{x}_\alpha}\,, \quad \vec{x} \in \Omega \tag{4}$$

Аппроксимация по времени: явные и неявные схемы.

$$\frac{\hat{y}_h - y_h}{\tau} = \sigma\left[\hat{\Lambda}_h \hat{y}_h + \hat{\varphi}_h\right] + (1-\sigma)\left[\Lambda_h y_h + \varphi_h\right], \quad \vec{x} \in \Omega\,, \quad t > 0\,. \tag{5}$$

$$\frac{\hat{y}_h - 2y_h + \breve{y}_h}{\tau^2} = \sigma\left[\hat{\Lambda}_h \hat{y}_h + \hat{\varphi}_h\right] + (1-2\sigma)\left[\Lambda_h y_h + \varphi_h\right] + \sigma\left[\breve{\Lambda}_h \breve{y}_h + \breve{\varphi}_h\right], \quad \vec{x} \in \Omega\,, \quad t > 0\,. \tag{6}$$

Неявные схемы: прямые методы или итерационные.

2) Численные методы в областях, составленных из прямоугольников и параллелепипедов:

а) Метод нерегулярных шаблонов;
б) Метод фиктивных областей.

3) Численные методы в областях произвольной формы.

а) Метод нерегулярных шаблонов;
б) Метод фиктивных областей;
в) Нерегулярные сетки.

2.1. Метод нерегулярных шаблонов (МНШ)

Этапы метода:

- построение ***минимального окаймляющего параллелепипеда*** D, содержащего объединение $D_0 = \bigcup_l D_l$;
- построение неравномерной сетки $\Omega = \prod_{k=1}^{m} \bar{\omega}_{\hat{x}_k}$ в D ($\bar{\omega}_{\hat{x}_k}$ – одномерная неравномерная сетка по координате x_k, содержащая все ***опорные точки*** областей D_l);
- запись разностной схемы либо в узлах, либо в центрах ячеек, которые входят в область D_0;
- формирование итоговой системы алгебраических уравнений (AY=F) и выбор метода ее решения (прямой или итерационный);
- разбиение вектора неизвестных Y на фрагменты Y_k (k=0,...,p-1, p – количество вычислителей) и распределение их по вычислителям;
- вычисление решения.

2.2. Метод фиктивных областей (МФО)

Этапы метода:

- построение ***минимального окаймляющего параллелепипеда*** D, содержащего объединение $D_0 = \bigcup_l D_l$;
- построение неравномерной сетки $\Omega = \prod_{k=1}^{m} \bar{\omega}_{\hat{x}_k}$ в D ($\bar{\omega}_{\hat{x}_k}$ – одномерная неравномерная сетка по координате x_k, содержащая все ***опорные точки*** областей D_l);
- дополнение уравнения вне области D_0:

$$\sum_{\alpha=1}^{m} \frac{\partial}{\partial x_\alpha}\left(\tilde{k}_{\alpha\alpha}\frac{\partial u}{\partial x_\alpha}\right) - \tilde{q}u = 0, \quad \vec{x} \in D / D_0,$$

и добавление дополнительных условий (Дирихле или Неймана) на соответствующих участках границы;
- запись разностной схемы либо в узлах, либо в центрах ячеек;
- формирование итоговой системы алгебраических уравнений (AY=F) и выбор метода ее решения (прямой или итерационный);
- разбиение вектора неизвестных Y на фрагменты Y_k (k=0,...,p-1, p – количество вычислителей) и распределение их по вычислителям;
- вычисление решения.

2.3. Метод нерегулярных сеток (МНС)

Этапы метода:

- построение объединения $D_0 = \bigcup_l D_l$;
- построение нерегулярной сетки Ω (симплексной – треугольники, тетраэдры, многогранники с плоскими гранями с числом вершин m+1, m – размерность пространства) в D_0;
- запись конечно-объемной схемы (метод конечных объемов, МКО) либо в узлах, либо в центрах ячеек, или конечно-элементной схемы (метод конечных элементов, МКЭ);
- формирование итоговой системы алгебраических уравнений (AY=F) и выбор метода ее решения (прямой или итерационный);
- разбиение вектора неизвестных Y на фрагменты Y_k (k=0,...,p-1, p – количество вычислителей) и распределение их по вычислителям;
- вычисление решения.

2.4. Сравнение методов

МНШ – универсальный, имеются лишние вычисления вследствие использования неравномерных сеток, использует стандартные методики обращения матриц, точность соответствует аппроксимации, трудно делать разбиение по вычислителям, неоднородные вычисления, нужна балансировка загрузки вычислителей.

МФО – специальный, имеются лишние вычисления вследствие использования фиктивных областей, использует стандартные методики обращения матриц, точность зависит от аппроксимации и выбора

фиктивных уравнений, легко делать разбиение по вычислителям, нужна балансировка загрузки вычислителей.
МНС – универсальный, нет лишних вычислений, использует оригинальные методики обращения матриц, точность соответствует аппроксимации, трудно делать разбиение по вычислителям, неоднородные вычисления, нужна балансировка загрузки вычислителей.

3. Методы решения квазилинейной алгебраической задачи.
В результате построения численной схемы получаем алгебраическую задачу вида $AY = F$, которая может оказаться симметричной или несимметричной, линейной или нелинейной.
Для решения в нелинейном случае применяем итерации по нелинейности:

МПИ: $\overset{0}{Y}, \quad \forall s = 0,1,2,\ldots: \quad A\left(\overset{s}{Y}\right)\overset{s+1}{Y} = F\left(\overset{s}{Y}\right).$ (7)

МН: $\overset{0}{Y}, \quad \forall s = 0,1,2,\ldots: \quad A\left(\overset{s}{Y}\right)\overset{s+1}{Y} + \theta\left(\frac{\partial A}{\partial Y}\left(\overset{s}{Y}\right)\overset{s}{Y} - \frac{\partial F}{\partial Y}\left(\overset{s}{Y}\right)\right)\left(\overset{s+1}{Y} - \overset{s}{Y}\right) = F\left(\overset{s}{Y}\right).$ (8)

Можно использовать метод Ньютона общий и стационарный, параметр $0 < \theta \le 1$.
При распараллеливании важную роль играет метод реализации итераций, а также характер нелинейности: локальная и нелокальная.

4. Проблемы решения реальных задач на суперкомпьютерах.
4.1. Решение задач в областях сложной формы.
1) Построение нерегулярных сеток.
2) Построение схем МКО, МКЭ или гибридов.
3) Методы решения получаемых алгебраических уравнений.
4) Проблемы распараллеливания
4.2. Неоднородность задач (на примере многослойных областей, где пишутся различные системы уравнений).
4.3. Нелокальная нелинейность (в операторном виде).
4.4. Изменение размерности вектора неизвестных N=N(t) (на примере задач с движущимися особенностями и границами).
4.5. Как решение многих проблем - разбиение графов с учетом весов ребер и вершин.
Спектральный алгоритм. Иерархический алгоритм. Геометрический алгоритм.

Лекция 15. Решение спектральных задач на МВС.

1. Линейные спектральные задачи в конечномерных пространствах:

$Ax = \lambda x, \quad x \neq \theta$. Пусть $A \in \mathbb{R}^{N\times N}, \mathbb{C}^{N\times N}$.

Методы решения:

1) $\det|A - \lambda I| = 0$ – уравнение для поиска собственных значений. Используется при малой размерности матрицы.

2) частные Рэлея: $\inf\limits_{x\neq\theta}\frac{(Ax,x)}{(x,x)}$, $\sup\limits_{x\neq\theta}\frac{(Ax,x)}{(x,x)}$, $\inf\limits_{x\neq\theta}\frac{|(Ax,x)|}{(x,x)}$, $\sup\limits_{x\neq\theta}\frac{|(Ax,x)|}{(x,x)}$. и $\sup\limits_{x\neq\theta}\frac{(Ax,x)}{(x,x)}$ – для оценки спектра.

3) Поиск спектра и собственных векторов для диагональных и треугольных матриц тривиален.

Преобразование подобия $B = FAF^{-1}$, $\det F \neq 0$ не меняет спектра матрицы: $Ax = \lambda x$ <=> $By = \lambda y$.

Чтобы не вносить дополнительные ошибки округления, F должна быть унитарной: $F = U$, $U^{-1} = U^*$.

Унитарное преобразование сохраняет эрмитовость матриц: $A = A^*$ => $B = B^*$. На основе этих идей созданы методы ***отражений (Хаусхолдера), Гивенса, Якоби***.

4) Метод ***обратных итераций со сдвигом*** для поиска пары (λ, ξ).

Будем предполагать, что собственные векторы $\boldsymbol{x}_m$, $1 \le m \le N$ матрицы A, соответствующие собственным значениям λ_m, образуют базис, а искомое собственное значение λ_M является простым. Пусть известно хорошее приближение $\tilde{\lambda} \approx \lambda_M$ и $\det|A - \tilde{\lambda} I| \neq 0$. Выберем вектор $\boldsymbol{x}^{(0)}$ и $\lambda^{(0)} = \tilde{\lambda}$ и используем итерации:

$$\left(A - \lambda^{(s)} I\right)\boldsymbol{x}^{(s+1)} = \boldsymbol{x}^{(s)}, \quad \lambda^{(s+1)} = \lambda^{(s)} + \left(\boldsymbol{x}^{(s+1)}, \boldsymbol{x}^{(s)}\right) \Big/ \left(\boldsymbol{x}^{(s+1)}, \boldsymbol{x}^{(s+1)}\right), \quad \boldsymbol{x}^{(s+1)} = \boldsymbol{x}^{(s+1)} / \left\|\boldsymbol{x}^{(s+1)}\right\|, \quad s = 0,1,2,....$$

Сходимость итераций квадратичная, а для ортогонального базиса – кубическая.

5) Процедура ортогонализации и понижение размерности задач (см. Голуб «Матричные вычисления»).

6) QR и QZ – алгоритмы для решения полной задачи в вещественном и комплексном случаях.

2. Обобщенные линейные спектральные задачи в конечномерных пространствах:

$Ax = \mu Bx, \quad x \neq \theta$.

Если применить указанные выше методы для матрицы $C = B^{-1}A$, а затем перейти в них к A, то получим обобщенные методы решения.

3. Нелинейные по спектральному параметру задачи в конечномерных пространствах:

$A(\lambda)x = 0, \quad x \neq \theta$.

Пример: $Ax - P(\lambda)x = 0, \quad x \neq \theta, \quad P(\lambda) = P_m(\lambda) = \sum\limits_{k=0}^{m} C_k \lambda^{m-k}$, C_k – матрицы.

1) Для поиска отдельных пар (λ, x) используется метод обратных итераций:

$\lambda^0, \lambda^1, x^0$ – два начальных приближения к собственному значению и одно к собственному вектору.

$$\forall\, n = 1,2,3,...: \quad \frac{\partial A}{\partial \lambda}\left(\lambda^n\right) \approx \frac{A(\lambda^n) - A(\lambda^{n-1})}{(\lambda^n - \lambda^{n-1})}, \quad x^{n,0} = x^{n-1}, \quad A(\lambda^n) y^{n,k+1} = -\frac{\partial A}{\partial \lambda}\left(\lambda^n\right) x^{n,k}, \quad x^{n,k+1} = \frac{y^{n,k+1}}{\left\|y^{n,k+1}\right\|},$$

$$(\delta\lambda)^{n,k+1} = \left(\frac{\partial A}{\partial \lambda}\left(\lambda^n\right) x^{n,k}, \left[\frac{\partial A}{\partial \lambda}\left(\lambda^n\right) y^{n,k+1}\right]^*\right)\left(\frac{\partial A}{\partial \lambda}\left(\lambda^n\right) y^{n,k+1}, \left[\frac{\partial A}{\partial \lambda}\left(\lambda^n\right) y^{n,k+1}\right]^*\right)^{-1}, \quad k = 0,1,...,k_0 - 1,$$

$$\lambda^{n+1} = \lambda^n + (\delta\lambda)^{n,k_0}, \quad x^{n+1} = x^{n,k_0}.$$

Фактически это комбинация метода обратных итераций для линейной задачи и метода Ньютона.

2) Метод Дэвиса – поиск всех нулей аналитической функции $f(\lambda) = \det A(\lambda)$ на комплексной плоскости. Определитель вычисляется с помощью LU разложения. Задача сводится к поиску корней полинома степени N, коэффициенты которого можно вычислить по рекуррентным формулам.

3) Комбинация методов позволяет решить полную задачу.

4. Нелинейные спектральные задачи в конечномерных пространствах:

$F(x,\lambda) = \theta, \quad F(\theta,\lambda) = \theta, \quad x \neq \theta$.

Обобщение предыдущего алгоритма в случае конечномерного пространства.

5. Спектральные задачи в бесконечномерных пространствах.

5.1. Примеры задач.

5.1.1. Задачи для линейного дифференциального уравнения 2го порядка с одной переменной

Простой случай:

$$u''+\lambda u=0,\quad 0<x<1;$$

$$u(0)=u(1)=0.$$

Общий случай:

$$Lu\equiv\frac{d}{dx}\left(k\frac{du}{dx}+r_0u\right)+r_1\frac{du}{dx}-qu=-\lambda u,\quad 0<x<1.$$

Варианты граничных условий приведены в лекции 10.

5.1.2. Задачи для линейного дифференциального уравнения 2го порядка со многими переменными

Простой случай:

$$\frac{\partial}{\partial x_1}\left(k_1\frac{\partial u}{\partial x_1}\right)+\frac{\partial}{\partial x_2}\left(k_2\frac{\partial u}{\partial x_2}\right)=-\lambda u,\quad \vec{x}\in D=\{(x_1,x_2):0<x_\alpha<1,\alpha=1,2\}.$$

Общий случай:

$$Lu\equiv\sum_{\alpha=1}^{m}\frac{\partial}{\partial x_\alpha}\left(\sum_{\beta=1}^{m}\left(k_{\alpha\beta}\frac{\partial u}{\partial x_\beta}+r_{0,\alpha\beta}u\right)\right)+\sum_{\alpha=1}^{m}\left(r_{1,\alpha}\frac{\partial u}{\partial x_\alpha}\right)-qu=-\lambda u,\quad \vec{x}\in D.$$

Варианты граничных условий приведены в лекции 14.

5.1.3 Задачи для интегральных уравнений

Простой случай:

$$\int_0^1 k(x,s)u(s)ds=\lambda u(x),\quad 0<x<1;$$

$$u(0)=u(1)=0.$$

Общий случай:

$$\int_D k(\mathbf{x},\mathbf{s})u(\mathbf{s})d\mathbf{s}+Lu(\mathbf{x})=-\lambda u(\mathbf{x}),\quad \mathbf{x}\in D.$$

Здесь L – алгебраический или дифференциальный оператор.

Варианты дополнительных условий приведены в лекции 14.

5.2. Методы решения

Методы решения спектральных задач в бесконечномерном пространстве связаны с дискретизацией и переходом к конечным базисам собственных функций.

Лекция 16. Решение экстремальных задач на МВС.

1. Постановки задачи.

Поиск минимумов и максимумов функций и функционалов.

1.1. Экстремум функции многих переменных.

$$f(x_1,...,x_m) \to Gf = 0$$

$$\Phi(u) \to \min, \quad u \in F(\Omega), \quad \Omega \subset \mathbb{R}^m$$

1.2. Минимум функционала.

2. Методы решения:

а) градиентные; $u^{s+1} = u^s - \alpha\left[\frac{\partial\Phi}{\partial u}(u^s)\right](u^{s+1} - u^s), \quad u^0, \quad s = 0,1,...$

МРЭЗ, ЧМРЭЗ - Васильев Ф.П.

б) координатные; $Gf(x_1,...,x_k,...,x_m) = 0, \quad x_k \in D \subset \mathbb{R}^1$

в) стохастические; $\min\left\|\mathbf{x}^* - \mathbf{x}^s\right\| \le C / \sqrt{M}$

г) гибридные.

3. Обобщение метода бисекции (метод Евтушенко).

см. метод неравномерных покрытий в Интернет

4. Метод кривых Пеано.

Применяется для задач умеренной размерности.

см. работы Гергель и др.

5. Метод Монте-Карло.

$l-\pi-\tau$ последовательности (автор проф. Соболь И.М.)

(http://bigor.bmstu.ru/?cnt/?doc=Paransys/par2_3.mod/?cou=Paransys/base.cou)

Алгоритм формирования LPTau последовательности

ЛПТау-последовательность является примером равномерно распределенной последовательности. Ее полное название - последовательность, любой двоичный участок которой представляет собой $\mathbf{\Pi}_\tau$-сетку. Приведем исходный алгоритм

1. Имеется таблица числителей направляющих чисел $\mathbf{r}_j^{(1)}$, $1 \le j \le 51$, $1 \le l \le 20$

2. Направляющие числа рассчитываются по формуле $\mathbf{V}_j^{(1)} = \mathbf{r}_j^{(1)}\ 2^{-l}$

3. В двоичной системе номер точки i записывается в форме $i=e_m...e_2e_1$, а декартовы координаты точки $Q_i=(q_{i,1},...,q_{i,n})$ вычисляются по единой формуле (j=1,2,..n):

$$q_{i,j}=e_1\mathbf{V}_j^{(1)} *e_2\mathbf{V}_j^{(2)} *...*e_m\mathbf{V}_j^{(m)} \quad (1)$$

где * означает поразрядное сложение по модулю 2 в двоичной системе.

Арифметический алгоритм, реализованный на ЭВМ выглядит так

1. По заданному номеру i вычисляем m=1+[ln i/ ln 2]

2. Для j=1,2,...n вычисляем

$$q_{i,j}=\sum_{k=1}^{m} 2^{-k+1}\left\{\frac{1}{2}\sum_{l=k}^{m}[2\{i2^{-l}\}][2\{r_j^{(l)}2^{k-l-1}\}]\right\} \quad (2)$$

причем использовано обозначение [z] - целая часть числа, {z}- дробная часть числа z.

Алгоритм нахождения точек равномерно распределенной последовательности в произвольной области по известному распределению в единичном многомерном кубе.

Для *нахождения точек ЛПтау последовательности в произвольной области* используем следующие Леммы

Лемма 1 . Если точки $\mathbf{Q}_i$ с декартовыми координатами ($\mathbf{q}_{i,1},,...\mathbf{q}_{i,n}$) образуют равномерно распределенную последовательность в $\mathbf{K}^n$, то точки $\mathbf{A}_i$ с декартовыми координатами ($\alpha_{i,1},,...\alpha_{i,n}$) , где при j=1,2...n:

$\alpha_{i,j}=a_j+(b_j-a_j)q_{i,j}$ (9)

образуют равномерно распределенную последовательность в параллелепипеде Π, состоящем из точек $(\alpha_1,...\alpha_n)$, координаты которых удовлетворяют неравенствам $a_j \le \alpha_j \le b_j$

Лемма 2 . Пусть $A_1,...,$ A_i - последовательность точек , равномерно распределенных в Π, а $G \subset \Pi$ - произвольная область с положительным объемом $V_G > 0$. Если среди точек A_i отобрать все точки, принадлежащие G, то получим последовательность точек , равномерно распределенных в G.

Использование метода ЛПТау последовательности

Точки ЛПТау - последовательности в простейшем случае (один критерий) можно использовать как пробные в случайном поиске. Выбираем N точек, равномерно распределенных в рассматриваемой области, вычисляем значения функционала и находим точку , где он принимает минимальное значение.

Однако особый интерес использование этих последовательностей имеет в двух классах задач.

1. Задачи, в которых требуется одновременно оценить максимумы и (или) минимумы нескольких функций (это можно сделать по одним и тем же пробным точкам).

2. Задачи, в которых для отыскания глобального экстремума многоэкстремальной функции используют локальные методы оптимизации. При этом, чтобы не попасть вместо глобального экстремума в какой- либо из локальных, приходится повторять локальный поиск много раз, начиная с различных начальных точек. Очевидно, что начальные точки должны быть равномерно расположены. Самым эффективным способом выбора начальных точек оказалось использование точек ЛПТау - последовательности.